

OneTwoPAKE: Two-Round Strong Asymmetric PAKE with Ideal Security

Yashvanth Kondi¹, Ian McQuoid, Kelsey Melissaris^{2*}, Claudio Orlandi³, Lawrence Roy³, and LaKyah Tyner⁴

¹ Silence Laboratories, Singapore

² Chalmers University of Technology, Göteborg, Sweden

³ Aarhus University, Aarhus, Denmark

⁴ Northeastern University, Boston, Massachusetts, USA

yash@ykondi.net, mcquoidi@oregonstate.edu, kelseym@chalmers.se
{orlandi,lance.roy}@cs.au.dk, tyner.l@northeastern.edu

Abstract. Strong Asymmetric Password-Authenticated Key Exchange (saPAKE) enables a client, holding only a low-entropy password, to repeatedly establish shared high-entropy session keys with a server holding a *digest* of the expected password. Integrally, the only online attacks afforded to the adversary are those inevitable impersonation and dictionary attacks. As opposed to previous modeling, saPAKE additionally requires that any offline password search against the server’s storage takes place *after* adaptive server compromise.

We present *OneTwoPAKE*, the first saPAKE protocol to simultaneously:

1. realize the full (*unweakened*) strong aPAKE functionality;
2. not admit a speedup in an offline password search; (*aka, has simulation-rate of 1*);
3. use only a single round trip, with the client speaking first; and
4. avoid generic algebraic models.

Similar to prior work, we instantiate our saPAKE from an OPRF over insecure channels secure against adaptive server compromise. In contrast to prior work, our OPRF is *online-extractable* and *input-committing*, enabling our protocol to realize the full saPAKE functionality.

Of independent interest are our OPRF functionality and construction. We introduce the first formal model of such an OPRF, and our OPRF protocol is the first Dodis-Yampolskiy-based OPRF proven UC-secure against malicious adversaries without authenticated channels.

Our framework demonstrates the feasibility of achieving all of the above properties simultaneously. Though our constructions are not as efficient as those of prior work, our saPAKE boasts the minimal round complexity, achieves full security, and, in terms of idealized models, relies *only on the random oracle model*. As future work may further close the efficiency gap, our framework may lead to practically deployable solutions.

1 Introduction

Password-authenticated key exchange (PAKE) is a broad class of protocols in which two parties can generate a shared key exactly when they hold the same password. Since passwords are often low-entropy secrets, an attacker—that wishes to impersonate either party—can usually *guess* the session password and learn the session key with non-negligible probability. A secure PAKE should guarantee that an adversary can perform *only* this online dictionary attack. Furthermore, the attacker should be restricted to a single password guess per party per session.

In most common settings, only the clients have their passwords *in the clear*, and servers store a salted hash of the client’s password. Asymmetric PAKE (aPAKE) [8] allows the parties to generate shared keys in this setting. Ideally, a secure aPAKE should ensure that an adversary that *adaptively compromises* the server, thereby learning the hash of the password, would still need to perform an offline dictionary attack

* Kelsey completed a majority of her work on this project while employed at Aarhus University.

to recover the underlying password. For security to be applicable to protocols with public salts [25] aPAKE allows *precomputation attacks*, in which the attacker is able to, for example, generate pairs $(\text{pwd}, H(\text{pwd}))$ prior to server compromise and then immediately learn the server’s password upon compromise.

As in symmetric PAKE, a complete modeling of aPAKE should allow only *inevitable* attacks. For instance, it is clear that a compromised server can always run the aPAKE “in their head” to test passwords against their own storage. Note that this attack has runtime proportional to the number of password guesses the attacker makes *after* the server is compromised. Ideally, aPAKE security would model that this is *the only offline attack available* against the protocol. As described above, aPAKE modeling fails to capture this form of security.

The seminal work of Jarecki, Krawczyk, and Xu [44], observed that all existing aPAKE protocols were vulnerable to precomputation attacks and thus fell short of meeting the complete form of security described above. The authors formulated *strong* asymmetric PAKE (saPAKE) to model this complete notion of security in the setting with long-term storage. Further, they proposed OPAQUE, one of two original saPAKE protocols, that has since been standardized by the Internet Engineering Task Force [13].

Idealized Models in (s)aPAKE. It is known that UC-secure PAKE and its variants cannot be instantiated in the plain model [18]. While symmetric PAKE can be realized in the standard model [18], extending this to aPAKE and saPAKE introduces additional model restrictions. In particular, aPAKE (and, therefore, saPAKE) *requires an idealized model with strong programmability* [39, 50, 51], precluding the standard-model constructions that remain feasible in the symmetric setting. Due to this impossibility, all existing UC-secure (s)aPAKE protocols rely on a programmable model, such as the random oracle model [60, 61] or the generic group model [14, 52].

This restriction on (s)aPAKE protocols may not be immediately obvious in part because the expected (s)aPAKE security properties cannot be understood from the functionality alone! To help illuminate where our intuitive security properties diverge from the functionality, we present a standard degenerate aPAKE protocol [34]. Consider an aPAKE parameterized by an enumerable dictionary where the server simply stores its password in the clear and runs a symmetric PAKE protocol with the client to derive a key. Relative to this protocol, the ideal adversary can simply test all possible passwords in the dictionary prior to interacting with the environment. Such an adversary can trivially learn the server’s password and perfectly align the environment’s view. Although this protocol is secure relative to the aPAKE functionality, it violates our intuitive notion of aPAKE security.

Restricting the Ideal Adversary. The standard (s)aPAKE literature restricts how an ideal adversary can interact with the ideal functionality to remediate this problem. There are several approaches to enforce this restriction [14, 39, 51, 61] and we adopt the notion of *simulation rate*, introduced by McQuoid, Rosulek, and Xu [51]. To achieve a bounded simulation rate, we limit how many queries an ideal adversary can make to the functionality based on the real adversary’s queries to the underlying idealized model. Specifically, if a protocol has a *simulation rate* r then the ideal adversary can make r queries to the ideal functionality for every query the real adversary makes to the ideal model. In the context of saPAKE, this translates to bounding the number of offline password guesses by an r -multiple of the number of *post-compromise* random oracle queries.

Requiring a low simulation rate rules out degenerate (s)aPAKE constructions, such as the one described above, but inherits the impossibility result. Towards achieving a definition of security that admits only *inevitable* attacks against the protocol, a simulation rate of $1/q$ is preferable, where q is the number of ideal model queries made by honest parties during an online interaction. However, to the best of our knowledge, no existing protocol achieves a simulation rate $r < 1$. Some protocols even allow $r > 1$, effectively enabling a speedup in attacks against the server’s storage.

Additionally, the ideal adversary must be able to extract the password guesses from the real adversary. Previous two-round saPAKE constructions based on GGM use the GGM to construct a password hash with some algebraic structure, making it easier to evaluate in two rounds. Therefore, no assumption on a group would suffice – the GGM is essential for extracting the real adversary’s hashing attempts. In contrast, our protocol extracts offline guesses from the RO, and uses a standard model group (with an interactive assumption) to instantiate an OPRF.

One practical consequence is that we can easily set a difficulty parameter for hashing. Usually password hashes are made computationally expensive to evaluate, to slow down offline attacks. In ROM it is clear how to do this: use the slow hash function to instantiate the random oracle. There are several options for how this slow down can be done including by iterating the hash function many times, or by making it memory hard, etc.. Increasing the computational complexity of evaluating GGM-based password hashing would require slowing down the group operations, and there are not many options for doing this other than by making the group bigger.

1.1 Related Work

Password Authenticated Key Exchange. Broadly speaking, existing saPAKE protocols fall into one of five categories, each with limitations (summarized in Table 1). Our work improves upon at least two independent parameters in comparison to all existing prior work.

Protocols	Rounds	Model	Complete Simulation	
			saPAKE	Rate
[44, 45]	3	ROM	×	1
[37]	3	ROM	✓ [?]	1
[52]	2 (Server-First)	ROM, GGM	✓	2
[52]	2 (Server-First)	ROM, GGAM	✓	1
[24]	2 (Server-First)	ROM, GGM (Pairings)	✓	1
[14]	2 (Client-First)	ROM, GGM	✓	$O(1)$
OneTwoPAKE 2 (Client-First)		ROM	✓	1

Table 1. Comparison of saPAKE protocol classes. **Protocols** cites the relevant prior work. **Rounds** indicates the round complexity of the protocol where, for example, **2 (Server-First)** denotes a two round protocol consisting of a single message from the Server to the Client followed by a response from the Client to the Server. **Model** indicates if the protocol assumes is in the random oracle model (ROM), generic group model (GGM), or the generic group action model (GGAM). **Complete saPAKE** indicates if the protocol achieves full security. Note that while [37] claims complete saPAKE security, it is currently unclear what security level is achieved. **Simulation Rate** indicates the simulation rate.

The first class, that of OPAQUE [44], constructs an saPAKE protocol by combining an oblivious pseudo-random function (OPRF) with either an authenticated key exchange (AKE) or an aPAKE. The OPRF must be secure against man-in-the-middle attacks and adaptive server compromise, while the AKE must resist key compromise impersonation. Additionally, all components must satisfy UC-security. This approach can be used to build three-round saPAKE protocols that are secure in the random oracle model. However, subsequent work [45] showed that these protocols fall short of full saPAKE security, instead realizing a weaker functionality: *saPAKE minus*. The relaxations of saPAKE minus are formally discussed elsewhere [45, 50, 52], and are briefly recalled here. First, as in existing relaxed PAKE functionalities [1], an adversary can perform *delayed* password tests on terminated sessions; an adversary can specify a password guess after a session is completed. Second, a novel relaxation seen in this class, including thresholdized variants [36], allows an adversary to exploit a correct password guess in one server session to compromise a previously interrupted session; an adversary can “interrupt” a server session by, *e.g.*, making an incorrect password guess and later “compromise” *that same session* with a correct password guess *from a different session*. These relaxations are neither necessary nor natural and, as previously observed, “appear to be the result of tailoring the UC saPAKE functionality to fit the OPAQUE protocol” [52].

The second class, that of strong-AuCPace [37] by Haase and Labrique, follows a similar pattern to OPAQUE and introduces an “augmentation layer” that upgrades a *symmetric* PAKE into a *strong asymmetric* PAKE. This approach can produce three-round protocols in the random oracle model. While the augmentation layer relies on a white-box application of the same OPRF used in OPAQUE, the authors additionally incorporate an ephemeral Diffie-Hellman message from the server. The authors claim strong-AuCPace realizes the full saPAKE functionality; however, it is a folklore result that attacks analogous to those causing the relaxations found in OPAQUE [44, 50] apply to strong-AuCPace as well.

The third class, McQuoid-Xu [52], avoids the first relaxation by augmenting a symmetric PAKE with a salting stage. The authors also eliminate the second relaxation by proving their online security in the algebraic group model [32]. The resulting protocols are efficient, require only two rounds, and can be instantiated with either elliptic curves or cryptographic group actions. However, this class still has some limitations. First, the offline security analysis relies on the generic group model (GGM) or the generic group action model (GGAM) [29]. It is unclear if protocols proven secure in these generic models remain secure when a given instantiation of the model (for instance a chosen elliptic curve) is used in another protocol as each client-server pair is assumed to be associated with a distinct generic group (or action). If security can not be proven against a global model or against model reuse, standardization, *e.g.*, relative to Curve25519 or CSIDH-512, will be difficult with rigorous guarantees. Second, while the compiled protocols can be carried out in two rounds, the server initiates the protocol. This flow pattern is counterintuitive unless the saPAKE functions as a subroutine of a higher-level protocol and the compiled protocols become three rounds when the client speaks first. Finally, while these protocols realize the complete saPAKE functionality, they do not achieve the strongest possible security notion.

As identified in previous work [14, 34, 39, 44, 50, 51], to achieve the intuitive notion of (s)PAKE security, we must restrict the saPAKE ideal adversary’s offline password test queries relative to observable actions by the real adversary. McQuoid and Xu prove that when instantiating their compiler with generic group actions, they achieve an saPAKE with offline security against an ideal adversary that issues a single offline test per action by the real adversary — in other words, their compiler has a simulation rate $r = 1$. Unfortunately, due to the additional algebraic properties of the generic group model, instantiating the compiler with elliptic curves only realizes a simulation rate $r = 2$, halving the password search space in an offline attack and introducing an unnecessary relaxation similar to other saPAKE variants.

The fourth class, CRISP [24], follows a similar approach but replaces the GGM/GGAM with bilinear pairings to compile a symmetric PAKE into a *strong identity-binding PAKE*. Like McQuoid-Xu, CRISP relies on the GGM for offline security, but CRISP requires extensions including a hash-to-curve interface and generic pairings. Reliance on a generic model is particularly troublesome when using pairings, since real-world pairing-based cryptography typically uses specific curves such as Barreto-Naehrig [6] and Barreto-Lynn-Scott [5], which use the Ate pairing and require hardness of the finite field discrete logarithm problem for which non-generic attacks exist, such as index-calculus algorithms.

The fifth and final class, Bradley-Jarecki-Xu [14], contains two-round protocols that realize the complete saPAKE functionality. This is achieved through the “commit-and-SPHF” (smooth projective hash function) paradigm along with an *algebraic password file*. Originally introduced by Katz, Ostrovsky, and Yung [48] for symmetric PAKE in the CRS model, the commit-and-SPHF approach operates as follows: relative to the CRS, the client and server exchange commitments to their respective passwords, hash the received commitment with a smooth projective hash for the language of commitments to that password and, if the hash evaluations match, derive the same key. Prior work [47] extended this technique to the asymmetric setting by using projective hashes for two separate languages, but fell short of strong security due to pre-compromise attacks. Bradley, Jarecki, and Xu resolve this issue by merging both SPHF evaluations to simultaneously authenticate both parties and by leveraging the GGM in their offline analysis. Their compiler relies on a *conditional key encapsulation mechanism* (CKEM) [41] combined with a suitable algebraic *salted tight one-way function*. Intuitively, CKEM can be viewed as an authenticated KEM conditioned on the receiving party holding a witness for an NP relation. An astute reader may notice that CKEMs mirror the intuition behind SPHF’s where a party can compute a projective hash (*i.e.*, derive the key) only if they know a valid witness for some

NP relation. This is not far from the truth and the authors provide a method for building CKEM from a suitable SPHF.

The authors additionally provide a concrete instantiation of their protocol. At a high-level, the client sends an ElGamal commitment to their password and the server samples an SPHF hashing key, computes the associated projection key, and sends the projection key along with verification material to the client *à la* Fujisaki-Okamoto [33]. To achieve precomputation security, the authors require their SPHF be compatible with the hashed Boneh-Boyen function [11], $F_s(x) = (1/(s + H(x)))G$, which is used to construct the server’s password storage. Since $H(x)$ can be computed precompromise, the authors require observable and programmable group operations, realized through the GGM, to prove saPAKE security. As we spoke to previously, it then is unclear if the protocol’s security holds relative to a global or reusable group. Furthermore, the authors only prove that their protocol achieves an asymptotic simulation rate $r = O(1)$ and do not provide a tight analysis⁵.

Efficiency Comparison. Unfortunately, our protocol is less computationally efficient than previous work in the ROM, making it unsuitable as a drop-in replacement for standardized protocols (such as OPAQUE). The comparison with protocols proven in the GGM is less clear, as soundly instantiating the GGM in practice may require generating new elliptic curve groups per protocol instance, hindering efficiency. Despite this, our work represents an important feasibility result, demonstrating that these three crucial saPAKE properties can be achieved together. Moreover, the modularity of our framework (see the technical overview) makes it well-suited for various extensions, such as threshold saPAKE and LAKE for more expressive languages.

Oblivious Pseudorandom Functions. An Oblivious PRF (OPRF) [31] is an interactive protocol between a server holding a key k and a client holding an input x , allowing the client to evaluate a PRF $F_k(\cdot)$ such that they receive the output $y := F_k(x)$ while the server learns nothing about x .

The first ideal OPRF functionality, introduced by Jarecki et al. [42], models a verifiable OPRF where the server registers a function at initialization (essentially a public key) and the client has certainty that an output was produced correctly relative to that function. Subsequently, a functionality without verifiability [43] was proposed. A crucial feature of an OPRF functionality—given its role in password-based authentication—is the explicit absence of authenticity. If a protocol runs over insecure channels it is inherently vulnerable to man-in-the-middle attacks, which must be reflected in the ideal functionality by allowing the ideal adversary to initialize *adversarial sessions* and deliver outputs in these sessions without server participation. Here, we consider UC-secure OPRF protocols that are *unauthenticated*, meaning that they do not rely on authenticated channels, but a complete formal treatment of OPRFs is provided by Casacuberta, Hesse, and Lehmann [19].

There are four classes of OPRF protocols, based on the underlying PRF: Hashed Diffie-Hellman [54], Naor-Reingold (NR) [55], Dodis-Yampolskiy (DY) [26], and other generic constructions. Nearly every UC-secure OPRF protocol computes the Two-Hashed Diffie-Hellman (2HDH) PRF [42]:

$$F_{2HDH}(k, x) = H'(x, H(x)^k).$$

This PRF is highly versatile, enjoys a range of useful properties, and is secure under the One-More Gap Diffie-Hellman assumption.

A seemingly unavoidable drawback of F_{2HDH} is *delayed* (or *lazy*) extraction; in the protocol, a client does not need to specify the input until the external random oracle query corresponding to the evaluation of H' . This is reflected by the functionality; the client does not need to specify an input to the functionality until immediately prior to output, meaning that the session can be completed from the server’s point of view prior to input specification. As discussed, delayed extraction presents challenges for protocols that rely on the OPRF-hybrid model, as the simulator cannot immediately extract the OPRF input. All of the existing OPRF functionalities model delayed extraction, making them unsuitable for settings requiring *online extraction*. Our functionality in Section 4 is the first to incorporate online extraction.

⁵ As the authors’ offline storage is algebraically similar to McQuoid-Xu, we believe it is likely that a simulation rate $r = 2$ is achievable for the concrete instantiation.

Existing composable OPRF constructions from a non-2HDH PRF include: an NR-based OPRF [42], a lattice-based verifiable OPRF (vOPRF) [4], and a DY-based OPRF [15] that is secure against honest-but-curious adversaries under the assumption of authenticated channels. A stand-alone (non-UC) distributed DY-PRF has been proven secure in the case of two servers [53]. In the multi-server case Doerner et al. [28] describe how to distributively evaluate the verifiable DY-PRF, but assume authenticated channels. Protocol 4.2 in Section 4 is the first DY-based OPRF that is UC-secure with security in all of the following cases: against malicious adversaries, over insecure channels, and against adaptive server compromise. Furthermore, our DY-based OPRF has an exponential size domain. To prove security, we rely on an assumption that is essentially equivalent to the assumption that Boneh–Boyen signatures are existentially unforgeable; we refer to Definition 3 for an explicit description.

Finally, the functionality in Section 4 is the first to incorporate committed inputs. The only prior OPRF protocols to consider committed inputs are those of [15, 46], where the former of Jarecki et al. is not UC-secure and the latter of Camenisch et al. is not secure against malicious adversaries.

1.2 Contribution and Technical Overview

A high-level overview of our saPAKE protocol appears in Figures 1.1 and 1.1. Our protocol is conceptually simple and improves over every previous saPAKE protocol in at least two of the following independent aspects:

- *Functionality relaxations*: our protocol realizes the non-relaxed saPAKE functionality $\mathcal{F}_{\text{saPAKE}}$.
- *Round-complexity*: our protocol consists of two communication rounds, a single round trip, with the client sending the first message.
- *Simulation Rate*: our protocol achieves a simulation rate of $r = 1$.
- *Idealized Model Assumptions*: our protocol is secure in the ROM only, as opposed to the GGM or GGAM.

Note that in our registration phase, the server executes its role independent of the client. This is an artifact of the saPAKE functionality which expects any communication during registration occurs out-of-band. In an implementation, it is expected that the client generates the server’s storage interactively⁶, but we leave modeling interactive saPAKE registration for future work. Intuitively, the login flow of our protocol can be divided into two main components: the top half consists of a committed-input, online-extractable OPRF secure against adaptive compromise, while the bottom half follows a commit-and-SPHF approach over the commitments output by the OPRF.

As noted in [50, Observation 1], two primary challenges arose when trying to construct previous saPAKE protocols from OPRFs [37, 44] leading to relaxing the saPAKE functionality: (1) delayed extraction in the OPRF functionality and (2) a lack of connection between the OPRF and the non-OPRF subsession halves. Delayed extraction is a problematic feature inherited from the 2HDH-OPRF used in prior work. The only viable solution, and the approach we take, is to redefine the functionality without delayed extraction and design a new OPRF protocol that realizes it.

Our OPRF is secure against man-in-the-middle attacks and adaptive server compromise, as needed for building saPAKE. Its novelty lies in two key properties: *online extractability* and *committed inputs*. In our design, the client must specify an input before the server can participate in evaluation, and the server receives a commitment to that input when prompted for evaluation.

An intermediate output of our OPRF is the Dodis-Yampolskiy PRF:

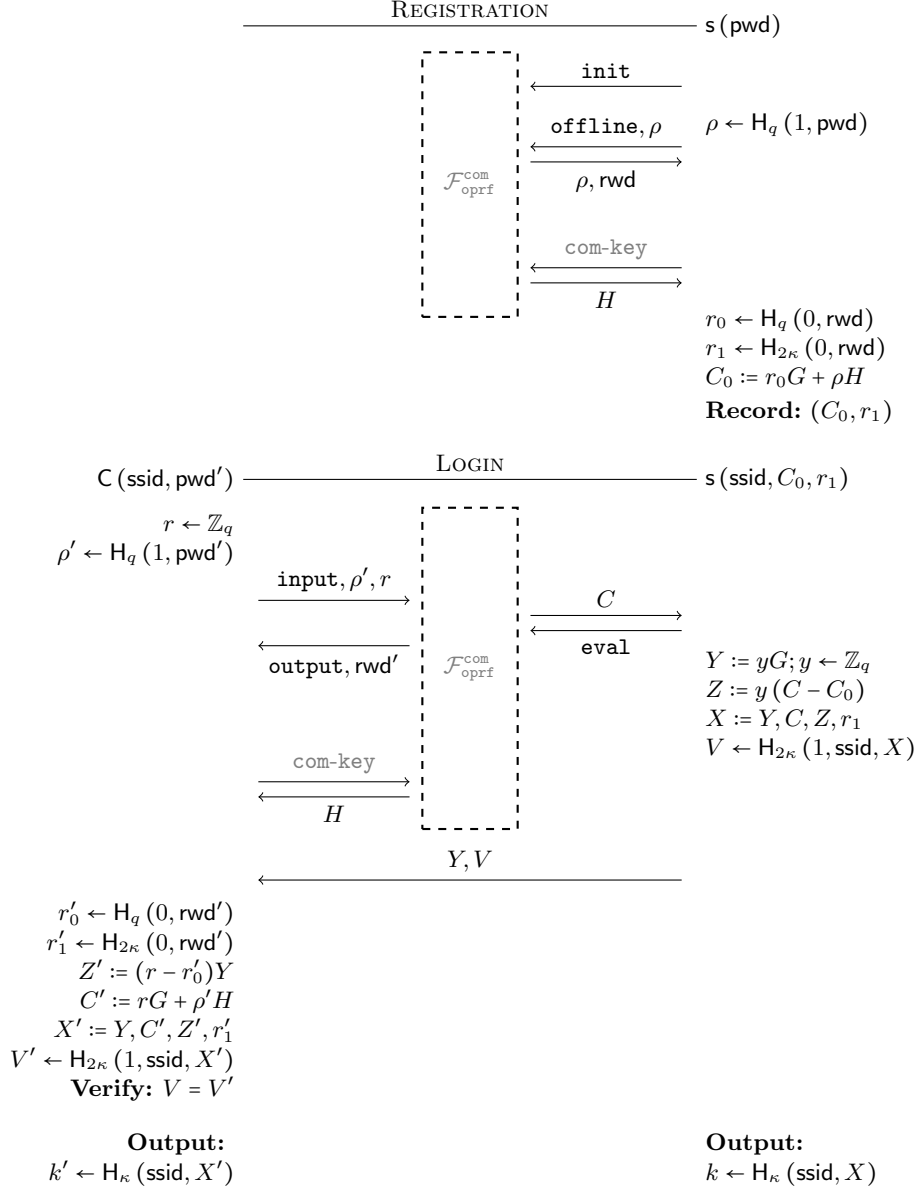
$$Y = F(\text{sk}, x) := \left(\frac{1}{\text{sk} + x} \right) G,$$

where $(\mathbb{G}, G, q)$ is a cyclic group, sk is the key, and x is the input. To compute Y , the server and client invoke a committed-input VOLE functionality $\mathcal{F}_{\text{vole}}^{\text{com}}$, which outputs a commitment to the client’s input,

⁶ This needs to be interactive since our OPRF functionality does not have an explicit notion of transferable keys.

Figure 1.1: saPAKE.

A high-level overview of the protocol flow for saPAKE registration and login. The password registration process is executed by s in the $(\mathcal{F}_{R0}, \mathcal{F}_{opr}^{com})$ -hybrid model. Password-based login is executed between the server s and the client C in the $(\mathcal{F}_{R0}, \mathcal{F}_{opr}^{com})$ -hybrid model. For clarity, H_q , H_κ , and $H_{2\kappa}$ are written as functions, but they represent separate instances of $\mathcal{F}_{R0}$ with handle sets $\mathbb{Z}_q$, $\{0, 1\}^\kappa$, and $\{0, 1\}^{2\kappa}$, respectively. We denote by rwd a *randomized password*.



$C = xH + rG$, to the server, along with additive shares of the product ux , where u is a uniform nonce known to the server. Online extractability is enforced through VOLE: the client's VOLE input x is also the OPRF input, guaranteed by a consistency check between the commitment generated by the client and the

commitment output by the VOLE. For server public key $\mathbf{pk} = \mathbf{sk}G$, the final OPRF output is:

$$y = H(x, Y, \mathbf{pk}),$$

Once delayed extraction is addressed, the second challenge remains: the disconnect between a given OPRF evocation and the associated saPAKE subsession. This disconnect is at the root of the relaxation, allowing an adversary to exploit a correct password guess in one session to compromise a previously interrupted session [44]. We note that McQuoid [50] already suggested using committing primitives to mitigate this issue, a feature that we incorporate into our OPRF functionality. However, while applying our OPRF construction directly in OPAQUE may remedy delayed extraction in their saPAKE functionality, it is not clear how to incorporate the session commitment into their protocol. To remove the final relaxation, we build a novel post-OPRF exchange.

Following our committing OPRF stage, we instantiate the post-OPRF exchange of our protocol following the commit-and-SPHF paradigm. Specifically, we define a PHF over the language of pairs of Pedersen commitments [58] to the same word:

$$\mathcal{L} = \left\{ (C, C_0) \in \mathbb{G}^2 \mid \exists m, r, r_0 \in \mathbb{Z}_q : \begin{array}{l} C = (r \cdot G + m \cdot H) \\ C_0 = (r_0 \cdot G + m \cdot H) \end{array} \right\}.$$

As any group element can be interpreted as a Pedersen commitment to any word (making $\mathcal{L} = \mathbb{G}^2$), the standard notion of smoothness used in usual SPHF constructions is not useful. Instead, our construction relies on the computational binding property of Pedersen commitments and a *pseudorandomness* property of the resulting PHF when a word's witness is unknown. More concretely, we use the following PHF over $\mathcal{L}$, following diverse vector space notation [9, 10]:

HashKG : Samples $\alpha \leftarrow \mathbb{Z}_q$ and outputs the hashing key $\mathbf{hk} := \alpha$.

ProjKG($\mathbf{hk}, \chi$) : Outputs the projection key $\mathbf{hp} := \gamma$, for the word $\chi = (C, C_0)$

$$\gamma = \alpha \cdot F(\chi) = \alpha \cdot G \in \mathbb{G}.$$

Hash($\mathbf{hk}, \chi$) : Outputs the hash value, for $\Theta : (C, C_0) \mapsto (C - C_0)$

$$\mathbf{H} := \alpha \cdot \Theta(\chi) = \alpha(C - C_0) \in \mathbb{G}.$$

ProjHash($\mathbf{hk}, \chi$) : Outputs the projected hash value from the witness $w = (r, r_0)$

$$\mathbf{pH} := \lambda(\chi, w) \cdot \gamma = (r - r_0) \cdot \gamma \in \mathbb{G}.$$

The main technique in our construction lies in how the client evaluates the hash. During registration, the server commits to $\mathbf{H}(\mathbf{pwd})$ using randomness $F(\mathbf{H}(\mathbf{pwd}))$, producing:

$$C_0 := F(\mathbf{H}(\mathbf{pwd})) \cdot G + \mathbf{H}(\mathbf{pwd}) \cdot H.$$

This commitment is not directly revealed to the client during login unless they guess the correct password. The client uses their chosen, subsession-specific, commitment randomness r along with the hashed OPRF output r_0 to compute the projection $(r - r_0) \cdot \gamma$. Clearly, the client only learns the word (C, C_0) that the server is hashing if they learn the correct OPRF output! This construction may have applications beyond saPAKE, potentially extending to other languages beyond string equality given a suitable OPRF.

Finally, while our construction builds on two established tools — OPRFs and PHFs — our work succeeds where previous attempts have failed. Since our OPRF is not weakened by delayed extraction, we do not need to relax the functionality as in strong AuCPace [37]. Additionally, our OPRF is committing, and the commitment is directly leveraged in key computation, ensuring that the OPRF evaluation for a subsession is tied to that subsession. This prevents the final relaxation in OPAQUE's functionality [44]. Finally, since we use the OPRF as an integral part of evaluating the PHF stage, we avoid the need for GGM-based analysis for offline security [14].

Roadmap of the Paper. Section 2 recalls relevant preliminaries, Section 3 defines an ideal functionality and protocol for VOLE over unauthenticated channels, Section 4 gives an ideal functionality for online-extractable, committed OPRF over unauthenticated channels and a protocol in the unauthenticated VOLE hybrid model, and finally Section 5 presents our saPAKE protocol in the OPRF hybrid model.

2 Preliminaries

Notation. We use $a := x$ for assignment of the value x to the variable a . We use $x \leftarrow \mathcal{X}$ to denote the uniform sampling of x from the set $\mathcal{X}$ or, similarly, $y \leftarrow \mathbf{Y}(x)$ to denote y as the output of a randomized process $\mathbf{Y}$ on input x . Vectors $\mathbf{v}_i = (v_{i,1}, \dots, v_{i,k})$ are boldface and indexed via subscripts. Scalar multiplication, the multiplication of a vector by a scalar and the multiplication of a group element by a vector, are denoted similarly: $x(y_1, \dots, y_n) = (xy_1, \dots, xy_n)$ and $(h_1, \dots, h_n)G = (h_1G, \dots, h_nG)$. Brackets are used to indicate ranges of integers: $[n] = \{1, \dots, n\}$ and $[a, b] = \{a, \dots, b\}$. Double brackets are used to represent additive shares of a value $[[a]]_{\mathcal{P}}^i$, where the superscript indicates the identity of the shareholder and the subscript indicates the set of parties between which the value is shared.

Model. Security is analyzed and our protocols are written in the UC framework of Canetti [16]. We consider a static malicious probabilistic polynomial time (PPT) adversary. We denote an ideal functionality by $\mathcal{F}$ and a protocol by Π . The set of parties $\mathcal{P}$ only have access to insecure channels.

Zero Knowledge. When indicated, parties executing a protocol are required to prove the relations below, all parameterized by a group $\text{pp}_{\mathbb{G}} = (\mathbb{G}, G, q)$.

$$\mathcal{R}_{\text{DLog}}^{\text{pp}_{\mathbb{G}}} := \{\phi = (A, B), w \mid \phi \in \mathbb{G}^2, w \in \mathbb{Z}_q, B = wA\} \quad (1)$$

$$\mathcal{R}_{\text{dht}}^{\text{pp}_{\mathbb{G}}} := \left\{ \phi = (G, A, B, C), w \mid \begin{array}{l} \phi \in \mathbb{G}^4, w \in \mathbb{Z}_q \\ B = wG, C = wA \end{array} \right\} \quad (2)$$

$$\mathcal{R}_{\text{ped}}^{\text{pp}_{\mathbb{G}}} := \{\phi = (G, H, C), w = (a, b) \mid C = aG + bH\} \quad (3)$$

In this work, we rely on zero knowledge proofs for the following relations $\mathcal{R}_{\text{DLog}}$ representing the discrete logarithm assumption, $\mathcal{R}_{\text{dht}}$ representing diffie hellman tuples, and $\mathcal{R}_{\text{ped}}$ representing valid pedersen commitments. For our purposes, we want that the proofs are non-interactive and straight-line extractable for compatibility with UC-security. Straight-line extractable non-interactive proofs can be constructed using the Fischlin Transformation [30, 49].

Definition 1 (Non-Interactive Zero Knowledge Proof (NIZK) [30]). A pair of PPT algorithms $(\mathbf{P}, \mathbf{V})$ is called a non-interactive zero knowledge (NIZK) proof for a relation $\mathcal{R}$ with an online extractor (in the random oracle model) if it satisfies the following properties,

Completeness. For any oracle $\mathbf{H}$, any $(x, w) \in \mathcal{R}$ and any proof $\pi \leftarrow \mathbf{P}^{\mathbf{H}}$ there exists a negligible function $\text{negl}(\kappa)$ such that,

$$\Pr[\mathbf{V}^{\mathbf{H}}(x, \pi) = 1] \geq 1 - \text{negl}(\kappa)$$

Zero-Knowledge. Let $\mathbf{H}$ be a random oracle. There exists a PPT simulator $\mathcal{S}_{\text{zk}} = (S_0, S_1)$ such that for any pair of PPT algorithms $D = (D_0, D_1)$ the following distributions are computationally indistinguishable:

- Let $(x, w, \text{st}) \leftarrow D_0^{\mathbf{H}}(\kappa)$ and $\pi \leftarrow \mathbf{P}^{\mathbf{H}}(x, w)$. If $(x, w) \in \mathcal{R}$ output $D_1^{\mathbf{H}}(\pi, \text{st})$ otherwise output $D_1^{\mathbf{H}}(\perp, \text{st})$
- Let $(H_0, \sigma) \leftarrow S_0(\kappa)$, $(x, w, \text{st}) \leftarrow D_0^{\mathbf{H}_0}(\kappa)$. If $(x, w) \in \mathcal{R}$ then, $(H_1, \pi) \leftarrow S_1(\sigma, x, 1)$ otherwise, $(H_1, \pi) \leftarrow S_1(\sigma, x, 0)$. Output $D_1^{\mathbf{H}_1}(\pi, \text{st})$

Straight-line Extraction. Let $\mathbf{H}$ be a random oracle, $(x, \pi) \leftarrow \mathbf{P}^*(\kappa)$, and $Q_{\mathbf{H}}(\mathbf{P}^*)$ be the sequence of queries made by $\mathbf{P}^*$ to the oracle and their answers. Let $w \leftarrow \text{Ext}(x, \pi, Q_{\mathbf{H}}(\mathbf{P}^*))$. There exists a PPT extractor Ext and a negligible function $\text{negl}(\kappa)$ such that for any algorithm $\mathbf{P}^*$ following holds,

$$\Pr[(x, w) \notin \mathcal{R} \wedge \mathbf{V}^{\mathbf{H}}(x, \pi) = 1] \leq \text{negl}(\kappa)$$

3 VOLE

There are several approaches to instantiating VOLE in the literature, including those based on Oblivious Transfer [27, 38], Linear Homomorphic Encryption [17, 20], and Pseudorandom Correlation Generators [2]. Our specific requirements are that of a two-round protocol where the first message encodes the scalar value, and additionally produces a Pedersen commitment to the scalar value, and moreover achieves UC security. This is specified formally in $\mathcal{F}_{\text{ua-voles}}^{\text{com}}$ (Figure 3.1). In essence, we augment the standard VOLE functionality with an interface for the adversary to intercept the messages of either party, and replace it with its own input.

Functionality 3.1: $\mathcal{F}_{\text{ua-voles}}^{\text{com}}(\text{sid}, \text{pp} \in \{q, \text{pp}_G\}, \ell)$ **VOLE.**

The functionality $\mathcal{F}_{\text{ua-voles}}^{\text{com}}$ with session identifier $\text{sid} = ([\dots], A, B)$ is parameterized by a prime q and the dimension ℓ . In the special case, the functionality is parameterized by a group $\text{pp}_G = (\mathbb{G}, G, H, q)$ instead of a prime q . The functionality interacts with the two parties, A, B , and the ideal adversary $\mathcal{A}^*$ via the queries below. Upon invocation the functionality receives the identities of any corrupted parties.

1. **On (sample-VOLE-input) from B .** Ignore the query if either (sample-VOLE-input, $[\dots]$) or **complete** is recorded. Forward (sample-VOLE-input, B) to $\mathcal{A}^*$. If B is corrupted then receive (sample-VOLE-input, r, ρ) from $\mathcal{A}^*$, and otherwise sample $\rho, r \leftarrow \mathbb{Z}_q$. Compute $C := \rho G + rH$. Record and send (sample-VOLE-input, r, ρ, C) to B . Send (sample-VOLE-input, C) to $\mathcal{A}^*$.
 - (a) **On (notify- A) from $\mathcal{A}^*$.** Send (sample-VOLE-input, C) to A .
 - (b) **On (adv-input, r^*, ρ^*) from $\mathcal{A}^*$.** Compute $C^* := \rho^* G + r^* H$. Record (adv-input, r^*, ρ^*, C^*). Send (sample-VOLE-input, C^*) to A .
2. **On (VOLE-multiply, a) from A .** Ignore the query if **complete** is recorded. Otherwise, send (VOLE-multiply, A) to $\mathcal{A}^*$. Record **complete**.
 - If B is corrupt or there is a record (adv-input, r^*, ρ^*, C^*): receive (VOLE-output, y_B) from $\mathcal{A}^*$, define $y_A := R \cdot a - y_B$ for $R = r$ if there is no record (adv-input, $[\dots]$) and $R := r^*$ otherwise.
 - If A is corrupt: receive (VOLE-output, y_A) from $\mathcal{A}^*$, define $y_B := r \cdot a - y_A$.
 - Otherwise: sample $y_A, y_B \leftarrow \mathbb{Z}_q^\ell$ such that $y_A + y_B := r \cdot a$.
 Record (a, y_A, y_B) . Send (VOLE-output, y_A) to A .
 - **On (deliver-output) from $\mathcal{A}^*$.** Output (VOLE-output, y_B) to B .
3. **On (adv-output, a^*, y_A^*) from $\mathcal{A}^*$.** Ignore the query if **complete** is recorded. Otherwise, record **complete**, define $y_B^* := r a^* - y_A^*$, and send (VOLE-output, y_B^*) to B .

The functionalities $\mathcal{F}_{\text{ua-voles}}^{\text{com}}$ and $\mathcal{F}_{\text{ua-voles}}$ are the above with and without the indicated text, respectively. Similarly, $\mathcal{F}_{\text{voles}}^{\text{com}}$ is the authenticated variant, in which $\mathcal{A}^*$ does not have access to the interfaces **adv-input** and **adv-output**.

These requirements are relatively straightforward to satisfy with known techniques, however to our knowledge they are not simultaneously achieved explicitly in previous work. Therefore, in the interest of completeness we provide such a VOLE protocol here, making use of Linear Homomorphic Encryption (augmented by ZK proofs) due to its conceptual simplicity. While our construction prioritizes conceptual simplicity over concrete efficiency, it is plausible that carefully instantiating the encryption scheme and corresponding ZK proofs as in previous work [3, 17, 20] would yield concretely efficient constructions.

We begin by defining Linear Homomorphic Encryption, as necessary for our VOLE construction.

Definition 2 (Linear Homomorphic Encryption). A linearly homomorphic encryption (LHE) scheme is a triple of algorithms:

$\text{KeyGen}(1^\lambda; \rho) \rightarrow (\text{ek}, \text{dk})$ key generation is a probabilistic algorithm that takes as input a security parameter λ and optionally explicit randomness ρ , and outputs a public key ek and a private key dk . The public key ek determines an integer $M \in \Omega(2^{3\kappa})$.

$\text{LHEnc}(\text{ek}, x; \varrho) \rightarrow \text{ct}_x$ encryption is a probabilistic algorithm which takes as input an encryption key ek , a string $x \in \mathbb{Z}_M$, and optionally explicit randomness ϱ , and outputs a ciphertext ct .
 $\text{LHDec}(\text{dk}, \text{ct}) \rightarrow x'$ decryption is an algorithm which takes as input a decryption key dk and a ciphertext ct_x , and outputs either string x or $\perp$.

An LHE is required to satisfy the properties of correctness, linear homomorphism, and CPA-security, described below. **Correctness.** $\forall (\text{dk}, \text{ek}) \leftarrow \text{KeyGen}(1^\lambda)$ and $\forall x \in \mathbb{Z}_M$, it holds that $\Pr[\text{LHDec}(\text{dk}, \text{LHEnc}(\text{ek}, x)) = x] = 1$.

Linear Homomorphism. There exist efficient algorithms mult, add such that for all $(\text{dk}, \text{ek}) \leftarrow \text{KeyGen}(1^\lambda)$ and for all $x, y, \alpha, \beta \in \mathbb{Z}_M$, the following equation holds with probability 1:

$$\text{LHDec} \left(\text{dk}, \text{add} \left(\begin{array}{c} \text{mult}(\alpha, \text{LHEnc}(\text{ek}, x)), \\ \text{mult}(\beta, \text{LHEnc}(\text{ek}, y)) \end{array} \right) \right) = \alpha x + \beta y \pmod{M}$$

CPA Security. For any PPT adversary $\mathcal{A}$,

$$\Pr \left[b' = b \mid \begin{array}{l} b \leftarrow \{0, 1\}, (\text{dk}, \text{ek}) \leftarrow \text{KeyGen}(1^\lambda) \\ M_0, M_1 \leftarrow \mathcal{A}(\text{ek}) \\ b' \leftarrow \mathcal{A}(\text{LHEnc}(\text{ek}, M_b)) \end{array} \right] \approx_c 1/2 \quad (4)$$

For the sake of readability, we abuse notation in our exposition and use regular addition and multiplication to denote ciphertext addition and scalar multiplication, rather than add or mult .

Such an encryption scheme can be instantiated with standard techniques using groups of unknown order, such as Paillier [56] or Class Groups [21].

Next, we employ zero-knowledge proofs by operating in the $\mathcal{F}_{\text{ZK}}$ -hybrid model, which we define formally in Figure 3.2. The functionality $\mathcal{F}_{\text{ZK}}$ can be instantiated using any straightline extractable and simulatable

Functionality 3.2: $\mathcal{F}_{\text{ZK}}^{P,V}(\text{sid}, R)$ Unauthenticated Zero-knowledge.

The functionality $\mathcal{F}_{\text{ZK}}$ with session identifier $\text{sid} = ([\dots], P, V)$ is parameterized by a relation R . The functionality interacts with the two parties, P, V , and the ideal adversary $\mathcal{A}^*$ via the queries below.

1. **On (prove, x, w) from P .** Ignore the query if either (prove, $[\dots]$) or **complete** is recorded. Forward (prove, $x, R(x, w), P$) to $\mathcal{A}^*$.
 - **On response (continue) from $\mathcal{A}^*$.** Send (proven, $x, R(x, w)$) to V .
 - **On response (prove – instead, x^*, w^*) from $\mathcal{A}^*$.** Send (proven, $x^*, R(x^*, w^*)$) to V .
2. Record **complete** in memory.

NIZK scheme as appropriate for the language [57]. General NIZKs to prove any NP relation suffice in our context, as we are not concerned with concrete efficiency.

We are now ready to give our protocol for unauthenticated VOLE. Roughly, we follow the template for VOLE introduced by Gilboa [35]: B sends A a ciphertext $\text{ct}_B = \text{LHEnc}(r)$, to which A responds with $\text{ct}_{\mathbf{A}_i} = \mathbf{a}_i \cdot \text{ct}_B + \text{LHEnc}(\eta_i)$ for each $i \in [\ell]$, where $\eta_i \leftarrow \mathbb{Z}_{2|q|+s}$. The parties A and B output $\mathbf{y}_A = -(\eta \pmod{q})$ and $\mathbf{y}_B = \{\text{LHDec}(\text{dk}, \text{ct}_{\mathbf{A}_i}) \pmod{q}\}_{i \in [\ell]}$ respectively, so that $\mathbf{y}_A + \mathbf{y}_B = r \cdot \mathbf{a} \pmod{q}$. Semantic security of the LHE scheme keeps r hidden from A , and η acts as a statistical one-time pad to keep $(r \cdot \mathbf{a})/q$ hidden from B . Finally, zero-knowledge proofs of executing the above steps correctly are used to enforce honest behaviour. Moreover the ZK proofs guarantee non-malleability so that an intercepting adversary can at best replace either party's message with an alternative "honestly generated" one.

The full protocol $\Pi_{\text{ua-vo1e}}^{\text{com}}$ is specified in Figure 3.3.

Protocol 3.3: $\Pi_{\text{ua-vole}}^{\text{com}}(\text{sid}, \text{pp} \in \{q, \text{pp}_{\mathbb{G}}\}, \ell)$ **Unauthenticated Vector OLE.**

The protocol $\Pi_{\text{ua-vole}}^{\text{com}}$ with session identifier $\text{sid} = ([\dots], A, B)$ is parameterized by a statistical security parameter s , prime q , the dimension ℓ , and a Linearly Homomorphic Encryption scheme $\text{KeyGen}, \text{LHEnc}, \text{LHDec}$. The message space of the LHE scheme is the integers modulo M such that $|M| > 2|q| + s + 1$. Party A has input $\mathbf{a} \in \mathbb{Z}_q^\ell$. In the special case, the protocol is parameterized by a group $\text{pp}_{\mathbb{G}} = (\mathbb{G}, G, H, q)$ as opposed to simply the prime q . The protocol is executed between two parties, A, B .

1. **On input** (sample-VOLE-input), **Party B** does the following:
 - (a) Sample input $\rho, r \leftarrow \mathbb{Z}_q$, and LHE keys $(\text{ek}, \text{dk}) \leftarrow \text{KeyGen}(1^\kappa; \rho^B)$
 - (b) Compute $\text{ct}_B = \text{LHEnc}(r)$ and $C = \rho \cdot G + r \cdot H$
 - (c) Send $\text{prove}, ((\text{ek}, \text{ct}_B, C), (\rho^B, \rho))$ to $\mathcal{F}_{\text{ZK}}^{B,A}(\text{sid-zk-B}, R_B)$, where

$$R_B = \left\{ \left(\begin{array}{c} (\text{ek}, \text{ct}_B, C) \\ (\rho^B, \rho) \end{array} \right) \middle| \begin{array}{l} (\text{ek}, \text{dk}) = \text{KeyGen}(1^\kappa; \rho^B) \wedge \\ r = \text{LHDec}(\text{dk}, \text{ct}_B), \text{ s.t.} \\ r < q \wedge C = \rho \cdot G + r \cdot H \end{array} \right\} \quad (5)$$

- (d) Send $\text{ek}, \text{ct}_B, C$ to A .
2. **On receiving** $\text{ek}, \text{ct}_B, C$ from B and (proven, $(\text{ek}, \text{ct}_B, C), 1$) from $\mathcal{F}_{\text{ZK}}^{B,A}(\text{sid-zk-B}, R_B)$, **Party A** does the following:
 - (a) Sample masks $\eta \leftarrow \mathbb{Z}_{2^{2|q|+s}}$, and encryption randomness $\varrho \leftarrow \{0, 1\}^{\kappa \times \ell}$
 - (b) Set output shares $\mathbf{y}_A = -(\eta \pmod q)$
 - (c) Compute $\text{ct}_A = \{\mathbf{a}_i \cdot \text{ct}_B + \text{LHEnc}(\text{ek}, \eta_i; \varrho_i)\}_{i \in [\ell]}$
 - (d) Send (prove, $(\text{ek}, \text{ct}_A), (\varrho, \eta, \mathbf{a})$) to $\mathcal{F}_{\text{ZK}}^{A,B}(\text{sid-zk-A}, R_A)$, where

$$R_A = \left\{ \left(\begin{array}{c} (\text{ek}, \text{ct}_A) \\ (\varrho, \eta, \mathbf{a}) \end{array} \right) \middle| \forall i \in [\ell]: \begin{array}{l} \mathbf{a}_i < q \wedge \eta_i < 2^{2|q|+s} \wedge \\ \text{ct}_{A_i} = \mathbf{a}_i \cdot \text{ct}_B + \text{LHEnc}(\text{ek}, \eta_i; \varrho_i) \end{array} \right\} \quad (6)$$

- (e) Send ct_A to **Party B** and output $\mathbf{y}_A$
3. **On receiving** ct_A from A and (proven, $(\text{ek}, \text{ct}_A), 1$) from $\mathcal{F}_{\text{ZK}}^{A,B}(\text{sid-zk-A}, R_A)$, **Party B** computes and outputs $\mathbf{y}_B = \{\text{LHDec}(\text{dk}, \text{ct}_{A_i}) \pmod q\}_{i \in [\ell]}$

The protocols $\Pi_{\text{ua-vole}}^{\text{com}}$ and $\Pi_{\text{ua-vole}}$ are the above with and without the indicated text, respectively.

Theorem 1. Assuming that $\text{KeyGen}, \text{LHEnc}, \text{LHDec}$ is a Linear Homomorphic Encryption scheme, Protocol $\Pi_{\text{ua-vole}}^{\text{com}}$ (Figure 3.3) UC-realizes Functionality $\mathcal{F}_{\text{ua-vole}}^{\text{com}}$ (Figure 3.1) in the $\mathcal{F}_{\text{ZK}}$ -hybrid model, in the presence of up to one static malicious corruption.

Proof. (Sketch) We begin by observing that there are essentially three cases:

- A corrupt and B honest.
- B corrupt and A honest.
- Both A, B honest.

The third case is notable only when the ideal adversary $\mathcal{A}^*$ sends one of $\text{adv-input}, \text{adv-output}$ to $\mathcal{F}_{\text{ua-vole}}^{\text{com}}$, neither of which are meaningful if either party is corrupt. Due to our use of more powerful ZK proofs, simulating the first two cases is a simplified version of previous work on LHE-based (V)OLE over authenticated channels [3, 17], and so we give only a brief account of the simulation strategy and indistinguishability:

- **A corrupt and B honest.** The simulator sends $\text{ek}, \text{ct}_B, C$ to A on behalf of B , where ek, ct_B are computed as per the honest algorithm, and C is received from $\mathcal{F}_{\text{ua-vole}}^{\text{com}}$. In addition, the simulator sends $\text{proven}, (\text{ek}, \text{ct}_B, C), 1$ to A on behalf of $\mathcal{F}_{\text{ZK}}^{B,A}(\text{sid-zk-B}, R_B)$. These messages are distributed identically to the real protocol, as C perfectly hides r given that ρ is not present in A 's view. Next, upon receiving ct_A on $Party B$'s behalf, and (prove, $(\text{ek}, \text{ct}_A), (\varrho, \eta, \mathbf{a})$) on behalf of $\mathcal{F}_{\text{ZK}}^{A,B}(\text{sid-zk-A}, R_A)$, the simulator sends $(\mathbf{a}, \mathbf{y}_A = \eta \pmod q)$ to $\mathcal{F}_{\text{ua-vole}}^{\text{com}}$. It remains to reason about the distribution of $\mathbf{y}_B$ in the real protocol and the simulation. In the real protocol, given that each $\eta_i < M/2$ and $\mathbf{a}_i < q$ as per $\mathcal{F}_{\text{ZK}}$ and that each ciphertext

decrypts to a unique plaintext, it holds that $r \cdot \mathbf{a}_i + \eta_i = \text{LHDec}(\text{dk}, \text{ct}_{\mathbf{A}_i})$ will never exceed the modulus M , and therefore

$$\begin{aligned} \mathbf{y}_{B,i} &= \text{LHDec}(\text{dk}, \text{ct}_{\mathbf{A}_i}) \pmod{M} \pmod{q} \\ &= r \cdot \mathbf{a}_i + \eta_i \pmod{q} \\ &= r \cdot \mathbf{a}_i - \mathbf{y}_{A,i} \pmod{q} \end{aligned}$$

implying that in the real protocol, $(\mathbf{a}, \mathbf{y}_A = \eta \pmod{q})$ as per $\mathcal{F}_{\text{zk}}$ forms a VOLE correlation $\pmod{q}$ along with $r, \mathbf{y}_B$ output by B . Given that $r, \mathbf{y}_B$ are sampled to form a VOLE correlation along with $\mathbf{a}, \mathbf{y}_A$ by $\mathcal{F}_{\text{ua-voile}}^{\text{com}}$ in the simulation, B 's outputs $r, \mathbf{y}_B$ are distributed identically in both experiments. The only remaining component is that in the simulation, ct_B encrypts an r value that is independent of what $\mathcal{F}_{\text{ua-voile}}^{\text{com}}$ actually uses to form the VOLE correlation—this is indistinguishable from the real protocol by virtue of CPA security of the linearly homomorphic encryption scheme.

- **B corrupt and A honest.** The simulator receives $\text{ek}, \text{ct}_B, \underline{C}$ from B on behalf of A , and $\text{prove}, ((\text{ek}, \text{ct}_B, \underline{C}), (\rho^B, \underline{\rho}))$ on behalf of $\mathcal{F}_{\text{zk}}^{B,A}(\text{sid-zk-B}, R_B)$. The simulator computes $(\text{ek}, \text{dk}) = \text{KeyGen}(1^\kappa; \rho^B)$ and $r = \text{LHDec}(\text{dk}, \text{ct}_B)$ and sends $(\text{sample-VOLE-input}, r, \underline{\rho})$ to $\mathcal{F}_{\text{ua-voile}}^{\text{com}}$, receiving $(\text{VOLE-output}, \mathbf{y}_B)$ in response. The simulator samples $\hat{\eta}^S \leftarrow \mathbb{Z}_{[2^{2|q|+s}/q]}^\ell$ and sends $\text{ct}_{\mathbf{A}} = \{\text{LHEnc}(\text{ek}, \hat{\eta}_i^S \cdot q - \mathbf{y}_{B,i})\}_{i \in [\ell]}$ to B on behalf of A , and $(\text{proven}, (\text{ek}, \text{ct}_{\mathbf{A}}), 1)$ on behalf of $\mathcal{F}_{\text{zk}}^{A,B}(\text{sid-zk-A}, R_A)$. Observe that in the real protocol, each $\text{LHDec}(\text{dk}, \text{ct}_{\mathbf{A}_i})$ is distributed as $\hat{\eta}_i^R \cdot q - \mathbf{y}_{B,i}$, where $\hat{\eta}_i^R = \lfloor (\eta_i + \mathbf{a}_i \cdot r) / q \rfloor$. Given that $\mathbf{a}_i \cdot r < q^2$ and that η_i is drawn uniformly from $\mathbb{Z}_{2^{2|q|+s}}$, the value $\hat{\eta}_i^R$ is effectively a statistically hiding one-time pad encryption of $\lfloor \mathbf{a}_i \cdot r / q \rfloor$, and it holds that $\hat{\eta}^S$ and $\hat{\eta}^R$ (and hence the real and simulated experiments) are statistically indistinguishable.

Finally, in the case where both A, B are honest, observe that when $\mathcal{A}^*$ intercepts a message from one party to the other via the $\text{adv-input}, \text{adv-output}$ messages, the task of simulation is identical to one of the above two cases. In particular, if $\mathcal{A}^*$ invokes the adv-input interface, simulation is identical to the corrupt B case, and $\mathcal{A}^*$ utilizing the adv-output interface is identical to the corrupt A case. $\square$

4 An Online-Extractable, Committed OPRF

The Ideal OPRF Functionality [Functionality 4.1](#), $\mathcal{F}_{\text{oprif}}^{\text{com}}$, is the committed-input ideal OPRF functionality. The non-committed variant, $\mathcal{F}_{\text{oprif}}$, is the result of omitting the [indicated text](#). Important properties of this functionality are discussed below.

Unauthenticated. This functionality does not assume any form of authenticity; analogously to the real world $\mathcal{A}$, the ideal adversary $\mathcal{A}^*$ can impersonate any party $p \in \mathcal{P}$ via a query $(\text{input-OPRF}, \text{ssid}, [\dots])$. These adversarial input queries can, potentially, be in association with an open subsession ssid , corresponding to man-in-the-middle attacks in the real world. The functionality will accept at most two input queries for each ssid , (ssid, p) from some party p and $(\text{ssid}, \mathcal{A}^*)$ from $\mathcal{A}^*$, but will only notify s of a single input chosen by $\mathcal{A}^*$.

The adversary can choose to deliver output with respect to any one of multiple independent adversarial output tables, corresponding to the real world adversary performing an evaluation instead of an honest s . The input-output behavior for a label lab is recorded in $F[\text{lab}, \cdot]$, where $\text{lab} = s$ is the honest session if s is honest. If an honest s is notified of the input provided by p , then $\mathcal{A}^*$ cannot deliver the output under the honest label to the session containing $\mathcal{A}^*$. For the adversary to deliver output in the honest session, an honest server s must indicate participation in evaluation *for that session*, i.e., there must exist a record (ssid, s) . This requirement is similar to the ticketing mechanism that counts the number of server evaluations in existing functionalities, except that in our functionality the adversary can only use a server response to deliver output in the indicated subsession ssid .

Adaptive Server Compromise. Our functionality models the ability of $\mathcal{A}^*$ to adaptively compromise an honest s , corresponding to real world leakage of the long-term storage of a server but not subsession specific

Functionality 4.1: $\mathcal{F}_{\text{opr}}^{\text{com}}(\text{sid}, \ell_{\text{opr}}, \text{pp}_{\mathbb{G}})$ OPRF.

Functionality $\mathcal{F}_{\text{opr}}^{\text{com}}$ (resp. $\mathcal{F}_{\text{opr}}$) are the below with (res. without) the indicated text.

The functionality $\mathcal{F}_{\text{opr}}^{\text{com}}$ with session identifier $\text{sid} = ([\dots], s)$ is parameterized by the output length ℓ_{opr} , and the group $\text{pp}_{\mathbb{G}} = (\mathbb{G}, G, g)$, interacts with the parties in $\mathcal{P}$ and $\mathcal{A}^*$ via the queries below, and upon invocation receives the set $\mathcal{C}$ of corrupted parties. The functionality maintains an initially undefined table $F[\cdot, \cdot]$. Sessions can be aborted by $\mathcal{A}^*$ at any point.

INITIALIZATION. Initialization of s is only executed once.

1. **On (init-OPRF) from s .** Ignore if (init, s) is recorded. Mark s as **active**. Output **active** to s .
2. **On (OPRF-offline, x) from s .** If $F[s, x]$ is undefined then sample $F[s, x] \leftarrow \{0, 1\}^{\ell_{\text{opr}}}$. Output (output-OPRF, $x, F[s, x]$) to s .

EVALUATION. Evaluation on an input x can be invoked multiple times.

3. **On (input-OPRF, ssid, p, x, r) from $p \in \mathcal{P} \cup \{\mathcal{A}^*\}$.** Ignore the query if there is a record (input-OPRF, ssid, $p', [\dots]$) and either $p, p' \in \{\mathcal{A}^*\}$ or $p, p' \in \mathcal{P}$. If (com-key, H) is not recorded, then sample $H \leftarrow \mathbb{Z}_q$ and create the record. Compute $C = rG + xH$. Record (input-OPRF, ssid, p, x, r, C) and send (input-OPRF, ssid, p, C) to $\mathcal{A}^*$. On (deliver-input, ssid, p) from $\mathcal{A}^*$, if there is no record (deliver-input, ssid, p') with $p' \neq p$ send (input-OPRF, ssid, C) to s and record (deliver-input, ssid, p).
4. **On (evaluate-OPRF, ssid) from $p \in \{s, \mathcal{A}^*\}$.** Ignore if (input-OPRF, ssid, $[\dots]$) is not recorded, if $p = s$ and s is not marked **active**, or if $p = \mathcal{A}^*$ and s is not marked **compromised**. Record (ssid, s). Send (evaluate-OPRF, ssid) to $\mathcal{A}^*$.
5. **On (output-OPRF, ssid, p^*, lab) from $\mathcal{A}^*$.** Ignore if either: (a) there is no record (input-OPRF, ssid, $p^*, x, [\dots]$), (b) s is honest and not marked **compromised**, $\text{lab} = s$ and either (ssid, s) or (deliver-input, ssid, p^*) is not recorded. If $F[\text{lab}, x]$ is undefined sample $F[\text{lab}, x] \leftarrow \{0, 1\}^{\ell_{\text{opr}}}$. Output (output-OPRF, ssid, $F[\text{lab}, x]$) to p^* .

ATTACKS. Can be queried by $\mathcal{A}^*$ at any time.

6. **On (adv-eval, lab, x) from $\mathcal{A}^*$.** If $\text{lab} = s$ and s is neither corrupt nor marked **compromised** then ignore the query. If $F[\text{lab}, x]$ is undefined, sample $F[\text{lab}, x] \leftarrow \{0, 1\}^{\ell_{\text{opr}}}$. Output (output-OPRF, $x, F[\text{lab}, x]$) to $\mathcal{A}^*$.
7. **On (steal-key, s) from $\mathcal{A}^*$.** Ignore the query if s is corrupt or is not marked **active**. Mark s as **compromised**.

AUXILIARY INTERFACES. Can be queried at any time.

9. **On (com-key) from $p \in \mathcal{P} \cup \{\mathcal{A}^*\}$.** If there is no record (com-key, $\cdot$), sample $H \leftarrow \mathbb{G}$ and record (com-key, H). Output the record to p .

ephemeral values. In contrast, during an adaptive corruption the adversary receives the entire state of a newly corrupted party and can subsequently impersonate that party in any interaction. After an adaptive compromise, subsequent queries of `evaluate-OPRF` can be made by $\mathcal{A}^*$ on behalf of the **compromised** server without activating the server, and $\mathcal{A}^*$ can deliver output for to any label. For more discussion on adaptive server compromise, see Section 5.1.

Security of our protocol is proven against a *static* adversary with the ability to adaptively compromise s , reflected in the functionality via the `steal-key` interface.

Online-Extractable. In contrast to all existing OPRF functionalities, $\mathcal{F}_{\text{opr}}$ does not model delayed extraction. Practically, this amounts to $\mathcal{F}_{\text{opr}}$ accepting the queries associated to `ssid` in a fixed, intuitive, order. The server can only query `evaluate-OPRF` for `ssid` *after* the functionality receives a `input-OPRF` query specifying the input x for `ssid`. In contrast, delayed input extraction is modeled when the functionality permits `evaluate-OPRF` queries *before* `input-OPRF` queries, modeling that a session can be completed from the point of view of a server prior to the specification of input by the evaluating party.

As delayed extraction is often inherited (rather, cannot be avoided) by any higher-level protocols written in the $\mathcal{F}_{\text{opr}}\text{-hybrid}$ model, an OPRF with online-extractability is our most impactful contribution.

Committed-Inputs. This functionality additionally outputs a commitment C to each input x when queried for evaluation. To this end, the functionality is additionally parameterized by a group $\text{pp}_{\mathbb{G}} = (\mathbb{G}, G, q)$ and emulates a CRS via the `com-key` interface. Commitment randomness is provided along with inputs, and a commitment is output to $\mathbf{s}$ along with the evaluation prompt.

The Oblivious Psuedorandom Function Protocol Our OPRF protocol $\Pi_{\text{opr}}\text{-prf}$ appears in Figure 4.2. For $\text{pp}_{\mathbb{G}} = (\mathbb{G}, G, q)$, the underlying PRF of Eq 7 was first introduced by Boneh and Boyen as a weakly secure signature scheme [12] under the $Q\text{-SDH}$ assumption, and was later proven to be a verifiable random function (VRF) by Dodis and Yampolskiy [26] for polynomial domain size Q , assuming $Q\text{-DBDHI}$.

$$F(\mathbf{sk}, x) := \left(\frac{1}{\mathbf{sk} + x} \right) G \quad (7)$$

At a High Level. For OPRF input x party $\mathbf{p}$ engages the server $\mathbf{s}$ to compute a blinded value $v_x = u(x + \mathbf{sk})$, and receives the corresponding nonce $U = uG$ and public key $\mathbf{pk} = \mathbf{sk}G$ from $\mathbf{s}$. Then, the intermediate value $Y := (v_x)^{-1}U = (\mathbf{sk} + x)^{-1}G$ and the OPRF output $y := H(x, Y, \mathbf{pk})$ can be computed by $\mathbf{p}$.

To compute v_x the pair first invokes a random VOLE which outputs additive shares $y_x, \bar{y}_x$ of $u\bar{x}$ for random $\bar{x}$. Once $\mathbf{s}$ receives the offset $\delta = x - \bar{x}$ from $\mathbf{p}$, the share $\bar{v}_x = \bar{y}_x + u(\delta + \mathbf{sk})$ such that $\bar{v}_x + y_x = u(x + \mathbf{sk}) = v_x$ is computed and returned to $\mathbf{p}$. This intermediate value v_x can be verified against $(x, U, \mathbf{pk})$ and $W := u\mathbf{pk}$ in the exponent, meaning that $\mathbf{p}$ will abort an evaluation if $xU + W \neq v_xG$ or if the received $(U, \mathbf{pk}, W)$ is not a Diffie-Hellman tuple, as proven with π_{dh} .

Securing against Selective Failure. As described above, the OPRF protocol is vulnerable to a selective failure attack in which a malicious server can force an abort all inputs $x \neq x^*$ for some chosen x^* . To mitigate this selective failure attack, we require that the party $\mathbf{p}$ initiate two OLEs with a each server where the input to the second one $z \leftarrow \mathbb{Z}_q$ is sampled uniformly. To demonstrate the need for this change we describe an attack against the single input version that is available to $\mathcal{A}$ through a server. Below, we consider the vulnerable version of the protocol, i.e., when evaluation is performed with a single OLE (as opposed to two, as is written in Figure 4.2).

In the second round, the server $\mathbf{s}$ receives an additive share $\bar{y}^x = \llbracket u\bar{x} \rrbracket$ of $u\bar{x}$ in addition to the offset $\delta^x := x - \bar{x}$. The server will guess an input x^* , assume that guesses is correct ($x^* = x$), and then:

1. Derandomizes the VOLE by computing the random input $\bar{x}^* := x^* - \delta^x = x^* - (x - \bar{x})$,
2. Computes $\mathbf{p}$'s share of the VOLE: $\llbracket u\bar{x}^* \rrbracket^{\mathbf{p}} := u\bar{x}^* - \bar{y}^x$,
3. Computes its share of the OPRF output $v^* := ux^* - \llbracket u\bar{x}^* \rrbracket^{\mathbf{p}} + u\mathbf{sk}$

If x^* is equal to x then $\bar{x}^* = \bar{x}$. Let y^x be the $\mathbf{p}$'s share of the VOLE. Then, the party computes v^x as

$$v^x := y^x + v^* = y^x + ux^* - \llbracket u\bar{x}^* \rrbracket^{\mathbf{p}} + u\mathbf{sk} = \llbracket u\bar{x}^* \rrbracket^{\mathbf{p}} + ux^* - \llbracket u\bar{x}^* \rrbracket^{\mathbf{p}} + u\mathbf{sk}$$

and the check in Step 9 ($x \times U + W = V$) is satisfied. Otherwise, the evaluation will fail with high probability, and $\mathbf{p}$ aborts.

With the aforementioned modification, i.e., requiring evaluation on two inputs inputs (x, z) for random z , avoids the attack. The check becomes

$$\mathbf{x} \times \mathbf{U} + \mathbf{W} = \mathbf{V}$$

$$\begin{bmatrix} x \\ z \end{bmatrix} \times \begin{bmatrix} uG \\ uG \end{bmatrix} + \begin{bmatrix} uskG \\ uskG \end{bmatrix} = \begin{bmatrix} v^xG \\ v^yG \end{bmatrix}.$$

In order for the check to pass, z must be equal to x which occurs with probability $1/q$ since z is uniformly random.

In summary, to protect against this attack the parties simultaneously compute what is essentially an additional OPRF evaluation on a random input z . An intermediate value $\bar{V}_z = (u(\mathbf{sk} + z) - y_z)G$ analogous to $\bar{v}_x$ is also returned to $\mathbf{p}$ in the exponent. Verification of $\bar{v}_x, \bar{V}_z$ against the same values $U, \mathbf{pk}, W$ ensures

Protocol 4.2: $\Pi_{\text{opr}}(\text{sid}, \ell_{\text{opr}}, \text{pp}_{\mathbb{G}})$ OPRF.

The protocol Π_{opr} is parameterized by the output length ℓ_{opr} and a group $\text{pp}_{\mathbb{G}} = (\mathbb{G}, G, q)$. The protocol is executed between a server s and a party p with access to hybrid functionalities $(\mathcal{F}_{\text{R0}}, \mathcal{F}_{\text{crs}}, \mathcal{F}_{\text{ua-vole}}, \mathcal{F}_{\text{ua-vole}}^{\text{com}})$, where H_{out} is an instance of $\mathcal{F}_{\text{R0}}$ with outputs in $\{0, 1\}^{\ell_{\text{opr}}}$. The protocol uses a $(P_{\text{d1}}, V_{\text{d1}})$ and $(P_{\text{dh}}, V_{\text{dh}})$ which satisfy Definition 1 for the relations $\mathcal{R}_{\text{DLog}}^{\mathbb{G}}, \mathcal{R}_{\text{dht}}^{\mathbb{G}}$.

SERVER'S SIDE.

Server s , on (init-OPRF).

1. Query $H \leftarrow \mathcal{F}_{\text{crs}}(\text{sid})$.
2. Sample $\text{sk} \leftarrow \mathbb{Z}_q$. Compute $\text{pk} := \text{sk}G$. Prove $\pi \leftarrow P_{\text{d1}}((G, \text{pk}), \text{sk})$.
3. Store $(\text{sk}, \widehat{\text{pk}})$ for $\widehat{\text{pk}} := (\text{pk}, \pi)$.

Server s , on (evaluate-OPRF, ssid, p , δ) from $p \in \mathcal{P}$, and

- (sample-VOLE-input, C) from $\mathcal{F}_{\text{ua-vole}}^{\text{com}}((\text{sid}, (\text{ssid}, 1), s, p), \text{pp}_{\mathbb{G}}, H, 1)$,
 - (sample-VOLE-input) from $\mathcal{F}_{\text{ua-vole}}((\text{sid}, (\text{ssid}, 2), s, p), q, 1)$.
4. Sample $u \leftarrow \mathbb{Z}_q$. Define $U := uG$.
 5. $\mathcal{F}_{\text{ua-vole}}^{\text{com}}((\text{sid}, (\text{ssid}, 1), s, p), \text{pp}_{\mathbb{G}}, H, 1)$:
Query (VOLE-multiply, u). Receive (VOLE-output, $\bar{y}_x$).
 6. $\mathcal{F}_{\text{ua-vole}}((\text{sid}, (\text{ssid}, 2), s, p), q, 1)$:
Query (VOLE-multiply, u). Receive (VOLE-output, $\bar{y}_z$).
 7. Define $\bar{v}_x := \bar{y}_x + u(\delta + \text{sk})$.
 8. Define $\bar{V}_z := (\bar{y}_z + u\text{sk})G$.
 9. Define $\text{dht} := (G, U, \text{pk}, u\text{pk})$. Prove $\pi_{\text{dh}} \leftarrow P_{\text{dh}}(\text{dht}, u)$.
 10. Send $(\text{ssid}, \bar{v}_x, \bar{V}_z, \pi_{\text{dh}}, \widehat{\text{pk}})$ to p .

PARTY'S SIDE.

Party p , on (ssid, x).

1. Query $H \leftarrow \mathcal{F}_{\text{crs}}(\text{sid})$.
2. $\mathcal{F}_{\text{ua-vole}}^{\text{com}}((\text{sid}, (\text{ssid}, 1), s, p), \text{pp}_{\mathbb{G}}, H, 1)$:
Query (sample-VOLE-input). Receive (sample-VOLE-input, $\bar{x}, \bar{x}$).
3. $\mathcal{F}_{\text{ua-vole}}((\text{sid}, (\text{ssid}, 2), s, p), q, 1)$:
Query (sample-VOLE-input). Receive (sample-VOLE-input, z).
4. Send (evaluate-OPRF, ssid, p , δ) to s , for $\delta := x - \bar{x}$.

Party p , on (ssid, $\bar{v}_x, \bar{V}_z, \pi_{\text{dh}}, \widehat{\text{pk}}$) from s , and

- (VOLE-output, y_x) from $\mathcal{F}_{\text{ua-vole}}^{\text{com}}((\text{sid}, (\text{ssid}, 1), s, p), \text{pp}_{\mathbb{G}}, H, 1)$,
 - (VOLE-output, y_z) from $\mathcal{F}_{\text{ua-vole}}((\text{sid}, (\text{ssid}, 2), s, p), q, 1)$.
5. Parse $(\text{pk}, \pi) \leftarrow \widehat{\text{pk}}$, $(G, U, \text{pk}, W) \leftarrow \text{dht}$.
 6. Verify $\top = V_{\text{d1}}(\text{pk}, \pi) = V_{\text{dh}}(\text{dht}, \pi_{\text{dh}})$.
 7. Define $v_x := \bar{v}_x + y_x$.
 8. Define $V_z := y_zG + \bar{V}_z$.
 9. If $(x, z)U + (W, W) \neq (v_xG, V_z)$: output (abort, ssid).
 10. Define $y := \text{H}_{\text{out}}(x, Y, \widehat{\text{pk}})$ for $Y := (v_x)^{-1}U$.
 11. Output (output-OPRF, ssid, y).

Party p , on (com-key, p). Query $H \leftarrow \mathcal{F}_{\text{crs}}$. Output (com-key, H).

that the OPRF was computed consistently on both x and z . The share $\bar{V}_z$ is sent in the exponent to protect against leakage of sk , as the two evaluations are computed with the same randomness u .

Unauthenticated. We do not require that parties locally store $\widehat{\text{pk}}$, and therefore parties cannot authenticate the server. Therefore, the adversary can always impersonate s and evaluate the PRF with respect to an arbitrary $\text{pk}_{\mathcal{A}}$. Essentially, any output accepted by an honest party corresponds to some public key. In our

security proof, these public keys are the labels of the adversarial sessions; each public key corresponds to an independent and consistent set of outputs.

Adaptive Server Compromise. Adaptive server compromise is reducible to the underlying security of the PRF. The environment provides an input x to a party for evaluation, and up until the point of a compromise cannot guess the intermediate value $Y = F(\text{sk}, x)$ when sk is sampled by an honest server, even given pk , the pairs $(x_{\text{ssid}}, y_{\text{ssid}} = H(F(x_{\text{ssid}}, Y_{\text{ssid}}, \text{pk})))$ and the evaluation transcripts. Upon adaptive compromise, the simulator merely presents the server’s key.

Online-Extractability. For an evaluation, a corrupt party sends $\delta = x - \bar{x}$ directly to $\mathbf{s}$ over an insecure channel, for random $(\bar{x}, \bar{y}) \leftarrow \mathcal{F}_{\text{ua-vole}}^{\text{com}}$. The server must receive $C \leftarrow \mathcal{F}_{\text{ua-vole}}^{\text{com}}$ and δ from $\mathbf{p}$ to participate in evaluation; at this point $\bar{x}$ and δ are fixed, and uniquely determine the PRF input x .

Committed-Input. The commitment key comes from $H \leftarrow \mathcal{F}_{\text{crs}}$. The OPRF makes fundamental use of our novel, unauthenticated, committed-input, random VOLE, Functionality $\mathcal{F}_{\text{ua-vole}}^{\text{com}}$, parameterized by that commitment key. The committed-input property is inherited from $\mathcal{F}_{\text{ua-vole}}^{\text{com}}$; the input to the VOLE is exactly the input to the OPRF, so the commitment output by the functionality is a commitment to the OPRF input.

4.1 Proof of Security

Computational Assumptions. While we are using the same pseudorandom function defined by Dodis and Yampolskiy, our OPRF has an exponential size domain. One common workaround is to apply a random oracle to inputs and program the oracle to output a predefined set of Q random values. Unfortunately, we cannot use this as $\mathbf{p}$ would have no way of proving to $\mathbf{s}$ that an input is in the range of the random oracle. Further, recall that we want to construct a OPRF with committed input. An evaluating party cannot prove that the commitment C is to the corresponding random oracle input.

Therefore, we reduce to a new assumption that is essentially equivalent to assuming that a signature scheme introduced by Boneh and Boyen⁷ is existentially unforgeable as opposed to merely *weakly* unforgeable, i.e., that security holds under *adaptive* choices of messages. The full assumption is stated in terms of a security game in Definition 3.

Definition 3 (The BB-EUF Assumption). *Let $(\mathbb{G}, G, q)$ be a group, and let $\mathcal{A}$ be a PPT adversary that accesses an oracle, such that it never output (x, Y) where it previously queried the oracle on x . Then the Boneh–Boyen unforgeability assumption for $(\mathbb{G}, G, q)$ says that the following probability must be negligible, for any such $\mathcal{A}$.*

$$\Pr \left[\left(\text{sk} + x \right)^{-1} \cdot G = Y \mid \begin{array}{l} \text{sk} \leftarrow \mathbb{Z}_q \\ \text{pk} := \text{sk} \cdot G \\ x, Y := \mathcal{A}^{(\text{sk} + \cdot)^{-1} G}(\text{pk}) \end{array} \right].$$

Unlike Q -SDH and Q -DBDHI, this is an interactive assumption; but unlike ideal models like GGM, the assumption is still *falsifiable*. We provide a few heuristic arguments for the soundness of this assumption:

1. The assumption is a natural strengthening of the game proven from Q -SDH proven by Boneh and Boyen. That is, if $\mathcal{A}$ is forced to fix the inputs to its signing/evaluation oracle in advance, then it would be equivalent to a weak chosen message forgery against Boneh–Boyen, and so imply an attack against Q -SDH [12].
2. The assumption is easy to prove in the GGM, as 1 , sk , and $(\text{sk} + x)^{-1}$ for distinct values of x are all $\mathbb{F}_q$ -linearly independent rational functions. In particular, extending [7, Theorem 5] to rational functions (by multiplying through by the LCM of the denominators to convert them into polynomials) gives an upper bound of $O((Q + Q')^2 \cdot Q/q)$ if $\mathcal{A}$ makes Q queries to the evaluation oracle and Q' queries to the generic group.

⁷ We note that the scheme we mention here is the “Weakly Secure Short Signature Scheme” found in Section 3.1 of [12] which is different from the *Boneh–Boyen signature scheme* that is introduced in the beginning of Section 3 of [12].

3. Over composite order groups, Chase and Meiklejohn proved that Dodis–Yampolskiy is a PRF over its entire domain, assuming subgroup hiding [22, Theorem 3.2]. This nearly implies our assumption over such groups: ignoring pk , this shows that Y should be indistinguishable from uniform, and so should be hard to guess. A straightforward extension of their argument implies that Y and the PRF output are jointly indistinguishable from random, which then directly implies Definition 3.
4. Finally, the function (Eq 7) has been studied for more than 20 years, and the only known nontrivial attacks on it match Boneh and Boyen’s GGM lowerbound [23, 40].⁸

Theorem 2. *Assuming that (P_{d1}, V_{d1}) and (P_{dh}, V_{dh}) satisfy Definition 1 for the relations $\mathcal{R}_{\text{DLog}}, \mathcal{R}_{\text{dht}}$, the protocol $\Pi_{\text{opr}}^{\text{com}}$ (Protocol 4.2) UC-realizes the functionality $\mathcal{F}_{\text{opr}}^{\text{com}}$ (Functionality 4.1) in the $(\mathcal{F}_{\text{RO}}, \mathcal{F}_{\text{crs}}, \mathcal{F}_{\text{ua-vole}}, \mathcal{F}_{\text{ua-vole}}^{\text{com}})$ -hybrid model in the presence of a static malicious adversary capable of adaptive compromise, under the Boneh-Boyen Unforgeability Assumptions.*

Figure 3. $\mathcal{S}_{\text{OPRF}}(\text{sid}, \ell_{\text{opr}}, \text{pp}_{\mathbb{G}})$

The simulator for Π_{opr} with session identifier $\text{sid} = ([\dots], s)$ is parameterized by the output length ℓ_{opr} and a group $\text{pp}_{\mathbb{G}} := (\mathbb{G}, G, q)$. The simulator is given oracle access to the real-world adversary $\mathcal{A}$ and routes all messages sent by the ideal functionalities and simulated honest parties through $\mathcal{A}$ for delivery. As in Π_{opr} , there is an active instance of $\mathcal{F}_{\text{RO}}$, denoted H_{out} , and two additional instances of $\mathcal{F}_{\text{RO}}$, denoted $H_{\text{zk-d1}}$ and $H_{\text{zk-dht}}$ which are programmed by the NIZK simulators $\mathcal{S}_{\text{zk-d1}}$ and $\mathcal{S}_{\text{zk-dh}}$, respectively.

Upon Invocation.

The simulator $\mathcal{S}_{\text{OPRF}}$ is initialized with $\text{sid} = ([\dots], s)$ that identifies the server $s \in \mathcal{P}$ associated with the instance of $\mathcal{F}_{\text{opr}}^{\text{com}}$. Upon invocation, $\mathcal{S}_{\text{OPRF}}$ executes the following instructions:

- Initialize tables $H_{\text{out}}[\cdot]$, $\text{OPRF}[\cdot]$, and $\text{mitm}[\cdot]$.
- Compute $(H_{\text{zk-d1}}[\cdot], \sigma_{d1}) \leftarrow \mathcal{S}_{\text{zk-d1}}(\kappa)$ for $\mathcal{S}_{\text{zk-d1}} = (\mathcal{S}_{\text{zk-d1}}^0, \mathcal{S}_{\text{zk-d1}}^1)$.
- Compute $(H_{\text{zk-dht}}[\cdot], \sigma_{dh}) \leftarrow \mathcal{S}_{\text{zk-dh}}^0(\kappa)$, for $\mathcal{S}_{\text{zk-dh}} = (\mathcal{S}_{\text{zk-dh}}^0, \mathcal{S}_{\text{zk-dh}}^1)$.
- Send (com-key) to $\mathcal{F}_{\text{opr}}^{\text{com}}$. Receive and record $(\text{com-key}, H)$.

OPRF.

Notation. For ease of notation we denote:

$\mathcal{F}_{\text{opr}}^{\text{com}}(\text{sid}, \ell_{\text{opr}}, \text{pp}_{\mathbb{G}})$ by $\mathcal{F}_{\text{opr}}^{\text{com}}$,
 $\mathcal{F}_{\text{ua-vole}}^{\text{com}}((\text{sid}, (\text{ssid}, 1), s, p), \text{pp}_{\mathbb{G}}, H, 1)$ by $\mathcal{F}_{\text{ua-vole}}^{\text{com}}(\text{ssid}, 1, s, p)$, and
 $\mathcal{F}_{\text{ua-vole}}^{\text{com}}((\text{sid}, (\text{ssid}, 2), s, p), q, 1)$ by $\mathcal{F}_{\text{ua-vole}}^{\text{com}}(\text{ssid}, 2, s, p)$.

Initialization of Server s.

If s is honest:

1. Sample $\text{sk} \leftarrow \mathbb{Z}_q$, compute $\text{pk} := \text{sk}G$.
2. Compute $(H_{\text{zk-d1}}, \pi) \leftarrow \mathcal{S}_{\text{zk-d1}}^1(\sigma_{d1}, \text{pk}, 1)$.
3. Define $\widehat{\text{pk}} := (\text{pk}, \pi)$.
4. Record $(\text{keys}, \text{sk}, \widehat{\text{pk}})$.

If s is corrupted:

1. Query $\mathcal{F}_{\text{opr}}^{\text{com}}$ with (init-OPRF) .
2. Receive **active**.

Evaluation Request by Corrupted $p \in \mathcal{C}$,

or $\mathcal{A}$ On Behalf of Honest $p \in \mathcal{P} \setminus \mathcal{C}$.

Await activations 1-3 and respond as indicated, then proceed.

1. On **(sample-VOLE-input)** to $\mathcal{F}_{\text{ua-vole}}^{\text{com}}(\text{ssid}, 1, s, p)$ from $P \in \{\mathcal{A}, p\}$:
 - Send **(sample-VOLE-input)** to $\mathcal{A}$.
 - Receive **(sample-VOLE-input, $\bar{x}^*, r^*$)**.
 - Compute $C^* := r^*G + \bar{x}^*H$.

⁸ These attacks exploit prime orders q where $q-1$ or $q+1$ have a factor of the right size — around $q^{1/3}$.

- Send (sample-VOLE-input, $\overline{x^*}, r^*, \overline{C^*}$) to P .
 - 2. On (sample-VOLE-input) to $\mathcal{F}_{\text{ua-voles}}(\text{ssid}, 2, s, p)$ from $P \in \{\mathcal{A}, p\}$:
 - Send (sample-VOLE-input) to $\mathcal{A}$.
 - Receive (sample-VOLE-input, z^*).
 - Send (sample-VOLE-input, z^*) to P .
 - 3. On $m_1^* = (\text{evaluate-OPRF}, \text{ssid}, p, \delta^*)$ to s :
 - Record the message.
- After 1-3 above:
- Compute $x^* := \delta^* + \overline{x^*}$.
 - With $\mathcal{F}_{\text{oprf}}^{\text{com}}$:
 - Set $p^* = p$ if $p \in \mathcal{C}$ and otherwise set $p^* = \mathcal{A}$.
 - Query (input-OPRF, ssid, p^*, x^*, r^*).
 - Receive (input-OPRF, ssid, p^*, C^*).
 - Record (ssid, $p, (P, x^*, C^*), (\overline{x^*}, r^*, \overline{C^*}), z^*, \delta^*$). *This records all values corresponding to the first round of the protocol for the p-side of ssid.*
 - See *Adversarial Routing of Round One Messages* below.

Evaluation Request by an Honest p.

On (input-OPRF, ssid, p, C) from $\mathcal{F}_{\text{oprf}}^{\text{com}}$:

1. Sample $z, \delta \leftarrow \mathbb{Z}_q$. Define $\overline{C} := C - \delta H$.
2. Set $\text{mitm}[\text{ssid}] \leftarrow 0$.
3. Record (ssid, $p, (p, \perp, C), (\perp, \perp, \overline{C}), z, \delta$). *This records all values corresponding to the first round of the protocol for the p-side of ssid.*
4. Emulate $\mathcal{F}_{\text{ua-voles}}^{\text{com}}(\text{ssid}, 1, s, p)$:
 - Send (sample-VOLE-input, $\overline{C}$) to $\mathcal{A}$.
5. Emulate $\mathcal{F}_{\text{ua-voles}}(\text{ssid}, 2, s, p)$:
 - Send (sample-VOLE-input) to $\mathcal{A}$.
6. Send $m_1 := (\text{evaluate-OPRF}, \text{ssid}, p, \delta)$ to s from p via $\mathcal{A}$.
7. See *Adversarial Routing of Round One Messages* below.

Adversarial Routing of Round One Messages.

For the record (ssid, $p, (P, x, C), (\overline{x}, r, \overline{C}), z, \delta$)

await activations 1-3, then proceed.

1. Emulate $\mathcal{F}_{\text{ua-voles}}^{\text{com}}(\text{ssid}, 1, s, p)$ and accept one of (1a, 1b) below:
 - (a) On (notify- $\mathcal{A}$) from $\mathcal{A}$:
 - Define $\overline{C^*} := \overline{C}$, $\overline{x^*} := \overline{x}$, $r^* := r$.
 - If s is corrupt: send (sample-VOLE-input, $\overline{C}$) to s .
 - (b) On (adv-input, $\overline{x^*}, r^*$) from $\mathcal{A}$:
 - Compute $\overline{C^*} := r^* G + \overline{x^*} H$.
 - If s is corrupt send (sample-VOLE-input, $\overline{C^*}$) to s .
2. Emulate $\mathcal{F}_{\text{ua-voles}}(\text{ssid}, 2, s, p)$ and accept one of (2a, 2b) below:
 - (a) On (notify- $\mathcal{A}$) from $\mathcal{A}$:
 - Define $z^* := z$.
 - If s is corrupt send (sample-VOLE-input) to s .
 - (b) On (adv-input, z^*) from $\mathcal{A}$:
 - If s is corrupt send (sample-VOLE-input) to s .
3. For message m_1 accept one of (3a, 3b) below:
 - (a) If $\mathcal{A}$ permits delivery of m_1 :
 - Define $\delta^* := \delta$.
 - If s is corrupt send m_1 to s .
 - (b) Else if $\mathcal{A}$ replaces m_1 with $m_1^* = (\text{evaluate-OPRF}, \text{ssid}, p, \delta^*)$:

- If s is corrupt send m_1^* to s .

After 1-3 above:

- Compute server session values:
 - Define $C^* := \overline{C^*} + \delta^* H$.
 - If $\overline{x^*} \neq \perp$: define $x^* := \overline{x^*} + \delta^*$.
 - Else: Set $x^* := \perp$.
- If $x^* \neq x$:
 - Set $\text{mitm}[\text{ssid}] := 1$ and $P^* := \mathcal{A}^*$.
 - Query $\mathcal{F}_{\text{opr}}^{\text{com}}$ with $(\text{input-OPRF}, \text{ssid}, P^*, x^*, r^*)$.
- Else: set $\text{mitm}[\text{ssid}] := 0$ and $P^* := p$.
- Record $(\text{ssid}, s, (P^*, x^*, C^*), (\overline{x^*}, r^*, \overline{C^*}), z^*, \delta^*)$. *This records all values corresponding to the first round of the protocol for the s-side of ssid.*
- Query $\mathcal{F}_{\text{opr}}^{\text{com}}$ with $(\text{deliver-input}, \text{ssid}, P^*)$.
- If s is honest then see *Evaluation Response from Honest s* below.
- Else, see *Evaluation Response from Corrupt s* below.

Evaluation Response from Corrupt s.

For record $(\text{ssid}, s, (P, x, C), (\overline{x}, r, \overline{C}), z, \delta)$

await activations 1-3, then proceed.

1. On $(\text{VOLE-multiply}, u_x^*)$ to $\mathcal{F}_{\text{ua-vole}}^{\text{com}}(\text{ssid}, 1, s, p)$ from s :
 - Send $(\text{VOLE-multiply}, s)$ to $\mathcal{A}$.
 - Receive $(\text{VOLE-output}, \overline{y_x^*})$ from $\mathcal{A}$.
 - Send $(\text{VOLE-output}, \overline{y_x^*})$ to s .
2. On $(\text{VOLE-multiply}, u_z^*)$ to $\mathcal{F}_{\text{ua-vole}}^{\text{com}}(\text{ssid}, 2, s, p)$:
 - Send $(\text{VOLE-multiply}, s)$ to $\mathcal{A}$.
 - Receive $(\text{VOLE-output}, \overline{y_z^*})$ from $\mathcal{A}$.
 - Send $(\text{VOLE-output}, \overline{y_z^*})$ to s .
3. On $m_2^* := (\text{ssid}, \overline{v_x^*}, \overline{V_z^*}, \text{dht}^*, \pi_{\text{dht}}^*, \widehat{\text{pk}}^*)$ to p : record the message.

After 1-3 above:

- Record $(\text{ssid}, s, (u_x^*, \overline{y_x^*}, \perp), (u_z^*, \overline{y_z^*}, \perp), m_2^*)$. *This records all values corresponding to the second round of the protocol for the s-side of ssid.*
- See *Adversarial Routing of Round Two Messages*.

Evaluation Response from Honest s.

On $(\text{evaluate-OPRF}, \text{ssid})$ from $\mathcal{F}_{\text{opr}}^{\text{com}}$:

1. Retrieve records:
 - $(\text{keys}, \text{sk}, \widehat{\text{pk}})$,
 - $(\text{ssid}, s, (P, x, C), (\overline{x}, r, \overline{C}), z, \delta)$.
2. Emulate $\mathcal{F}_{\text{ua-vole}}^{\text{com}}(\text{ssid}, 1, s, p)$:
 - Send $(\text{VOLE-multiply}, s)$ to $\mathcal{A}$.
 - If $\overline{x} \neq \perp$: Receive $(\text{VOLE-output}, y_x)$ from $\mathcal{A}$.
 - Else: Sample $y_x \leftarrow \mathbb{Z}_q$.
3. Emulate $\mathcal{F}_{\text{ua-vole}}^{\text{com}}(\text{ssid}, 2, s, p)$:
 - Send $(\text{VOLE-multiply}, s)$ to $\mathcal{A}$.
 - If $p \in \mathcal{C}$ or z was received from $\mathcal{A}$ via a query of adv-input :
Receive $(\text{VOLE-output}, y_z)$ from $\mathcal{A}$.
 - Else: Sample $y_z \leftarrow \mathbb{Z}_q$.
4. Server response message:
 - (a) Sample $a \leftarrow \mathbb{Z}_q$.
 - (b) If $x = \perp$ sample $x \leftarrow \mathbb{Z}_q$.

i.e., if $p \notin \mathcal{C} \cup \{\mathcal{A}^\}$ and $\text{mitm}[\text{ssid}] = 0$.*

- (c) Compute $Y := \left(\frac{1}{s_k+x}\right)G$.
 - (d) Compute $U := aY$, and $W := aG - xU$.
 - (e) Define $\text{dht} := (G, U, \text{pk}, W)$.
 - (f) Simulate $(H_{\text{zk-dht}}, \pi_{\text{dh}}) \leftarrow S_{\text{zk-dh}}^1(\sigma_{\text{dh}}, \text{dht})$.
 - (g) Compute $\overline{v_x} := a - y_x$ and $\overline{V_z} := zU + W - y_zG$.
 - (h) If $\text{mitm}[\text{ssid}] = 1$ or $p \in \mathcal{C}$:
 - Query $\mathcal{F}_{\text{opr}}^{\text{com}}$ with $(\text{output-OPRF}, \text{ssid}, \mathcal{A}^*, s)$.
 - Receive $(\text{output-OPRF}, \text{ssid}, y)$.
 - Program $H_{\text{out}}[x, Y, \widehat{\text{pk}}] := y$.
 - (i) Send $m_2 := (\text{ssid}, \overline{v_x}, \overline{V_z}, \text{dht}, \pi_{\text{dh}}, \widehat{\text{pk}})$ from s to p .
5. Record $(\text{ssid}, s, (\frac{a}{s_k+x}, \perp, y_x), (\frac{a}{s_k+x}, \perp, y_z), m_2)$. *This records all values corresponding to the second round of the protocol for the s-side of ssid.*
6. See *Adversarial Routing of Round Two Messages*.

At this point, the adversary can compute the output for any session for which it provided the input. This includes impersonation of an honest party, sessions including a corrupted party, and the server side of a man in the middle session.

Adversarial Routing of Round Two Messages.

For the records:

- $(\text{ssid}, p, (P, x, C), (\overline{x}, r, \overline{C}), z, \delta)$
- $(\text{ssid}, s, (P^*, x^*, C^*), (\overline{x^*}, r^*, \overline{C^*}), z^*, \delta^*)$, and
- $(\text{ssid}, s, (u_x, \overline{y_x}, y_x), (u_z, \overline{y_z}, y_z), m_2)$,

await activations 1-3, then proceed.

1. Emulate $\mathcal{F}_{\text{ua-vole}}^{\text{com}}(\text{ssid}, 1, s, p)$ and accept one of (1a, 1b) below:
 - (a) On (deliver-output) from $\mathcal{A}$:
 - Ignore the query if there was a prior query of adv-input .
 - Define $(u_x^*, \overline{y_x^*}, y_x^*) := (u_x, \overline{y_x}, y_x)$.
 - If p is corrupt send $(\text{VOLE-output}, y_x)$ to p .
 - (b) On (adv-output, $u_x^*, \overline{y_x^*}$) from $\mathcal{A}$:
 - Set $\text{mitm}[\text{ssid}] := 1$.
 - If p is corrupt send $(\text{VOLE-output}, \overline{x}u_x^* - \overline{y_x^*})$ to p .
2. Emulate $\mathcal{F}_{\text{ua-vole}}(\text{ssid}, 2, s, p)$ and accept one of (2a, 2b) below:
 - (a) On (deliver-output) from $\mathcal{A}$:
 - Ignore the query if there was a prior query of adv-input .
 - Define $(u_z^*, \overline{y_z^*}, y_z^*) := (u_z, \overline{y_z}, y_z)$.
 - If p is corrupt send $(\text{VOLE-output}, y_z)$ to p .
 - (b) On (adv-output, $u_z^*, \overline{y_z^*}$) from $\mathcal{A}$:
 - Set $\text{mitm}[\text{ssid}] := 1$.
 - If p is corrupt send $(\text{VOLE-output}, zu_z^* - \overline{y_z^*})$ to p .
3. For message m_2 accept one of (3a, 3b) below:
 - (a) If $\mathcal{A}$ permits delivery of m_2 :
 - Define $m_2^* := m_2$, $P' := s$.
 - If p is corrupt send m_2 to p .
 - (b) Else if $\mathcal{A}$ replaces m_2 with $m_2^* = (\text{ssid}, \overline{v_x^*}, \overline{V_z^*}, \text{dht}^*, \pi_{\text{dh}}^*, \widehat{\text{pk}}^*)$:
 - Set $\text{mitm}[\text{ssid}] := 1$, and $P' := \mathcal{A}$.

After 1-3 above:

- Record $(\text{ssid}, p, (u_x^*, \overline{y_x^*}, y_x^*), (u_z^*, \overline{y_z^*}, y_z^*), (P', m_2^*))$. *This records all values corresponding to the second round of the protocol for the p-side of ssid.*
- If p is honest, see *Output Computation for Honest p*.

Output Computation for Honest p.

For the records

- $(\text{ssid}, p, (\perp, C), (\perp, \perp, \overline{C}), z, \delta)$,
 - $(\text{ssid}, p, (u_x, \overline{y}_x, \perp), (u_z, \overline{y}_z, \perp), (P^*, (\text{ssid}, \overline{v}_x, \overline{V}_z, \text{dht}, \pi_{\text{dht}}, (\text{pk}^*, \pi^*))))$,
- query $\mathcal{F}_{\text{opr}}^{\text{com}}$ as below:

1. Query (output-OPRF, ssid, p, s) if all of the following conditions hold:
 - If s is honest
 - s is not marked **compromised**, and
 - $\text{mitm}[\text{ssid}] = 0$.
2. Else, query (abort, ssid, p) if any of the following conditions hold:
 - $\perp \leftarrow \text{Ext}_{\text{dh}}(\text{dht}, \pi_{\text{dh}}, H_{\text{zk-dht}})$ or $\perp \leftarrow \text{Ext}_{\text{d1}}((G, \text{pk}^*), \pi^*, H_{\text{zk-d1}})$,
 - $(G, uG, \text{pk}^*, u\text{pk}^*) \neq \text{dht}$ for $u \leftarrow \text{Ext}_{\text{dh}}(\text{dht}, \pi_{\text{dh}}, H_{\text{zk-dht}})$,
 - $\text{sk}^* G \neq \text{pk}^*$ for $\text{sk}^* \leftarrow \text{Ext}_{\text{d1}}((G, \text{pk}^*), \pi^*, H_{\text{zk-d1}})$,
 - $u \neq u_x, u_x \neq u_z, \overline{v}_x \neq \overline{y}_x + u_x(\delta + \text{sk}^*)$, or $\overline{V}_z \neq (\overline{y}_z + u_z \text{sk}^*) G$.
3. Else if s is honest and $(\text{pk}^*, \pi^*) = \widehat{\text{pk}}$ for the record $(\text{keys}, \text{sk}, \widehat{\text{pk}})$:
 (output-OPRF, ssid, p, s).
4. Else: Query (output-OPRF, ssid, p, (pk^*, π^*)).

ATTACKS.

On (steal-key, s) from $\mathcal{A}$: mark s as **compromised** and send sk to $\mathcal{A}$.

HYBRID FUNCTIONALITIES.

On (rand, z) from $p^* \in \{\mathcal{A}\} \cup \mathcal{C}$ to H_{out} :

- If $H_{\text{out}}[z]$ is defined then set $y := H_{\text{out}}[z]$.
- Else if the input is of the format $z = (x^*, Y^*, \widehat{\text{pk}}^*) \in \mathbb{Z}_q \times \mathbb{G}^2 \times \{0, 1\}^*$, and $Y^* = \left(\frac{1}{(\text{sk}^* + x^*)}\right) G$ for $\text{sk}^* \leftarrow \text{Ext}_{\text{d1}}((G, \text{pk}^*), \pi^*, H_{\text{zk-d1}})$ and $\widehat{\text{pk}}^* = (\text{pk}^*, \pi^*)$:
 - If s is active, not **compromised**, and $\widehat{\text{pk}}^* = \widehat{\text{pk}}$ for record $(\text{keys}, \text{sk}, \widehat{\text{pk}})$:
 output **error** and halt simulation.
 - Send (adv-eval, $(\text{pk}^*, \pi^*), x^*$) to $\mathcal{F}_{\text{opr}}^{\text{com}}$.
 - Receive (output-OPRF, x^*, y).
 - Define $H_{\text{out}}[z] := y$.
- Else: sample $H_{\text{out}}[z] := y \leftarrow \{0, 1\}^{\ell_{\text{opr}}}$.
- Output (rand, z, y) to p^* .

simulation fails.

On (CRS) to $\mathcal{F}_{\text{crs}}$ from $p^* \in \mathcal{C} \cup \{\mathcal{A}\}$:

For the record (com-key, H) output (CRS, H) to p^* .

Proof. Hybrid 0. The Real World. In the real world, the server s and the parties in $\mathcal{P}$ execute Π_{opr} with access to hybrid functionalities $(\mathcal{F}_{\text{R0}}, \mathcal{F}_{\text{crs}}, \mathcal{F}_{\text{ua-vole}}, \mathcal{F}_{\text{ua-vole}}^{\text{com}})$ in the presence of the real-world (dummy) adversary $\mathcal{A}$:

$\text{REAL}_{\Pi_{\text{opr}}, \mathcal{A}, \mathcal{Z}}$

Hybrid 1. The Ideal Layout. In this hybrid, we introduce a simulator $\mathcal{S}$, that emulates the honest parties and hybrid functionalities as per their code in an execution of the protocol with the dummy adversary $\mathcal{A}$, forwards all messages between $\mathcal{Z}$ and $\mathcal{A}$ without modification, and records all of the messages for each ssid. We also introduce dummy parties and a functionality $\mathcal{F}$ between $\mathcal{Z}$ and $\mathcal{S}$ that, for now, both forward the inputs provided by $\mathcal{Z}$ to $\mathcal{S}$. Over the following hybrids we change $\mathcal{S}$ to that in Figure 3 and $\mathcal{F}$ to that in Figure 4.1. The environment's view is unchanged as the differences with Hybrid 0 are exclusively syntactic;

Hybrid 1 is perfectly indistinguishable from Hybrid 0.

Hybrid 2. Corrupt Party Input Queries. In this hybrid, the simulator $\mathcal{S}$ queries $\mathcal{F}_{\text{opr}}^{\text{com}}$ with $(\text{input-OPRF}, \text{ssid}, x^*, r^*)$ on behalf of any corrupted parties. These queries are computed by $\mathcal{S}$ from the messages received from $\mathcal{A}$ on behalf of $p \in \mathcal{P}$, as in *Evaluation Request by Corrupted p* of $\mathcal{S}_{\text{OPRF}}$. The simulator continues to emulate $\mathcal{F}_{\text{ua-vole}}^{\text{com}}((\text{sid}, (\text{ssid}, 1), s, p), \text{pp}_{\mathcal{G}}, H, 1)$ and $\mathcal{F}_{\text{ua-vole}}((\text{sid}, (\text{ssid}, 1), s, p), q1)$, and thus can extract the values $\overline{x^*}$ and $\overline{r^*}$ for ssid . Upon receipt of the corresponding offset δ^* for ssid , $\mathcal{S}$ computes the corrupt input as $x^* := \delta^* + \overline{x^*}$. In this hybrid, $\mathcal{S}$ additionally creates the corresponding record

$$(\text{ssid}, p, (P, x^*, C^*), (\overline{x^*}, \overline{r^*}, \overline{C^*}), z^*, \delta^*)$$

that stores values in the view of $\mathcal{A}$ on the p -side of subsession ssid . In terms of record keeping, $\mathcal{S}$ also stores these values for ssid involving honest p from the values leaked by $\mathcal{F}$ similarly to the manner above. In the case that p is corrupted, all of these values are defined. This change syntactic and Hybrid 2 is perfectly indistinguishable from Hybrid 1.

Hybrid 3. Impersonation & Man in the Middle Attack Input Queries. In this hybrid, $\mathcal{S}$ queries $\mathcal{F}_{\text{opr}}^{\text{com}}$ with $(\text{input-OPRF}, \text{ssid}, \mathcal{A}^*, x^*, r^*)$ on behalf of the adversary for any protocol subsession ssid against which $\mathcal{A}$ mounted a man in the middle attack, or in the case that $\mathcal{A}$ sends the first round message of the protocol on behalf of an honest $p \in \mathcal{P}$. This query is computed similarly to the corrupt input queries above: as $\mathcal{F}_{\text{ua-vole}}^{\text{com}}$, $\mathcal{S}$ receives $(\text{adv-input}, x^*, r^*)$ from $\mathcal{A}$ and, as the honest s , receives $m_1^* = (\text{evaluate-OPRF}, \text{ssid}, p, \delta^*)$ from $\mathcal{A}$, then computes $x^* := \overline{x^*} + \delta^*$. In this hybrid, $\mathcal{S}$ also records

$$(\text{ssid}, s, (P, x^*, C^*), (\overline{x^*}, \overline{r^*}, \overline{C^*}), z^*, \delta^*)$$

which stores values in the view of $\mathcal{A}$ on the s -side of subsession ssid . In the event that $\mathcal{A}$ did not mount a man in the middle, this same record is created with the same values as the p -side of ssid stored in Hybrid 2. This change is syntactic and Hybrid 3 is perfectly indistinguishable from Hybrid 2.

Hybrid 4. Simulated Proofs in Honest Server Keys. In this hybrid, if s is honest then $\mathcal{S}$ invokes the zero-knowledge simulator $\mathcal{S}_{\text{zk-dl}} = (\mathcal{S}_{\text{zk-dl}}^0, \mathcal{S}_{\text{zk-dl}}^1)$ from Definition 1 to simulate the NIZK in public key $\widehat{\text{pk}} = (\text{pk}, \pi)$. More specifically, as in *Initialization of an Honest Server* Step 2 of $\mathcal{S}_{\text{OPRF}}$, this $\mathcal{S}$ computes $\pi \leftarrow \mathcal{S}_{\text{zk-dl}}^1(\sigma_{\text{dl}}, \text{pk}, 1)$ for $(H_{\text{zk-dl}}, \sigma_{\text{dl}}) \leftarrow \mathcal{S}_{\text{zk-dl}}^0(\kappa)$. By zero-knowledge of the NIZK (Definition 1), Hybrid 4 is computationally indistinguishable from Hybrid 3.

Hybrid 5. Simulated Proofs in Honest Server Protocol Messages. In this hybrid, if s is honest then $\mathcal{S}$ again invokes the zero-knowledge simulator $\mathcal{S}_{\text{zk-dh}} = (\mathcal{S}_{\text{zk-dh}}^0, \mathcal{S}_{\text{zk-dh}}^1)$ from Definition 1 to simulate the proofs output by s during evaluation that the tuple dht is a Diffie-Hellman tuple. As in Step 4f of *Evaluation Response from Honest s* of $\mathcal{S}_{\text{OPRF}}$, this $\mathcal{S}$ computes $(H_{\text{zk-dht}}, \pi_{\text{dh}}) \leftarrow \mathcal{S}_{\text{zk-dh}}^1(\sigma_{\text{dh}}, \text{dht}, 1)$ for $(H_{\text{zk-dht}}, \sigma_{\text{dh}}) \leftarrow \mathcal{S}_{\text{zk-dh}}^0(\kappa)$ executed during initialization. Here, the outputs of the instance of $\mathcal{F}_{\text{R0}}$ recorded in table $H_{\text{zk-dht}}$ are programmed by $\mathcal{S}_{\text{zk-dh}}^1$. By the zero-knowledge of the NIZK (Definition 1), Hybrid 5 is computationally indistinguishable from Hybrid 4.

Hybrid 6. Simulated Honest Server Protocol Messages. In this hybrid, when $\mathcal{S}$ computes the honest server response message m_2 , it follows the instructions of $\mathcal{S}_{\text{OPRF}}$ in *Evaluation Response from Honest s* Step 4. $\mathcal{S}$ stores all values corresponding to s -side of subsession ssid in the record

$$\left(\text{ssid}, s, \left(u_x = \left(\frac{a}{\text{sk} + x} \right), \overline{y_x}, y_x \right), \left(u_z = \left(\frac{a}{\text{sk} + x} \right), \overline{y_z}, y_z \right), m_2 \right)$$

for which all values are defined in Hybrid 6 as it still receives the leakage of the honest party inputs from $\mathcal{F}$.

In Hybrid 5, the values of m_2 are computed according to the code of an honest s . Specifically, in Hybrid 6, $\mathcal{S}$ computes m_2 such that the values are dependent upon $F_x = (\frac{1}{sk+x})G$ as opposed to sk directly.

Hybrid 5	$\sim$	Hybrid 6
$(G, U = uG, pk, W = u \cdot pk)$	dht	$(G, U = aF_x, pk, W = aG - xU)$
$\bar{y}_x + u(\delta + sk)$	$\bar{v}_x$	$a - y_x$
$(\bar{y}_z + usk)G$	$\bar{V}_z$	$zU + W - y_zG$

The change to m_2 is essentially setting $a = u(sk + x)$ (and so $u = \frac{a}{sk+x}$), and sampling $a \leftarrow \mathbb{Z}_q$ instead of $u \leftarrow \mathbb{Z}_q$, then applying algebra and rewriting in terms of the VOLE's output to the client party. These changes are statistically indistinguishable.

In more detail, we have that

$$W = aG - xU = a(G - xF_x) = a\left(1 - \frac{x}{sk+x}\right)G = \frac{a \cdot sk}{sk+x}G = skuG = skU,$$

so dht is still a valid Diffie-Hellman tuple. The $\bar{v}_x$ and $\bar{V}_z$ are computed uniquely from the outputs y_x and y_z of the emulated instances of $\mathcal{F}_{ua-vole}^{com}$ and $\mathcal{F}_{ua-vole}$ to p such that verification, in the case that p is corrupted, still passes:

$$\begin{aligned} aG &= xaF_x + aG - xaF_x = xU + W \stackrel{?}{=} v_xG = (\bar{v}_x + y_x)G = (a - y_x + y_x)G = aG \\ zU + W &\stackrel{?}{=} y_zG + \bar{V}_z = y_zG + (zU + W - y_zG) = zU + W \end{aligned}$$

Therefore, the adversary's view contains the same values as before (as a function of u), and the only significant change is that u is now sampled in terms of a fresh value a . Therefore, Hybrid 6 is statistically indistinguishable from Hybrid 5.

Hybrid 7. Honest Server Output to Honest Parties, and Adaptive Server Compromise. In this hybrid, when s is honest and a protocol subsession $ssid$ outputs a correct evaluation under the recorded public key $\widehat{pk}$, $\mathcal{S}$ instructs $\mathcal{F}$ to deliver output under the label s via a query $(\text{output-OPRF}, ssid, p, s)$. Instead of forwarding output directly from $\mathcal{S}$ to p , $\mathcal{F}$ records and outputs a uniformly sampled output to p . We consider two cases.

The first case is a subsession between the honest s and a corrupted p , or $\mathcal{A}$ via an impersonation or man-in-the-middle attack. At the end of the protocol execution for $ssid$, $\mathcal{S}$ receives an input $z^* = (x^*, (\frac{1}{sk+x^*})G, \widehat{pk})$ to H_{out} and must program $H_{out}[z^*]$ to match the value y output by $\mathcal{F}$. If p is corrupted, $\mathcal{S}$ queries $(\text{output-OPRF}, ssid, p, s)$ and receives $(\text{output-OPRF}, ssid, y)$. If p is honest and the query to H_{out} corresponds to a man-in-the-middle session or an impersonation then $\mathcal{S}$ queries $(\text{output-OPRF}, ssid, \mathcal{A}, s)$ and receives $(\text{output-OPRF}, ssid, y)$.

The second case is a subsession between the honest s and an honest p . In this case, $\mathcal{S}$ does not have to program H_{out} until after an adaptive server compromise, which is then treated as the first case. In the event of a previous adaptive server compromise, meaning that s is now marked as **compromised**, $\mathcal{S}$ programs H_{out} from the response of $\mathcal{F}$ to a query $(\text{adv-eval}, s, x)$. Prior to an adaptive compromise, the view of $\mathcal{Z}$ includes the inputs x , the honest public key $\widehat{pk}$, the protocol transcript trans , and the views of corrupted p^* in corrupt subsessions $ssid^*$.

$$\text{trans} = (ssid^*, p, s, \bar{C}, \delta, m_2 = (\bar{v}_x, \bar{V}_z, \text{dht}, \pi_{dh}, \widehat{pk})),$$

Hybrid 7 is different from Hybrid 6 when the output of H_{out} is inconsistent with the output of the functionality $\mathcal{F}$. This occurs when the environment $\mathcal{Z}$ first makes a query of the form $z^* = (x^*, (\frac{1}{sk+x^*})G, \widehat{pk})$ to H_{out} and later queries the functionality $\mathcal{F}$ with $(\text{input-OPRF}, ssid, x^*, \cdot)$ before an adaptive server compromise. In this case, $\mathcal{S}$ does not program the output of H_{out} to be consistent with that of $\mathcal{F}$.

By Definition 3, the value $(\frac{1}{sk+x^*})G$ is unforgeable for a PPT adversary who knows only skG and who is able to make queries of the form $(\frac{1}{sk+}).G$. So, $\mathcal{Z}$ who knows only $\widehat{pk} = (skG, \pi)$ can with only negligible

probability predict the value $(\frac{1}{sk+x^*}G)$ in order to make the query of the correct form $z^* = (x^*, (\frac{1}{sk+x^*}G), \widehat{pk})$. If we assume otherwise, then there exists an adversary $\mathcal{A}_{BB}$ for the assumption described Definition 3 who can find a forgery with non-negligible probability.

The reduction is as follows: Let $\mathcal{A}_{BB}$ be the adversary for the assumption described in Definition 3 and $\mathcal{O}_{BB}^{sk}$ be the oracle which it can query. $\mathcal{A}_{BB}$ on input $pk_{BB} = skG$ begins by simulating the honest server initialization, sending $\widehat{pk} := pk_{BB}$ to the environment $\mathcal{Z}$ as output. For queries on input x from any party, $\mathcal{A}_{BB}$ outputs $y \leftarrow \mathcal{O}_{BB}^{sk}(x)$. If $\mathcal{Z}$ makes a query to H_{out} of the form $z^* = (x^*, (\frac{1}{sk+x^*}G), \widehat{pk})$ then, $\mathcal{A}_{BB}$ outputs $(x^*, (\frac{1}{sk+x^*}G))$ as its forgery.

Therefore, Hybrid 7 is computationally indistinguishable from Hybrid 6.

Hybrid 8. Corrupted Server Output. In this hybrid, $\mathcal{S}$ instructs $\mathcal{F}$ to either deliver output or abort based on the values received from $\mathcal{A}$ on behalf of a corrupted s , as in *Output Computation for Honest p* of $\mathcal{S}_{OPRF}$. Instead of following the code of an honest p to verify the protocol message, $\mathcal{S}$ verifies the messages received from s by first extracting $sk^* \leftarrow \text{Ext}_{d1}((G, pk^*), \pi^*, H_{zk-d1})$ and $u \leftarrow \text{Ext}_{dh}(dht, \pi_{dh}, H_{zk-dht})$ and verifying that the remaining values are computed as required, as in Step 2:

$$u = u_x = u_z, \overline{v_x} = \overline{y_x} + u_x(\delta + sk^*), \text{ and } \overline{V_z} = (\overline{y_z} + u_z sk^*)G$$

Here, instead of verifying this fact by computing $V_{dh}(dht, \pi_{dh})$, $\mathcal{S}$ checks if $(G, uG, pk^*, upk^*) \neq dht$ as in section *Output Computation for Honest p* of $\mathcal{S}_{OPRF}$. If $(G, uG, pk^*, upk^*) \neq dht$, or any of the other checks fail, $\mathcal{S}$ instructs $\mathcal{F}_{opr}^{com}$ to abort. Otherwise, $\mathcal{S}$ instructs $\mathcal{F}_{opr}^{com}$ to deliver the output to p . By straight-line extractability (Definition 1), if $\tau = V_{dh}(dht, \pi_{dh})$ then with all but negligible probability Ext returns a valid witness. Therefore, the view of $\mathcal{Z}$ is only negligibly different in Hybrid 8 than in Hybrid 7.

Hybrid 9. Session Labeling & Man in the Middle Outputs. In this hybrid, instead of forwarding the outputs provided by $\mathcal{S}$, the functionality $\mathcal{F}$ samples an output when either s is corrupted, or the received message passes the verification process in Hybrid 8 under some corrupted public key $\widehat{pk}^* = (pk^*, \pi^*)$. The session label indicated by $\mathcal{S}$ in the *output-OPRF* query is set to the public key $\widehat{pk}^*$ under which the received message verifies, resulting in consistent evaluations for every distinct pair.

To ensure that the protocol outputs match those of $\mathcal{F}$, upon receipt of a verifying message and prior to querying *output-OPRF* for *ssid*, $\mathcal{S}$ programs the random oracle H_{out} as in *HYBRID FUNCTIONALITIES* section of $\mathcal{S}_{OPRF}$. Specifically, all queries to H_{out} are checked. If an input $z^* = (x^*, Y^*, \widehat{pk}^*)$ is of the form

$$\left(x, \left(\frac{1}{sk^* + x^*}\right)G, (pk^*, \pi^*)\right) \text{ for } sk^* \leftarrow \text{Ext}_{d1}((G, pk^*), \pi^*, H_{zk-d1})$$

, the output of H_{out} is programmed to match the output of $\mathcal{F}$. In this case, $\mathcal{S}$ queries $\mathcal{F}$ with $(adv\text{-eval}, \widehat{pk}^*, x^*)$ for an offline evaluation, receives $(output\text{-OPRF}, x^*, y^*)$ and defines $H_{out}(z^*) := y^*$.

By straight-line extractability (Definition 1), if $\tau = V_{d1}(pk^*, \pi^*)$ then with all but negligible probability Ext returns a valid witness. Therefore, the view of $\mathcal{Z}$ is again only negligibly different in Hybrid 9 than in Hybrid 8.

Hybrid 10. Honest Party Input Leakage. In this hybrid, $\mathcal{F}$ does not leak the entire input query of an honest $p \in \mathcal{P}$ to $\mathcal{S}$ and instead sends $(input\text{-OPRF}, ssid, p, C)$, excluding the input x . In this case, $\mathcal{S}$ essentially simulates the honest p with a random input x .

Specifically, $\mathcal{S}$ samples a random offset δ and does not specify the random sample $\bar{x}$ for $\mathcal{F}_{ua-vole}^{com}$ in the session *ssid*.

In the event that s is honest, the input is fully specified during the simulation of the honest s protocol response. In the event that s is corrupted, $\mathcal{S}$ verifies the responses from s are of the correct format (as since Hybrid 9). This change is statistical; by the linearity of the verification of the server responses, a check at a

random location will also pass everywhere with overwhelming probability, and the output is already verified on a random input z . □

5 Strong Asymmetric PAKE

5.1 The saPAKE Functionality

The ideal strong asymmetric password authenticated key exchange functionality $\mathcal{F}_{\text{saPAKE}}$ is presented as Functionalities 5.1 and 5.2.

Functionality 5.1: $\mathcal{F}_{\text{saPAKE}}(\mathcal{D}, \kappa)$ Strong Asymmetric PAKE.

The functionality $\mathcal{F}_{\text{saPAKE}}$ with session identifiers sid and subsession identifiers ssid is parameterized by a password dictionary $\mathcal{D}$ and a security parameter κ . The functionality interacts with a client C , a server s , and an ideal adversary $\mathcal{A}^*$ via the queries below. Upon invocation, the functionality receives from $\mathcal{A}^*$ the set $\mathcal{C} \subseteq \{C, s\}$ of statically corrupted parties. The functionality maintains four initially undefined tables $\text{file}[x]$, $\text{fileStatus}[x]$, $\text{session}[x]$, and $\text{sessionStatus}[x]$ recording the server's password file, the status of the server's password file, active and completed sessions, and the status of those sessions, respectively, on input x .

PASSWORD REGISTRATION. Initialization of the server s . Only executed once.

1. **On** (`storepwdfile`, sid , C , pwd) **from** s **where** $\text{pwd} \in \mathcal{D}$. Ignore if $\text{file}[(\text{sid}, \square, \square)]$ is defined. Set $\text{file}[(\text{sid}, C, s)] := \text{pwd}$ and set $\text{fileStatus}[(\text{sid}, C, s)] := \text{unstolen}$.

STEALING PASSWORD DATA. Adaptively compromising servers and offline password testing. Can be queried multiple times.

2. **On** (`stealpwdfile`, sid) **from** $\mathcal{A}^*$. If $\text{file}[(\text{sid}, C, s)]$ is not defined, return “no password file” to $\mathcal{A}^*$. Otherwise, return “password file stolen” to $\mathcal{A}^*$ and set $\text{fileStatus}[(\text{sid}, C, s)] := \text{stolen}$.
3. **On** (`offline-test`, sid , pwd^*) **from** $\mathcal{A}^*$. Ignore if $\text{fileStatus}[(\text{sid}, C, s)]$ is not defined or is `unstolen`. If $\text{file}[(\text{sid}, C, s)] = \text{pwd}^*$, return “correct guess” to $\mathcal{A}^*$. Otherwise, return “wrong guess” to $\mathcal{A}^*$.

PASSWORD AUTHENTICATION. Initializing online sessions. Can be queried multiple times.

4. **On** (`PAKE-clientsession`, sid , ssid , s , pwd') **from** C . Ignore if $\text{sessionStatus}[(\text{sid}, \text{ssid}, C, s)]$ is already defined. Set $\text{session}[(\text{sid}, \text{ssid}, C, s)] := \text{pwd}'$. Set $\text{sessionStatus}[(\text{sid}, \text{ssid}, C, s)] := \text{fresh}$. Send (`PAKE-clientsession`, sid , ssid , C , s) to $\mathcal{A}^*$.
5. **On** (`PAKE-serversession`, sid , ssid , C) **from** s . Ignore if $\text{pwd} := \text{file}[(\text{sid}, C, s)]$ is not defined. Ignore if $\text{sessionStatus}[(\text{sid}, \text{ssid}, s, C)]$ is already defined. Set $\text{session}[(\text{sid}, \text{ssid}, s, C)] := \text{pwd}$. Set $\text{sessionStatus}[(\text{sid}, \text{ssid}, s, C)] := \text{fresh}$. Send (`PAKE-serversession`, sid , ssid , C , s) to $\mathcal{A}^*$.

Registration. The functionality models a one-time out-of-band registration, followed by multiple password-based authentication subsessions for a single client-server pair $\{C, s\}$. This “out-of-band” registration has the server, as opposed to the client, register and store a password. The functionality records the password pwd in the table $\text{file}[]$, and marks the entry as `unstolen` in $\text{fileStatus}[]$.

Authentication. To authenticate, the client C queries `PAKE-clientsession` with a unique identifier ssid and a candidate password pwd' . This password is recorded in the table $\text{session}[]$ and, upon receipt of such a query, the subsession is marked as `fresh` by having the corresponding entry of $\text{sessionStatus}[]$ set to `fresh`. Similarly, a subsession is initialized by the server s via a query `PAKE-serversession` for a unique ssid . The functionality always assigns to the server subsession the password stored during registration as opposed to accepting a candidate subsession password along with the query (as we saw with the client), modeling that the server only knows the client's password *in the clear* during registration and uses the password file for authentication. Similarly, the corresponding entry of $\text{sessionStatus}[]$ is set to `fresh`. Upon either query, $\mathcal{A}^*$ is notified that the honest party has been activated in this subsession by receiving (ssid, C, s) or (ssid, s, C) .

Functionality 5.2: $\mathcal{F}_{\text{saPAKE}}(\mathcal{D}, \kappa)$ saPAKE, part 2.

ACTIVE SESSION ATTACKS. Online guessing attacks and server impersonation. Can be queried multiple times.

6. **On** (`testpwd`, `sid`, `ssid`, `p`, `pwd*`) **from** $\mathcal{A}^*$ **where** $p \in \{s, C\}$. Ignore if `sessionStatus`[(`sid`, `ssid`, `p`, `p'`)] is not fresh. If `session`[(`sid`, `ssid`, `p`, `p'`)] = `pwd*`, set `sessionStatus`[(`sid`, `ssid`, `p`, `p'`)] := `compromised` and send “correct guess” to $\mathcal{A}^*$. Otherwise, set `sessionStatus`[(`sid`, `ssid`, `p`, `p'`)] := `interrupted` and send “wrong guess” to $\mathcal{A}^*$.
7. **On** (`impersonate`, `sid`, `ssid`) **from** $\mathcal{A}^*$. Ignore if `sessionStatus`[(`sid`, `ssid`, `C`, `s`)] is not fresh. Ignore if `fileStatus`[(`sid`, `C`, `s`)] is not defined or is unstolen. If `file`[(`sid`, `C`, `s`)] = `session`[(`sid`, `ssid`, `C`, `s`)] then set `sessionStatus`[(`sid`, `ssid`, `C`, `s`)] := `compromised` and send “correct guess” to $\mathcal{A}^*$. Otherwise, set `sessionStatus`[(`sid`, `ssid`, `C`, `s`)] := `interrupted` and send “wrong guess” to $\mathcal{A}^*$.

KEY GENERATION. Derivation and output of session keys including explicit Server-to-Client authentication. Can be queried multiple times.

8. **On** (`clientnewkey`, `sid`, `ssid`, `k*`) **from** $\mathcal{A}^*$ **where** $k^* \in \{0, 1\}^\kappa$. Ignore if `sessionStatus`[(`sid`, `ssid`, `C`, `s`)] is not defined or is completed. If `sessionStatus`[(`sid`, `ssid`, `C`, `s`)] = `compromised`, set $k := k^*$. If `sessionStatus`[(`sid`, `ssid`, `C`, `s`)] = `fresh`, there is a counter-session `session`[(`sid`, `ssid`, `s`, `C`)] = `session`[(`sid`, `ssid`, `C`, `s`)], and `sessionStatus`[(`sid`, `ssid`, `s`, `C`)] was fresh when (`newkey`, `sid`, `ssid`, `k'`) was sent to `s`, set $k := k'$. If `sessionStatus`[(`sid`, `ssid`, `C`, `s`)] = `interrupted` or is fresh and there is no counter-session `session`[(`sid`, `ssid`, `s`, `C`)] = `session`[(`sid`, `ssid`, `C`, `s`)], then set $k := \perp$. If k is not defined, sample $k \leftarrow \{0, 1\}^\kappa$. Set `sessionStatus`[(`sid`, `ssid`, `C`, `s`)] := `completed`. Send (`newkey`, `sid`, `ssid`, k) to `C`.
9. **On** (`servernewkey`, `sid`, `ssid`, `k*`) **from** $\mathcal{A}^*$ **where** $k^* \in \{0, 1\}^\kappa$. Ignore if `sessionStatus`[(`sid`, `ssid`, `s`, `C`)] is not defined or is completed. If `sessionStatus`[(`sid`, `ssid`, `s`, `C`)] = `compromised`, set $k := k^*$. If `sessionStatus`[(`sid`, `ssid`, `s`, `C`)] = `fresh`, there is a counter-session `session`[(`sid`, `ssid`, `C`, `s`)] = `session`[(`sid`, `ssid`, `s`, `C`)], and `sessionStatus`[(`sid`, `ssid`, `C`, `s`)] was fresh when (`newkey`, `sid`, `ssid`, `k'`) was sent to `C`, set $k := k'$. If k is not defined, sample $k \leftarrow \{0, 1\}^\kappa$. Set `sessionStatus`[(`sid`, `ssid`, `s`, `C`)] := `completed`. Send (`newkey`, `sid`, `ssid`, k) to `s`.

from the functionality. In the case where the parties have *the same password* for subsession `ssid`, and both the client’s and server’s subsessions are marked as *fresh*, a consistent and uniform key k is output. Output is delivered at the discretion of $\mathcal{A}^*$ to both `C` and `s`, on `clientnewkey` and `servernewkey` from $\mathcal{A}^*$, respectively. In this case, the case where $\mathcal{A}^*$ has not performed an attack against the subsession, the parties authenticate and derive a shared key as expected.

Password Tests. The functionality models an inevitable online attack in which $\mathcal{A}^*$ guesses one of the two parties’ passwords. We afford to $\mathcal{A}^*$ this attack against *fresh* subsessions through the `testpwd` interface, which allows at most two password guesses per `ssid`: one against `C` and one against `s`. If $\mathcal{A}^*$ guesses correctly, the corresponding side of the subsession is marked as *compromised*. On an incorrect guess, the side is marked as *interrupted*. In either case, $\mathcal{A}^*$ is informed of the guess’s correctness. In all compromised subsessions, a key k^* of $\mathcal{A}^*$ ’s choosing is output to the compromised party.

Explicit Authentication. The output behavior of $\mathcal{F}_{\text{saPAKE}}$ is recorded in Table 2. While the presented functionality is substantially similar to $\mathcal{F}_{\text{saPAKE}}$ in the literature [52], there is one key difference: the subsession’s key delivery interfaces `clientnewkey` and `servernewkey`. In the standard functionality neither party is privy to the success of a subsession, whereas $\mathcal{F}_{\text{saPAKE}}$ models *explicit authentication*. Prior work on explicit authentication for PAKE has incorporated an explicit `TestAbort` interface [34] that allows $\mathcal{A}^*$ to equality test the passwords associated to each side of a subsession, then force an abort in the case of inequality. As Hesse has previously identified [39], this form of explicit authentication is too weak, and instead a `TestAbort` should be baked into the key delivery interface as in $\mathcal{F}_{\text{saPAKE}}$. Our protocol achieves explicit authentication for only the server, so the key deliver interface is split into a client and a server query. This bifurcation is syntactic, for cleanliness and legibility of the already dense functionality.

Adaptive Server Compromise. Additionally, as is standard, we model adaptive server compromise as a special adaptive **corruption** query sent by the environment. Explicitly, the adversary can “steal” a registered

[h]	C's session	s's session	passwords	output
	fresh	fresh	=	(k, k)
			$\neq$	$(\perp, k'')$
		interrupted	=	(k', k'')
	interrupted		$\neq$	$(\perp, k'')$
		compromised	=	(k', k^{**})
			$\neq$	$(\perp, k^{**})$
	interrupted	fresh	= or $\neq$	$(\perp, k'')$
		interrupted	= or $\neq$	$(\perp, k'')$
		compromised	= or $\neq$	$(\perp, k^{**})$
	compromised	fresh	= or $\neq$	(k^*, k'')
		interrupted	= or $\neq$	(k^*, k'')
		compromised	= or $\neq$	(k^*, k^{**})

Table 2. Outputs of $\mathcal{F}_{\text{saPAKE}}$: In the first column, named “C’s session”, is the entry of `sessionStatus[(sid, ssid, C, s)]`. The second column, named “s’s session”, is the entry of `sessionStatus[(sid, ssid, s, C)]`. The third column indicates the relationship between the entries `session[(sid, ssid, C, s)]` and `session[(sid, ssid, s, C)]`, namely the equality or inequality of the passwords associated to each side of the subsession. The final column indicates the output of the functionality, an entry of (a, b) means that a is output to C and b is output to s. The outputs k', k'' are independently sampled, k^*, k^{**} are adversarially input, $\perp$ is an indicator of failure, and k is a consistent uniform key.

password via the `stealpwdfile` interface, which either marks the password file as `stolen`, or informs $\mathcal{A}^*$ that no password was registered to C if there is no password on record. Once the password file has been marked as `stolen`, the `offline-test` interface is opened to $\mathcal{A}^*$ and allows an offline brute-force attack against the password. Importantly, although this query is a corruption query, this does not imply that the adversary learns the server’s local state and certainly does not imply the adversary gains control over the server. To disambiguate which form of corruption we are referencing, we will refer to adaptive corruption through the `stealpwdfile` interface as resulting in a `compromised` server, while we will continue to refer to static Byzantine corruption of a server as leading to a “corrupted” server.

Finally, we note that the only authentication material provided to the server is exactly their long-term storage. Once this information is leaked, an attacker can always perform an *impersonation attack* by imitating the honest server in an authentication subsession with the client. To model this inevitable attack, we afford $\mathcal{A}^*$ the `impersonate` query after the server is marked `compromised`. When the client’s password matches the honest server’s, the client subsession is `compromised` and is `interrupted` otherwise, modeling whether the client would have normally authenticated with the honest server or not.

5.2 The Strong Asymmetric PAKE Protocol

Protocol 5.3, Π_{saPAKE} , UC-realizes $\mathcal{F}_{\text{saPAKE}}$ in the $(\mathcal{F}_{\text{RO}}, \mathcal{F}_{\text{opr}}^{\text{com}})$ -hybrid model. A visual presentation of the protocol flow is provided in Figures 1.1 and 1.1.

Registration. As is standard, our protocol models registration as preceded by an out-of-band exchange between the client-to-be and the server s . In this exchange, s is given its password `pwd` for future authentication which it maps to the long-term storage (C_0, r_1) . First, s initializes and evaluates an OPRF instance on the hash of `pwd` to compute $\text{rwd} := F(\rho)$ for $\rho := H_q(\text{pwd})$. This step is analogous to generating a salted hash of `pwd`, in which the OPRF instance (key) represents the s ’s private salt. Upon receipt of $\text{rwd} \leftarrow \mathcal{F}_{\text{opr}}^{\text{com}}$, s generates a Pedersen commitment $C_0 := H_q(\text{rwd}) \cdot G + \rho \cdot H$ to ρ with randomness $H_q(\text{rwd})$. Registration is complete once s stores `file[sid] := (C_0, r_1)` for $r_1 := H_{2\kappa}(\text{rwd})$.

Authentication. To authenticate with s with a candidate password `pwd'`, the client C computes $\rho' := H_q(\text{pwd}')$ and then attempts to reconstruct the salted hash `rwd`. To do so, C runs an committing OPRF session with s learning $\text{rwd}' \leftarrow \mathcal{F}_{\text{opr}}^{\text{com}}(\rho', r)$. As part of the exchange, C commits to their input ρ' through a Pedersen commitment $C := r \cdot G + \rho' \cdot H$ which s receives. After the OPRF exchange, the client receives s ’s projection

Protocol 5.3: $\Pi_{\text{saPAKE}}(\mathcal{D}, \kappa, \text{pp}_G)$ saPAKE.

The protocol Π_{saPAKE} is parameterized by password dictionary $\mathcal{D}$, security parameter κ , and group $\text{pp}_G = (G, G, q)$. The protocol is executed by a client C and a server s with access to hybrid functionalities $(\mathcal{F}_{\text{RO}}, \mathcal{F}_{\text{OPRF}}^{\text{com}})$, for $\ell_{\text{oprf}} := 2\kappa$. For clarity, $(H_q(\cdot), H_{2\kappa}(\cdot), H_\kappa(\cdot))$ denote instances of $\mathcal{F}_{\text{RO}}$ with handle sets $(\mathbb{Z}_q, \{0, 1\}^{2\kappa}, \{0, 1\}^\kappa)$, respectively.

REGISTRATION.

Server s on $(\text{storepwdfile}, \text{sid}, \text{pwd})$ from $\mathcal{Z}$.

1. $\mathcal{F}_{\text{OPRF}}^{\text{com}}(\text{sid}, \ell_{\text{oprf}}, \text{pp}_G)$: Query $(\text{init-OPRF}, s)$.
2. Evaluate $\rho := H_q((1, \text{pwd}))$.
3. $\mathcal{F}_{\text{OPRF}}^{\text{com}}(\text{sid}, \ell_{\text{oprf}}, \text{pp}_G)$:
Query $(\text{OPRF-offline}, \rho)$. Receive $(\text{OPRF-offline}, \rho, \text{rwd})$.
Query (com-key) . Receive $(\text{com-key}, H)$.
4. Evaluate $r_0 := H_q((0, \text{rwd}))$ and $r_1 := H_{2\kappa}((0, \text{rwd}))$.
5. Store $\text{file}[\text{sid}] := (C_0, r_1) = (r_0 \cdot G + \rho \cdot H, r_1)$.

LOGIN.

Client C , on $(\text{PAKE-clientsession}, \text{sid}, \text{ssid}, s, \text{pwd}')$ from $\mathcal{Z}$.

1. Evaluate $\rho' := H_q((1, \text{pwd}'))$.
2. Sample $r \leftarrow \mathbb{Z}_q$.
3. $\mathcal{F}_{\text{OPRF}}^{\text{com}}(\text{sid}, \ell_{\text{oprf}}, \text{pp}_G)$: Query (com-key) . Receive $(\text{com-key}, H)$.
Query $(\text{input-OPRF}, \text{ssid}, \rho', r)$. Receive $(\text{output-OPRF}, \text{ssid}, \text{rwd}')$.

Client C , on $(\text{sid}, \text{ssid}, Y, V)$ from s .

4. If $Y = 0_G$: output $(\text{newkey}, \text{sid}, \text{ssid}, \perp)$ and abort.
5. Evaluate $r'_0 := H_q((0, \text{rwd}'))$ and $r'_1 := H_{2\kappa}((0, \text{rwd}'))$.
6. Define $Z' := (r - r'_0) \cdot Y$, and $C' := (r \cdot G + \rho' \cdot H)$.
7. Evaluate $V' := H_{2\kappa}((1, \text{ssid}, Y, C', Z', r'_1))$.
8. If $V \neq V'$: output $(\text{newkey}, \text{sid}, \text{ssid}, \perp)$ and abort.
9. Evaluate $k := H_\kappa((\text{ssid}, Y, C', Z', r'_1))$.
10. Output $(\text{newkey}, \text{sid}, \text{ssid}, k)$.

Server s on $(\text{PAKE-serversession}, \text{sid}, \text{ssid})$ from $\mathcal{Z}$.

1. Retrieve $(C_0, r_1) := \text{file}[\text{sid}]$.
2. $\mathcal{F}_{\text{OPRF}}^{\text{com}}(\text{sid}, \ell_{\text{oprf}}, \text{pp}_G)$:
Receive $(\text{input-OPRF}, \text{ssid}, C')$. Query $(\text{evaluate-OPRF}, \text{ssid})$.
3. Sample $y \leftarrow \mathbb{Z}_q^*$. Define $Y := y \cdot G$, and $Z := y \cdot (C - C_0)$.
4. Evaluate $V := H_{2\kappa}((1, \text{ssid}, Y, C, Z, r_1))$, and $k := H_\kappa((\text{ssid}, Y, C, Z, r_1))$.
5. Send $(\text{sid}, \text{ssid}, Y, V)$ to C .
6. Output $(\text{newkey}, \text{sid}, \text{ssid}, k)$.

COMPROMISE.

Server s on $(\text{stealpwdfile}, \text{sid})$ from $\mathcal{A}$.

1. If $\text{file}[\text{sid}]$ is not defined, send “no password file” to $\mathcal{A}$.
2. Otherwise, retrieve $(C_0, r_1) := \text{file}[\text{sid}]$ and send (C_0, r_1) to $\mathcal{A}$.

key $Y = y \cdot G$, and s 's authentication tag V which is used to indicate success or failure of the exchange. The exchange is completed when C computes the projected hash value $Z' := (r - r'_0) \cdot Y$ and verifies s 's tag. If verification succeeds, the client outputs a key k_C , and otherwise outputs $\perp$ then aborts. Since our protocol only models explicit authentication for the server, the server will output a key k_s in every case.

$$k_s = H_\kappa((\text{ssid}, Y, C, Z, r_1)) \stackrel{?}{=} H_\kappa((\text{ssid}, Y, C', Z', r'_1)) = k_C$$

How the parties arrive at a shared key becomes clear when peeking into the server's view:

$$\begin{aligned}
Z &= y(C - C_0) && \text{Input for the Server's key} \\
&= y((rG + \rho'H) - (uG + \rho H)) && \text{for } u \leftarrow H_q(\text{rwd}) \\
&= y(r - u)G + y(\rho' - \rho)H \\
&= y(r - u)G && \text{if } \rho = H_q(\text{pwd}) = H_q(\text{pwd}') = \rho' \\
&= y(r - r'_0)G && \text{if } u = H_q(\text{rwd}) = H_q(\text{rwd}') = r'_0 \\
&= (r - r'_0)Y = Z' && \text{Input for the Client's key}
\end{aligned}$$

Equality ($Z = Z'$) only occurs, with non-negligible probability, when $\text{pwd} = \text{pwd}'$. In such a case, $V = V'$, $k = k'$, and the parties will both output the same key. In the case where the parties have different passwords, we have that $Z \neq Z'$ (and therefore $V \neq V'$) with overwhelming probability, and the client will output $\perp$ then abort.

Client Speaks First. An important feature of our protocol is not only the round complexity—a single round trip—but that $\mathcal{C}$ sends the first message. This is because our OPRF protocol $\Pi_{\text{opr}}^{\text{com}}$ has the same communication pattern; $\mathcal{s}$ can send $(\text{sid}, \text{ssid}, Y, V)$ along with the OPRF response.

5.3 Proof of Security

We now aim to show that the our saPAKE protocol Π_{saPAKE} securely UC-realizes the full saPAKE functionality $\mathcal{F}_{\text{saPAKE}}$ in the $(\mathcal{F}_{\text{RO}}, \mathcal{F}_{\text{opr}}^{\text{com}})$ -hybrid model. Further, we provide an explicit simulator $\mathcal{S}_{\text{saPAKE}}$ by which our proof is carried out and which has simulation rate $r = 1$. To prevent standard degenerate PAKE protocols, our simulator is additionally strong perfectly reasonable [59]. All this means is that $\mathcal{S}_{\text{saPAKE}}$ will not call `testpwd` or `impersonate` against a subsession where $\mathcal{Z}$ has instructed the real adversary $\mathcal{A}$ to take no action beyond faithfully passing messages.

Theorem 4. *The protocol Π_{saPAKE} securely UC-realizes the ideal functionality $\mathcal{F}_{\text{saPAKE}}$ in the $(\mathcal{F}_{\text{RO}}, \mathcal{F}_{\text{opr}}^{\text{com}})$ -hybrid model by way of a strong perfectly reasonable simulator $\mathcal{S}_{\text{saPAKE}}$ with simulation rate $r = 1$ assuming that The Decisional Diffie-Hellman (DDH) Assumption holds in group $\mathbb{G}$.*

Figure 5. $\mathcal{S}_{\text{saPAKE}}(\text{sid}, \mathcal{D}, \kappa)$

This is the simulator for Π_{saPAKE} . It is parameterized by a password dictionary $\mathcal{D}$ and a security parameter κ . The simulator is given oracle access to the adversary $\mathcal{A}$. At the start of simulation $\mathcal{A}$ outputs static corruption instructions for $\mathcal{C} \subseteq \{\mathcal{C}, \mathcal{s}\}$. The simulator routes all messages sent by the ideal functionalities and the honest parties to $\mathcal{A}$ for delivery. Just as in Π_{saPAKE} , there are three active instances of $\mathcal{F}_{\text{RO}}$ denoted $\mathcal{F}_{\text{RO}}^0$, $\mathcal{F}_{\text{RO}}^1$, and $\mathcal{F}_{\text{RO}}^2$.

Initialization.

$\mathcal{S}_{\text{saPAKE}}$ is assumed to know the session identifier sid and server identity $\mathcal{s}$ associated with the $\mathcal{F}_{\text{saPAKE}}$ session. At the beginning of time, $\mathcal{S}_{\text{saPAKE}}$ will

1. Initialize $\text{DL}[\cdot]$, $\text{RO}^0[\cdot]$, $\text{RO}^1[\cdot]$, $\text{RO}^2[\cdot]$, $\text{OPRF}[\cdot]$, $\text{CRS}[\cdot]$, $\text{SrvSession}[\cdot]$, and $\text{OPRFSession}[\cdot]$.
2. Record the honest server OPRF label $\text{hlab} := \{\mathcal{s}\}$.
3. Sample $t \leftarrow \mathbb{Z}_q^*$, compute $H = t \cdot G$, then set $\text{CRS}[\text{sid}] := H$ and $\text{DL}[H] := t$.
4. Sample $r_1 \leftarrow \{0, 1\}^{2\kappa}$, $e \leftarrow \mathbb{Z}_q$, compute $C_0 = e \cdot G$, set $\text{file}[\text{sid}] := (C_0, r_1)$, and set $\text{DL}[C_0] := e$.

Stealing the Password File and Offline Queries.

On $(\text{stealpwdfile}, \text{sid})$ from $\mathcal{A}$ sent to $\mathcal{s}$ along with $(\text{steal-key}, \text{sid}, \mathcal{s})$ from $\mathcal{A}$ sent to $\mathcal{F}_{\text{opr}}^{\text{com}}$:

1. Send $(\text{stealpwdfile}, \text{sid})$ to $\mathcal{F}_{\text{saPAKE}}$.
 - If $\mathcal{F}_{\text{saPAKE}}$ responds “password file stolen”
 - (a) Mark $\mathcal{s}$ as compromised.
 - (b) Send $\text{file}[\text{sid}]$ to $\mathcal{A}$.

- Otherwise, send “no password file” to $\mathcal{A}$.

On $\mathcal{A}$ evaluating $\mathcal{F}_{\text{RO}}^i$ on input (sid, x^*) :

1. If $\text{RO}^i[x^*]$ is not defined, sample $\text{RO}^i[x^*] \leftarrow H_i$.
2. If there exists $x' \neq x^*$ such that $\text{RO}^i[x^*] = \text{RO}^i[x']$, output Collision and abort.
3. Send $(\text{sid}, \text{RO}^i[x^*])$ to $\mathcal{A}$.

On $(\text{com-key}, \text{sid})$ from $\mathcal{A}$ sent to $\mathcal{F}_{\text{opr}}^{\text{com}}$:

1. Send $(\text{com-key}, \text{CRS}[\text{sid}])$ to $\mathcal{A}$.

On $(\text{adv-eval}, \text{sid}, \text{lab}, \rho^*)$ from $\mathcal{A}$ sent to $\mathcal{F}_{\text{opr}}^{\text{com}}$:

- If $\text{lab} \neq \text{hlab}$
 1. If $\text{OPRF}[\text{lab}, \rho^*]$ is not defined, sample $\text{OPRF}[\text{lab}, \rho^*] \leftarrow \{0, 1\}^{2\kappa}$.
 2. If there exists $x' \neq x''$ such that $\text{OPRF}[x'] = \text{OPRF}[x'']$, output Collision and abort.
 3. Send $(\text{output-OPRF}, \text{sid}, \rho^*, \text{OPRF}[\text{lab}, \rho^*])$ to $\mathcal{A}$.
- Otherwise,
 1. Ignore this query if s is not marked compromised.
 2. If $\text{OPRF}[\text{lab}, \rho^*]$ is not defined, sample $\text{OPRF}[\text{lab}, \rho^*] \leftarrow \{0, 1\}^{2\kappa}$.
 3. If there exists $x' \neq x''$ such that $\text{OPRF}[x'] = \text{OPRF}[x'']$, output Collision and abort.
 4. If there exists $\text{pwd}^* \in \mathcal{D}$ such that $\text{RO}^0[(1, \text{pwd}^*)] = \rho^*$, send $(\text{offline-test}, \text{sid}, \text{pwd}^*)$ to $\mathcal{F}_{\text{saPAKE}}$.
 - If $\mathcal{F}_{\text{saPAKE}}$ responds “correct guess”
 - (a) Retrieve $(C_0, r_1) := \text{file}[\text{sid}]$, $e := \text{DL}[C_0]$, and $H := \text{CRS}[\text{sid}]$.
 - (b) Set $\text{RO}^0[(0, \text{OPRF}[\text{lab}, \rho^*])] := e - \text{DL}[H] \cdot \rho^*$.
 - (c) If there exist $x' \neq x''$ such that $\text{RO}^0[x'] = \text{RO}^0[x'']$, output Collision and abort.
 - (d) Set $\text{RO}^1[(0, \text{OPRF}[\text{lab}, \rho^*])] := r_1$.
 - (e) If there exist $x' \neq x''$ such that $\text{RO}^1[x'] = \text{RO}^1[x'']$, output Collision and abort.
 5. Send $(\text{output-OPRF}, \text{sid}, \text{lab}, \text{OPRF}[\text{lab}, \rho^*])$ to $\mathcal{A}$.

Password Authentication.

On $(\text{PAKE-clientsession}, \text{sid}, \text{ssid}, C, s)$ from $\mathcal{F}_{\text{saPAKE}}$:

1. Sample $r_S \leftarrow \mathbb{Z}_q$ and send $(\text{input-OPRF}, \text{sid}, \text{ssid}, C, C_S := r_S \cdot G)$ to $\mathcal{A}$.
2. From this point on, if $\mathcal{A}$ sends $(\text{evaluate-OPRF}, \text{sid}, \text{ssid})$ to $\mathcal{F}_{\text{opr}}^{\text{com}}$ when s is marked compromised, send $(\text{evaluate-OPRF}, \text{sid}, \text{ssid})$ to $\mathcal{A}$.
3. Await either
 - $(\text{deliver-input}, \text{sid}, \text{ssid}, C)$ from $\mathcal{A}$ sent to $\mathcal{F}_{\text{opr}}^{\text{com}}$ while $\text{OPRFSession}[\text{sid}, \text{ssid}]$ is not defined.
 - (a) Set $\text{OPRFSession}[\text{sid}, \text{ssid}] := C$.
 - (b) Await $(\text{output-OPRF}, \text{sid}, \text{ssid}, C, \text{lab})$ from $\mathcal{A}$ sent to $\mathcal{F}_{\text{opr}}^{\text{com}}$.
 - $(\text{output-OPRF}, \text{sid}, \text{ssid}, C, \text{lab})$ from $\mathcal{A}$ sent to $\mathcal{F}_{\text{opr}}^{\text{com}}$ where $\text{lab} \neq \text{hlab}$ or s is marked compromised.
4. Await $(\text{sid}, \text{ssid}, Y, V)$ from s or $\mathcal{A}$ sent to C .
 - If $\text{lab} = \text{hlab}$ and $(Y', V', k) := \text{SrvSession}[\text{sid}, \text{ssid}]$ is defined with $Y = Y'$ and $V = V'$:
 - (a) Let $k' := k$.
 - Otherwise:
 - (a) If $Y = 0_G$ send $(\text{testpwd}, \text{sid}, \text{ssid}, C, \perp)$ to $\mathcal{F}_{\text{saPAKE}}$.
 - (b) If there does not exist Z^*, r_1^* such that $\text{RO}^1[(1, \text{ssid}, Y, C_S, Z^*, r_1^*)] = V$, send $(\text{testpwd}, \text{sid}, \text{ssid}, C, \perp)$ to $\mathcal{F}_{\text{saPAKE}}$.
 - (c) If (1) s is marked compromised with $(C_0, r_1) := \text{file}[\text{sid}]$, (2) $\text{lab} = \text{hlab}$, and (3) $r_1^* = r_1$ and $Z^* = (r_S - \text{DL}[C_0]) \cdot Y$ then send $(\text{impersonate}, \text{sid}, \text{ssid})$ to $\mathcal{F}_{\text{saPAKE}}$.
 - If $\mathcal{F}_{\text{saPAKE}}$ responds “correct guess”:
 - i. If $\text{RO}^2[(\text{ssid}, Y, C_S, Z^*, r_1)]$ is not defined, sample $\text{RO}^2[(\text{ssid}, Y, C_S, Z^*, r_1)] \leftarrow \{0, 1\}^\kappa$.

- ii. If there exist $x' \neq x''$ such that $\text{RO}^2[x'] = \text{RO}^2[x'']$, output Collision and abort.
 - iii. Let $k' := \text{RO}^2[(\text{ssid}, Y, C_S, Z^*, r_1)]$.
 - (d) If (1) $r_1^* = \text{RO}^1[(0, \text{OPRF}[\text{lab}, \rho^*])]$ such that there exists $\text{pwd}^* \in \mathcal{D}$ with $\text{RO}^0[(1, \text{pwd}^*)] = \rho^*$, (2) there exists $r_0^* := \text{RO}^0[(0, \text{OPRF}[\text{lab}, \rho^*])]$, and (3) $Z^* = (r_S - (r_0^* + \text{DL}[H] \cdot \rho^*)) \cdot Y$ where $H := \text{CRS}[\text{sid}]$, then send $(\text{testpwd}, \text{sid}, \text{ssid}, C, \text{pwd}^*)$ to $\mathcal{F}_{\text{saPAKE}}$.
 - If $\mathcal{F}_{\text{saPAKE}}$ responds “correct guess”:
 - i. If $\text{RO}^2[(\text{ssid}, Y, C_S, Z^*, r_1)]$ is not defined, sample $\text{RO}^2[(\text{ssid}, Y, C_S, Z^*, r_1)] \leftarrow \{0, 1\}^\kappa$.
 - ii. If there exist $x' \neq x''$ such that $\text{RO}^2[x'] = \text{RO}^2[x'']$, output Collision and abort.
 - iii. Let $k' := \text{RO}^2[(\text{ssid}, Y, C_S, Z^*, r_1)]$.
 - and send $(\text{testpwd}, \text{sid}, \text{ssid}, C, \perp)$ to $\mathcal{F}_{\text{saPAKE}}$ otherwise.
 - 5. If k' is not defined, sample $k' \leftarrow \{0, 1\}^\kappa$.
 - 6. Send $(\text{clientnewkey}, \text{sid}, \text{ssid}, k')$ to $\mathcal{F}_{\text{saPAKE}}$.
- On $(\text{PAKE-serversession}, \text{sid}, \text{ssid}, C, s)$ from $\mathcal{F}_{\text{saPAKE}}$:
1. Sample $V \leftarrow \{0, 1\}^{2\kappa}$.
 2. Sample $k \leftarrow \{0, 1\}^\kappa$.
 3. Await either
 - $\text{OPRFSession}[\text{sid}, \text{ssid}]$ being defined as C .
 - (a) Sample $y \leftarrow \mathbb{Z}_q^*$ and define $Y := y \cdot G$.
 - (b) Set $\text{SrvSession}[\text{sid}, \text{ssid}] := (Y, V, k)$.
 - (c) Send $(\text{sid}, \text{ssid}, Y, V)$ from s to C .
 - (d) Send $(\text{servernewkey}, \text{sid}, \text{ssid}, k)$ to $\mathcal{F}_{\text{saPAKE}}$.
 - $\text{OPRFSession}[\text{sid}, \text{ssid}]$ being defined as $\mathcal{A}$ through the following process.
 - (a) Await $(\text{input-OPRF}, \text{sid}, \text{ssid}, \mathcal{A}, \rho^*, r^*)$ from $\mathcal{A}$ sent to $\mathcal{F}_{\text{oprf}}^{\text{com}}$.
 - (b) Send $(\text{input-OPRF}, \text{sid}, \text{ssid}, \mathcal{A}, C^* := r^* \cdot G + \rho^* \cdot \text{CRS}[\text{sid}])$ to $\mathcal{A}$.
 - (c) From this point on, if $\mathcal{A}$ sends $(\text{evaluate-OPRF}, \text{sid}, \text{ssid})$ to $\mathcal{F}_{\text{oprf}}^{\text{com}}$ when s is marked compromised, send $(\text{evaluate-OPRF}, \text{sid}, \text{ssid})$ to $\mathcal{A}$.
 - (d) From this point on, if $\mathcal{A}$ sends $(\text{output-OPRF}, \text{sid}, \text{ssid}, \mathcal{A}, \text{lab})$ to $\mathcal{F}_{\text{oprf}}^{\text{com}}$ where $\text{lab} \neq \text{hlab}$ or s is marked compromised, treat the query as $(\text{adv-eval}, \text{sid}, \text{lab}, \rho^*)$.
 - (e) Await $(\text{deliver-input}, \text{sid}, \text{ssid}, \mathcal{A})$ from $\mathcal{A}$ sent to $\mathcal{F}_{\text{oprf}}^{\text{com}}$.
 - (f) Set $\text{OPRFSession}[\text{sid}, \text{ssid}] := \mathcal{A}$.
 4. If $\text{OPRFSession}[\text{sid}, \text{ssid}] = \mathcal{A}$:
 - (a) Sample $y \leftarrow \mathbb{Z}_q^*$ and define $Y := y \cdot G$.
 - (b) If there exists $\text{pwd}^* \in \mathcal{D}$ such that $\text{RO}^0[(1, \text{pwd}^*)] = \rho^*$, send $(\text{testpwd}, \text{sid}, \text{ssid}, s, \text{pwd}^*)$ to $\mathcal{F}_{\text{saPAKE}}$ and otherwise, send $(\text{testpwd}, \text{sid}, \text{ssid}, s, \perp)$ to $\mathcal{F}_{\text{saPAKE}}$.
 - If $\mathcal{F}_{\text{saPAKE}}$ responds “correct guess”:
 - i. Retrieve $(C_0, r_1) := \text{file}[\text{sid}]$, $e := \text{DL}[C_0]$, and $H := \text{CRS}[\text{sid}]$.
 - ii. Compute $Z = y \cdot (C^* - C_0)$.
 - iii. Set $\text{RO}^1[(1, \text{ssid}, Y, C^*, Z, r_1)] := V$.
 - iv. If there exist $x' \neq x''$ such that $\text{RO}^1[x'] = \text{RO}^1[x'']$, output Collision and abort.
 - v. Set $\text{RO}^2[(\text{ssid}, Y, C^*, Z, r_1)] := k$.
 - vi. If there exist $x' \neq x''$ such that $\text{RO}^2[x'] = \text{RO}^2[x'']$, output Collision and abort.
 - vii. If $\text{OPRF}[\text{hlab}, \rho^*]$ is not defined, sample $\text{OPRF}[\text{hlab}, \rho^*] \leftarrow \{0, 1\}^{2\kappa}$.
 - viii. If there exists $x' \neq x''$ such that $\text{OPRF}[x'] = \text{OPRF}[x'']$, output Collision and abort.
 - ix. Set $\text{RO}^0[(0, \text{OPRF}[\text{hlab}, \rho^*])] := e - \text{DL}[H] \cdot \rho^*$.
 - x. If there exist $x' \neq x''$ such that $\text{RO}^0[x'] = \text{RO}^0[x'']$, output Collision and abort.
 - xi. Set $\text{RO}^1[(0, \text{OPRF}[\text{hlab}, \rho^*])] := r_1$.
 - xii. If there exist $x' \neq x''$ such that $\text{RO}^1[x'] = \text{RO}^1[x'']$, output Collision and abort.
 - (c) Set $\text{SrvSession}[\text{sid}, \text{ssid}] := (Y, V, k)$.
 - (d) Send $(\text{sid}, \text{ssid}, Y, V)$ from s to C .
 - (e) Send $(\text{servernewkey}, \text{sid}, \text{ssid}, k)$ to $\mathcal{F}_{\text{saPAKE}}$.

- (f) Await (output-OPRF, sid, ssid, $\mathcal{A}$, hlab) from $\mathcal{A}$ sent to $\mathcal{F}_{\text{oprf}}^{\text{com}}$.
- (g) If OPRF[hlab, ρ^*] is not defined, sample $\text{OPRF}[\text{hlab}, \rho^*] \leftarrow \{0, 1\}^{2\kappa}$.
- (h) If there exists $x' \neq x''$ such that $\text{OPRF}[x'] = \text{OPRF}[x'']$, output Collision and abort.
- (i) Send (output-OPRF, sid, ssid, $\text{OPRF}[\text{hlab}, \rho^*]$) to $\mathcal{A}$.

Proof. Real World In the real world, the protocol Π_{saPAKE} is run between two parties $\mathcal{C}$ and $\mathcal{S}$ with access to the ideal functionalities $\mathcal{F}_{\text{RO}}$ and $\mathcal{F}_{\text{oprf}}^{\text{com}}$.

$\text{REAL}_{\Pi_{\text{saPAKE}}, \mathcal{A}, \mathcal{Z}}$

Hybrid 1: Simulating Functionalities In this hybrid, we introduce two dummy parties for the client and server and introduce an entity $\mathcal{F}$ which lies between the parties and their dummy counterparts. This entity will be referred to as the “partial functionality” and we will build this entity in each successive hybrid into the full functionality. At this stage, the partial functionality simply relays messages between the real and dummy parties. We additionally introduce a new entity $\mathcal{S}$ referred to as the “partial simulator” which encapsulates the real world adversary, the real parties, and the hybrid functionalities.

At this stage, the partial simulator implements the code of the hybrid functionalities exactly as written and runs the execution of the protocol through knowledge of each party’s input password. Because these are all conceptual alterations to the environment’s view, this hybrid is perfectly indistinguishable from the previous hybrid.

Hybrid 2: Removing Collisions Recall that the full simulator $\mathcal{S}_{\text{saPAKE}}$ will abort whenever it observes a collision from any of the three random oracle instances or the OPRF. Specifically, after sampling a handle from the functionality’s handle space (or programming one), if the same functionality has returned that handle before, $\mathcal{S}_{\text{saPAKE}}$ will abort. In this hybrid, we modify the partial simulator $\mathcal{S}$ to abort if

1. When setting a fresh handle $y \in \mathbb{Z}_q$ for $H_q(\text{sid}, x)$, there exists an $x' \neq x$ such that y was returned on the query $H_q(\text{sid}, x')$.
2. When setting a fresh handle $y \in \{0, 1\}^{2\kappa}$ for $H_{2\kappa}(\text{sid}, x)$, there exists an $x' \neq x$ such that y was returned on the query $H_{2\kappa}(\text{sid}, x')$.
3. When setting a fresh handle $y \in \{0, 1\}^\kappa$ for $H_\kappa(\text{sid}, x)$, there exists an $x' \neq x$ such that y was returned on the query $H_\kappa(\text{sid}, x')$.
4. When setting a fresh handle $y \in \{0, 1\}^{2\kappa}$ for $\mathcal{F}_{\text{oprf}}^{\text{com}}$ on input (adv-eval, sid, lab, x) or (output-OPRF, sid, ssid, lab, x), there exists an $x' \neq x$ such that y was returned on either (adv-eval, sid, lab, x') or (output-OPRF, sid, ssid, lab, x').

As $q \approx 2^{2\kappa}$, this hybrid is indistinguishable from the previous hybrid by a birthday bound over all queries $\mathcal{Z}$ makes to the interfaces above.

Hybrid 3: Recording OPRF and Random Oracle Queries As a further modification to the previous hybrid, we modify the partial simulator $\mathcal{S}$ to record all input-output pairs for the OPRF and RO queries in their respective tables as found in $\mathcal{S}_{\text{saPAKE}}$. Specifically, whenever a query ($H(\cdot)$, output-OPRF, or adv-eval) is made on input x with corresponding output y , the query pair is recorded in the correct table from $\text{RO}^0[\cdot]$, $\text{RO}^1[\cdot]$, $\text{RO}^2[\cdot]$, and $\text{OPRF}[\cdot]$.

These are purely bookkeeping changes to the partial simulator and do not affect $\mathcal{Z}$ ’s view. Therefore, this hybrid is perfectly indistinguishable from the previous hybrid.

Hybrid 4: Recording Sessions In this hybrid, we modify the functionality to do some additional bookkeeping. Recall that until this hybrid, the partial functionality $\mathcal{F}$ simply relays all messages between the real and dummy parties. In $\mathcal{F}_{\text{saPAKE}}$, there are specific rules for each of the interfaces which rely on a record keeping system to know when and how to respond to queries the parties make.

Our first change to $\mathcal{F}$ is to introduce a record of sessions when $\mathcal{F}$ receives either `storepwdfile`, `PAKE-clientsession`, or `PAKE-serversession`. Specifically, we have the new partial functionality record

the server's password on registration file[(sid, C, s)] := pwd, as well as the passwords for the client session session[(sid, ssid, C, s)] := pwd', and both the client session status and the server session status.

These are purely bookkeeping changes to the partial functionality and do not affect $\mathcal{Z}$'s view. Therefore, this hybrid is perfectly indistinguishable from the previous hybrid.

Hybrid 5: Adversary Interfaces minus NewKeys In the previous hybrid, our partial functionality $\mathcal{F}$ only had the interfaces which can be queried by the client or server. In this hybrid, we modify $\mathcal{F}$ to have the complete code of each of the interfaces which can be queried by the ideal adversary with one exception. The newly added interfaces are stealpwdfile, offline-test, testpwd, and impersonate. We have explicitly left the two newkey interfaces clientnewkey and servernewkey because they will be built in multiple steps.

We additionally change the partial simulator $\mathcal{S}$ to send (stealpwdfile, sid) to the partial functionality $\mathcal{F}$ whenever $\mathcal{Z}$ sends (stealpwdfile, sid) to $\mathcal{S}$ and (steal-key, sid, s) to $\mathcal{F}_{\text{opr}, \text{com}}^{\text{com}}$. As $\mathcal{S}$ currently receives the server query (storepwdfile, sid, C, pwd), this query is just a formality for later hybrids.

In the previous hybrid, the partial simulator $\mathcal{S}_{\text{saPAKE}}$ is acting honestly (aside from aborts in case of collision) with full knowledge of the parties' passwords and, therefore, does not make use of any of $\mathcal{F}$'s interfaces. Because of this, this hybrid is perfectly indistinguishable from the previous hybrid.

Hybrid 6: NewKey Skeleton In this hybrid we add modified versions of the two newkey interfaces to our partial functionality $\mathcal{F}$. These new interfaces will have the following code:

Changes to newkey

On (clientnewkey, sid, ssid, k^*) from $\mathcal{A}^*$ where $k^* \in \{0, 1\}^\kappa \cup \{\perp\}$.

1. Ignore this query if sessionStatus[(sid, ssid, C, s)] is not defined or is completed.
2. If sessionStatus[(sid, ssid, C, s)] = interrupted, then set $k := \perp$.
3. If k is not defined, set $k := k^*$.
4. Set sessionStatus[(sid, ssid, C, s)] := completed.
5. Send (newkey, sid, ssid, k) to C.

On (servernewkey, sid, ssid, k^*) from $\mathcal{A}^*$ where $k^* \in \{0, 1\}^\kappa$.

1. Ignore this query if sessionStatus[(sid, ssid, s, C)] is not defined or is completed.
2. If sessionStatus[(sid, ssid, s, C)] = interrupted, then sample $k \leftarrow \{0, 1\}^\kappa$.
3. If k is not defined, set $k := k^*$.
4. Set sessionStatus[(sid, ssid, s, C)] := completed.
5. Send (newkey, sid, ssid, k) to s.

As we can see, these interfaces only handle the case where the party's session is marked interrupted and otherwise allow the ideal adversary to choose the session key including allowing the ideal adversary to explicitly have the client output $\perp$. We also modify our partial simulator to use the corresponding newkey interface instead of outputting the parties' keys directly.

Since our partial simulator never uses any of the other interfaces other than stealpwdfile, neither party's session will be marked interrupted and this hybrid is perfectly indistinguishable from the previous hybrid.

Hybrid 7: Offline OPRF Queries In this hybrid, we extract offline guesses from $\mathcal{Z}$'s offline and self-answered OPRF queries to $\mathcal{F}_{\text{opr}, \text{com}}^{\text{com}}$ and modify the partial simulator $\mathcal{S}$ to catalog the successful tests.

In the previous hybrid, whenever $\mathcal{S}$ receives (storepwdfile, sid, C, pwd) from $\mathcal{F}$, $\mathcal{S}$ generates the server's storage (C_0, r_1) honestly and simulates the server honestly relative to this storage. Now, we modify $\mathcal{S}$ to, instead, store pwd and simulate the server relative to this store. This change is currently just bookkeeping as the server's storage is still honestly generated, but in future hybrids we will move to a simulated storage.

In addition to this change, whenever $\mathcal{Z}$ either queries (adv-eval, sid, hlab, ρ^*) to $\mathcal{F}_{\text{opr}, \text{com}}^{\text{com}}$ while s is marked compromised or queries (input-OPRF, sid, ssid, $\mathcal{A}$, ρ^* , r^*) followed by (output-OPRF, sid, ssid, $\mathcal{A}$, hlab) while s is marked compromised, if there exists $\text{pwd}^* \in \mathcal{D}$ such that $\text{RO}^0[(1, \text{pwd}^*)] = \rho^*$, we modify $\mathcal{S}$ to now send (offline-test, sid, pwd*) to $\mathcal{F}_{\text{saPAKE}}$. Finally, in the case where $\mathcal{F}_{\text{saPAKE}}$ returns "correct guess", we modify $\mathcal{S}$ to record the server's password as pwd^* and simulates the server honestly against this password.

Notice that since our partial functionality $\mathcal{F}$ still relays the real server's (`storepwdfile`, `sid`, `C`, `pwd`) query to $\mathcal{S}$, this change only overwrites the server's password with the same password. Further, this change does not change how $\mathcal{S}$ handles `adv-eval` or `output-OPRF` queries on other labels or when `s` is not marked `compromised`. Because of these reasons, this hybrid is perfectly indistinguishable from the previous hybrid.

Hybrid 8: Server Session Dictionary Attacks In this hybrid we modify the partial simulator $\mathcal{S}$ to handle the case where either the client's OPRF session is replaced by an $\mathcal{Z}$ -driven session, or $\mathcal{Z}$ queries the server's online OPRF session without activating the client.

Specifically, we consider the situation where:

- $\mathcal{Z}$ activates `s` with (`PAKE-serversession`, `sid`, `ssid`, `C`) causing (`PAKE-serversession`, `sid`, `ssid`, `C`, `s`) to be relayed to $\mathcal{S}$.
- $\mathcal{Z}$ sends (`input-OPRF`, `sid`, `ssid`, ρ^* , r^*) to $\mathcal{F}_{\text{oprf}}^{\text{com}}$. $\mathcal{Z}$ sends (`deliver-input`, `sid`, `ssid`, $\mathcal{A}$) leading to (`deliver-input`, `ssid`, $\mathcal{A}$) being recorded.

In this situation, if there exists $\text{pwd}^* \in \mathcal{D}$ such that $\rho^* := \text{RO}^0[(1, \text{pwd}^*)]$, we modify $\mathcal{S}$ to send (`testpwd`, `sid`, `ssid`, `s`, pwd^*) to $\mathcal{F}$ and send (`testpwd`, `sid`, `ssid`, `s`, $\perp$) to $\mathcal{F}$ otherwise. Additionally, if ρ^* does not have an appropriate associated random oracle query, instead of faithfully computing the server-to-client authentication tag V , we modify $\mathcal{S}$ to no longer compute Z and instead sample a uniform $V \leftarrow \{0, 1\}^{2\kappa}$ and output key $k \leftarrow \{0, 1\}^\kappa$ for the subsession. Finally, we began recording the server's password in the previous hybrid, so if $\mathcal{F}$ returns “correct guess”, we modify $\mathcal{S}$ to store pwd^* and simulate the server relative to this store.

Now let us verify that the environment's view only changes with negligible probability between the previous hybrid and this hybrid. First, consider the situation where $\mathcal{S}$ sends (`testpwd`, `sid`, `ssid`, `s`, $\perp$) to $\mathcal{F}$. We have that in the previous hybrid, the server sampled $y \leftarrow \mathbb{Z}_q^*$, retrieved storage (C_0, r_1) , and computed

$$\begin{aligned} Z &= y \cdot (C - C_0) \\ &= y \cdot (r^* \cdot G + \rho^* \cdot H - r_0 \cdot G - \rho \cdot H) \\ &= (r^* - r_0) \cdot Y + (\rho^* - \rho) \cdot y \cdot H \\ V &= \text{RO}^1[(1, \text{ssid}, Y, C, Z, r_1)] \\ k &= \text{RO}^2[(\text{ssid}, Y, C, Z, r_1)] \end{aligned}$$

Where $r_0 = \text{RO}^0[(0, \text{OPRF}[\text{hlab}, \rho])]$ and $\rho = \text{RO}^0[(1, \text{pwd})]$. While in this hybrid, the simulated server will sample fresh and independent V and k without updating the random oracle tables or computing Z .

Since $\mathcal{S}$ sent (`testpwd`, `sid`, `ssid`, `s`, $\perp$), we have that ρ^* does not have an appropriate associated random oracle query. Then (as $\mathcal{S}$ still runs the honest server's registration code), we have that $\rho^* - \rho \neq 0$ and if $\mathcal{Z}$ can distinguish with non-negligible probability, we arrive at a reduction to $\text{DDH}(G, A, B)$. To perform this reduction, we replace the honestly sampled OPRF CRS $H \leftarrow \mathbb{G}$ with $H := A$ and replace each subsession's honestly generated $y \leftarrow \mathbb{Z}_q^*$, $Y := y \cdot G$ with $Y := y \cdot B$. This change does not affect other subsessions and does not impair $\mathcal{S}$'s ability to run the simulated server as the simulated server no longer computes Z .

Notice that since in both the previous hybrid and the current hybrid, V and k are uniformly sampled, the only difference is whether they are rederivable from the random oracle interface. Therefore, to distinguish, $\mathcal{Z}$ must make one of the two associated queries to the random oracle. When $\mathfrak{R}_{\text{DDH}}(G, A, B, D)$ observes $\mathcal{Z}$'s query $(\text{ssid}, Y, C, Z^*, r_1)$ to $\mathcal{F}_{\text{RO}}^2$ or $(1, \text{ssid}, Y, C, Z^*, r_1)$ to $\mathcal{F}_{\text{RO}}^1$, the reduction performs the following check:

$$D \stackrel{?}{=} (1/(y \cdot (\rho^* - \rho))) \cdot (Z^* - (r^* - r_0) \cdot Y)$$

which is possible as $\rho^* - \rho \neq 0$ by assumption. If the check succeeds, $\mathfrak{R}_{\text{DDH}}(G, A, B, D)$ outputs 1. If D is a uniformly sampled group element, then it is completely independent of $\mathcal{Z}$'s view and $\mathfrak{R}_{\text{DDH}}(G, A, B, D)$ will output 1 with negligible probability. However, if (A, B, D) is a DH tuple and $Z^* = Z$, the reduction will

calculate

$$\begin{aligned}
& (1/(y \cdot (\rho^* - \rho))) \cdot (Z^* - (r - r_0) \cdot Y) \\
&= (1/(y \cdot (\rho^* - \rho))) \cdot ((r^* - r_0) \cdot Y + (\rho^* - \rho) \cdot y \cdot D - (r^* - r_0) \cdot Y) \\
&= (1/(y \cdot (\rho^* - \rho))) \cdot ((\rho^* - \rho) \cdot y \cdot D) \\
&= D
\end{aligned}$$

Now, consider the situation where $\mathcal{S}$ sends $(\text{testpwd}, \text{sid}, \text{ssid}, \text{s}, \text{pwd}^* \neq \perp)$ to $\mathcal{F}$. If $\mathcal{F}$ returns “incorrect guess” then we have that $\rho^* \neq \rho$ and we may follow the same reduction to DDH as before. If $\mathcal{F}$ returns “correct guess”, then the server will not change its output behavior and will produce the same $k = \text{RO}^2[(\text{ssid}, Y, C, Z, r_1)]$ and $V = \text{RO}^1[(1, \text{ssid}, Y, C, Z, r_1)]$ in both hybrids.

Therefore, under the assumption that the DDH problem is hard in $\mathbb{G}$, this hybrid is indistinguishable from the previous hybrid.

Hybrid 9: Embedding a Trapdoor in the OPRF’s CRS In this hybrid, we modify the partial simulator $\mathcal{S}$ such that whenever $\mathcal{S}$ would sample the commitment key $H \leftarrow \mathbb{G}$, $\mathcal{S}$ samples a trapdoor value $t \leftarrow \mathbb{Z}_q$, sets $H := t \cdot G$, stores $\text{DL}[H] := t$ and $\text{CRS}[\text{sid}] := H$, and proceeds as before.

Since $\{H \leftarrow \mathbb{G}\}$ and $\{H := t \cdot G \mid t \leftarrow \mathbb{Z}_q^*\}$ are indistinguishable distributions, this hybrid is indistinguishable from the previous hybrid.

Hybrid 10: Aborting on Zero Element In this hybrid, we modify the partial simulator $\mathcal{S}$ when, after $\mathcal{A}$ receives $(\text{input-OPRF}, \text{ssid}, \text{p}, C)$, $\mathcal{Z}$ instructs $\mathcal{A}$ to send server-to-client message $(\text{sid}, \text{ssid}, Y^*, V^*)$ such that $Y^* = 0_{\mathbb{G}}$. We modify $\mathcal{S}$ to, in this case, immediately send $(\text{testpwd}, \text{sid}, \text{ssid}, C, \perp)$ to $\mathcal{F}_{\text{saPAKE}}$.

As $\mathcal{S}$ does not currently send $(\text{testpwd}, \text{sid}, \text{ssid}, C, \square)$ in any situation, there is no case where the subsession can be marked compromised. So in all cases, the client’s subsession will be marked interrupted and the client will abort.

Since, in the previous hybrid, the client would have output $(\text{newkey}, \text{sid}, \text{ssid}, \perp)$ and aborted with certainty, this hybrid is perfectly indistinguishable from the previous hybrid.

Hybrid 11: Impersonation Attacks In this hybrid, we modify the partial simulator $\mathcal{S}$ ’s simulation of the client session in the case of an impersonation attack and additionally prepare for replacing the client’s OPRF session. Specifically, when

1. $\mathcal{Z}$ has already compromised s ,
 2. $\mathcal{Z}$ allowed the client to learn the subsession’s OPRF output for the honest label hlab , and
 3. $\mathcal{Z}$ asked $\mathcal{A}$ to send a modified server-to-client message $(\text{sid}, \text{ssid}, Y^*, V^*)$ satisfying some properties,
- we modify $\mathcal{S}$ to send the query $(\text{impersonate}, \text{sid}, \text{ssid})$ to $\mathcal{F}_{\text{saPAKE}}$. Additionally, we modify $\mathcal{S}$ in all client sessions to sample $r_{\mathcal{S}} \leftarrow \mathbb{Z}_q$ and simulate the client relative to $r := r_{\mathcal{S}} - t \cdot \rho'$.

More specifically, we modify $\mathcal{S}$ to send an `impersonate` query when all the following are satisfied:

1. $\mathcal{Z}$ has already sent the query $(\text{stealpwdfile}, \text{sid})$ to s receiving (C_0, r_1) and $(\text{steal-key}, \text{sid}, \text{s})$ to $\mathcal{F}_{\text{oprf}}^{\text{com}}$.
2. C was activated on ssid by $(\text{PAKE-clientsession}, \text{sid}, \text{ssid}, \text{s}, \text{pwd}')$.
3. C queried $\mathcal{F}_{\text{RO}}^0$ receiving $\rho' = \text{RO}^0[(1, \text{pwd}')].$
4. C sent $(\text{input-OPRF}, \text{sid}, \text{ssid}, \rho', r')$ to $\mathcal{F}_{\text{oprf}}^{\text{com}}$.
5. $\mathcal{Z}$ sent $(\text{deliver-input}, \text{sid}, \text{ssid}, C)$ leading to $(\text{deliver-input}, \text{ssid}, C)$ being recorded.
6. $\mathcal{Z}$ sent $(\text{output-OPRF}, \text{sid}, \text{ssid}, C, \text{hlab})$ to $\mathcal{F}_{\text{oprf}}^{\text{com}}$ causing $(\text{output-OPRF}, \text{sid}, \text{ssid}, \square)$ to be sent to C .
7. The client received $(\text{sid}, \text{ssid}, Y^*, V^*)$ from $\mathcal{A}$ as the server-to-client message.
8. s did not send $(\text{sid}, \text{ssid}, Y^*, V^*)$ to C .
9. $Y^* \neq 0_{\mathbb{G}}$.
10. There exists a previous random oracle query $\text{RO}^1[(1, \text{ssid}, Y^*, C_{\mathcal{S}}, Z^*, r_1^*)] = V^*$ made by $\mathcal{Z}$.
11. $r_1^* = r_1$, and

$$\begin{aligned}
& \text{With } t = \text{DL}[\text{CRS}[\text{sid}]], \rho = \text{RO}^0[(1, \text{pwd})], \\
& \text{rwd} = \text{OPRF}[\text{hlab}, \rho], r_0 = \text{RO}^0[(0, \text{rwd})] : \\
& Z^* = (r_{\mathcal{S}} - (r_0 + t \cdot \rho)) \cdot Y^*.
\end{aligned}$$

Note that the partial simulator will ensure that all these table entries are defined at this point of the execution as the client password pwd' and the server password pwd are still within $\mathcal{S}$'s view and $\mathcal{S}$ still honestly generates the server's storage.

Note that the partial functionality $\mathcal{F}$ correctly handles the case where the client session is marked **interrupted** or **compromised** by an **impersonate** query, so we are charged with showing that $(\text{impersonate}, \text{sid}, \text{ssid})$ is only sent at the correct time.

First, let us consider the case where the partial functionality $\mathcal{F}$ responds “correct guess”. In this case, the simulated client should compute $V' = V^*$. To begin, let us take stock of the execution from the simulated client's view:

- The client was activated with $(\text{PAKE-clientsession}, \text{sid}, \text{ssid}, \text{s}, \text{pwd})$.
 - The client queried $\mathcal{F}_{\text{RO}}^0$ receiving $\rho = \text{RO}^0[(1, \text{pwd})]$.
 - The client sent r as its randomness for the OPRF commitment to $\mathcal{F}_{\text{oprf}}^{\text{com}}$.
 - The client received honest OPRF output $(\text{output-OPRF}, \text{sid}, \text{ssid}, \text{rwd})$ from $\mathcal{F}_{\text{oprf}}^{\text{com}}$.
 - The client received $(\text{sid}, \text{ssid}, Y^* \neq 0_{\mathbb{G}}, V^*)$ from $\mathcal{A}$.
 - The client queried $\mathcal{F}_{\text{RO}}^0$, and $\mathcal{F}_{\text{RO}}^1$ receiving $r_0 = \text{RO}^0[(0, \text{rwd})]$ and $r_1 = \text{RO}^1[(0, \text{rwd})]$ respectively.
- The client will then compute (such that $t := \text{DL}[\text{CRS}[\text{sid}]]$)

$$\begin{aligned}
r_1 &= r_1^* \\
Z' &= (r - r_0) \cdot Y^* \\
&= (r_{\mathcal{S}} - (r_0 + t \cdot \rho)) \cdot Y^* \\
&= Z^* \\
V' &= \text{RO}^1[(1, \text{ssid}, Y^*, r \cdot G + \rho \cdot H, Z', r_1)] \\
&= \text{RO}^1[(1, \text{ssid}, Y^*, C_{\mathcal{S}}, Z^*, r_1^*)] \\
&= V^*
\end{aligned}$$

and the client session will have identical output behavior. Because of this, $\mathcal{Z}$'s view does not change between this hybrid and the previous hybrid.

Now, let us consider the case where the partial functionality $\mathcal{F}$ responds “wrong guess”. In this case, the view is set up in the same way as the previous case, but now, the client will be activated with $(\text{PAKE-clientsession}, \text{sid}, \text{ssid}, \text{s}, \text{pwd}')$ where $\text{pwd}' \neq \text{pwd}$. In this case, the client will receive $\rho \neq \rho'$ and $\text{rwd}' \neq \text{rwd}$ as we removed collisions in a previous hybrid and will compute $r_1' \neq r_1^*$ and $V' \neq V^*$. In this case, the client will always abort their session in the previous hybrid and will always abort their session in the current hybrid. The only remaining case is that $\mathcal{Z}$ was able to generate an accepting V^* in the previous hybrid without making the corresponding random oracle query. As no honest party has made this query yet, V' is independent of $\mathcal{Z}$'s view and the two hybrids are identical with overwhelming probability. Therefore, this hybrid is indistinguishable from the previous hybrid.

Hybrid 12: Client Session Dictionary Attacks In this hybrid, we modify the partial simulator $\mathcal{S}$'s simulation of the client session in the case of an online dictionary attack.

To begin, we modify $\mathcal{S}$ in all client sessions to no longer perform the client's OPRF and random oracle queries. Explicitly, whenever the simulated client would query $\mathcal{F}_{\text{RO}}^0$ receiving $\rho' = \text{RO}^0[(1, \text{pwd}')]$ and send $(\text{input-OPRF}, \text{sid}, \text{ssid}, \rho', r)$ to $\mathcal{F}_{\text{oprf}}^{\text{com}}$, $\mathcal{S}$ just sends $(\text{input-OPRF}, \text{sid}, \text{ssid}, C, C_{\mathcal{S}})$ to $\mathcal{A}$ from $\mathcal{F}_{\text{oprf}}^{\text{com}}$. This change must be propagated backwards as ρ' was used to perform the simulated client's actions. We make the following changes to $\mathcal{S}$:

- $\mathcal{S}$ no longer checks if the client queried $\mathcal{F}_{\text{RO}}^0$ receiving $\rho' = \text{RO}^0[(1, \text{pwd}')]$.
- Whenever $\mathcal{S}$ would check if $\mathcal{C}$ sent $(\text{input-OPRF}, \text{sid}, \text{ssid}, \rho', r)$ to $\mathcal{F}_{\text{oprf}}^{\text{com}}$, $\mathcal{S}$ now checks if $(\text{input-OPRF}, \text{sid}, \text{ssid}, C, C_{\mathcal{S}})$ was sent to $\mathcal{A}$ from $\mathcal{F}_{\text{oprf}}^{\text{com}}$.
- Whenever $\mathcal{S}$ would compute $Z' := (r - r_0') \cdot Y^*$, $\mathcal{S}$ now computes
 - $Z' := Z^*$ if a successful $(\text{impersonate}, \text{sid}, \text{ssid})$ query has been made against this subsession.
 - $Z' := Z^*$ if a successful $(\text{testpwd}, \text{sid}, \text{ssid}, C, \text{pwd}^*)$ query has been made against this subsession.

- $Z' := y \cdot (C_S - C_0)$ (for simulated server message ($Y = y \cdot G, V$)) if no $(\text{testpwd}, \text{sid}, \text{ssid}, C, \text{pwd}^*)$ or $(\text{impersonate}, \text{sid}, \text{ssid})$ query has been made against this subsession and $\text{pwd}' = \text{pwd}$.
- $Z' \leftarrow \mathbb{G}$ if no $(\text{testpwd}, \text{sid}, \text{ssid}, C, \text{pwd}^*)$ or $(\text{impersonate}, \text{sid}, \text{ssid})$ query has been made against this subsession and $\text{pwd}' \neq \text{pwd}$.

Additionally, this hybrid modifies the partial simulator $\mathcal{S}$'s code when the following hold

1. C was activated on ssid by $(\text{PAKE-clientsession}, \text{sid}, \text{ssid}, s, \text{pwd}')$.
2. $(\text{input-OPRF}, \text{sid}, \text{ssid}, C, C_S)$ was sent to $\mathcal{A}$ from $\mathcal{F}_{\text{oprf}}^{\text{com}}$.
3. $\mathcal{Z}$ then sent $(\text{output-OPRF}, \text{sid}, \text{ssid}, C, \text{lab})$ to $\mathcal{F}_{\text{oprf}}^{\text{com}}$ causing $(\text{output-OPRF}, \text{sid}, \text{ssid}, \square)$ to be sent to C .
4. $\mathcal{Z}$ asks $\mathcal{A}$ to send a server-to-client message $(\text{sid}, \text{ssid}, Y^*, V^*)$.
5. $Y^* \neq 0_{\mathbb{G}}$.
6. $\text{lab} \neq \text{hlab}$ or $\text{lab} = \text{hlab}$ and the simulated server did not send $(\text{sid}, \text{ssid}, Y^*, V^*)$.
7. If $\text{lab} = \text{hlab}$ and this subsession is not covered in the previous hybrid.

This final check says that when $\text{lab} = \text{hlab}$, at least one of the following holds

- $\mathcal{Z}$ has not sent the query $(\text{stealpwdfile}, \text{sid})$ to s receiving (C_0, r_1) and $(\text{steal-key}, \text{sid}, s)$ to $\mathcal{F}_{\text{oprf}}^{\text{com}}$.
- There does not exist a previous random oracle query $\text{RO}^1[(1, \text{ssid}, Y^*, C_S, Z^*, r_1^*)] = V^*$ made by $\mathcal{Z}$.
- There is such a query, but at least one of the following holds
 - $r_1^* \neq r_1$
 - There do not exist queries to the random oracles and OPRF

$$\begin{aligned}\rho &= \text{RO}^0[(1, \text{pwd})] \\ \text{rwd} &= \text{OPRF}[\text{hlab}, \rho] \\ r_0 &= \text{RO}^0[(0, \text{rwd})]\end{aligned}$$

such that

$$\begin{aligned}\text{With } t &= \text{DL}[\text{CRS}[\text{sid}]], \\ Z^* &= (r_S - (r_0 + t \cdot \rho)) \cdot Y^*.\end{aligned}$$

When all the conditions are met, we modify the partial simulator $\mathcal{S}$ so that when there exists $\text{pwd}^* \in \mathcal{D}$ such that $\rho^* := \text{RO}^0[(1, \text{pwd}^*)]$, $\text{rwd}^* = \text{OPRF}[\text{lab}, \rho^*]$, $r_0^* = \text{RO}^0[(0, \text{rwd}^*)]$, $r_1^* = \text{RO}^1[(0, \text{rwd}^*)]$, $Z^* = (r_S - (r_0^* + t \cdot \rho^*)) \cdot Y^*$ and $V^* = \text{RO}^1[(1, \text{ssid}, Y^*, C, Z^*, r_1^*)]$, $\mathcal{S}$ sends $(\text{testpwd}, \text{sid}, \text{ssid}, C, \text{pwd}^*)$ to $\mathcal{F}_{\text{saPAKE}}$. If no such pwd^* exists, we modify $\mathcal{S}$ to send $(\text{testpwd}, \text{sid}, \text{ssid}, C, \perp)$ to $\mathcal{F}_{\text{saPAKE}}$.

First let us verify that if such a pwd^* exists, it is unique. Let pwd_a and pwd_b be two satisfying passwords. Let $\rho_* := \text{RO}^0[(1, \text{pwd}_*)]$, $\text{rwd}_* := \text{OPRF}[\text{lab}, \rho_*]$, and $r_{1,*} = \text{RO}^1[(0, \text{rwd}_*)]$. As we have removed collisions:

$$\begin{aligned}\text{RO}^1[(1, \text{ssid}, Y^*, C, Z_a, r_{1,a})] &= V^* = \text{RO}^1[(1, \text{ssid}, Y^*, C, Z_b, r_{1,b})] \\ \text{RO}^1[(0, \text{rwd}_a)] &= r_{1,a} = r_{1,b} = \text{RO}^1[(0, \text{rwd}_b)] \\ \text{OPRF}[\text{lab}, \rho_a] &= \text{rwd}_a = \text{rwd}_b = \text{OPRF}[\text{lab}, \rho_b] \\ \text{RO}^0[(1, \text{pwd}_a)] &= \rho_{1,a} = \rho_{1,b} = \text{RO}^0[(1, \text{pwd}_b)]\end{aligned}$$

and $\text{pwd}_a = \text{pwd}_b$ as we expect.

Now, let us consider the situation where $\mathcal{S}$ sends $(\text{testpwd}, \text{sid}, \text{ssid}, C, \text{pwd}^* \neq \perp)$ and the partial functionality $\mathcal{F}$ responds “correct guess”. In the previous hybrid, the client would have computed

$$\begin{aligned}
\rho' &= \rho^* \\
r'_0 &= r_0^* \\
r'_1 &= r_1^* \\
Z' &= (r - r'_0) \cdot Y^* \\
&= (r - r_0^*) \cdot Y^* \\
&= (r_S - (r_0^* + t \cdot \rho^*)) \cdot Y^* \\
&= Z^* \\
V' &= \text{RO}^1[(1, \text{ssid}, Y^*, C_S, Z^*, r_1^*)] \\
&= \text{RO}^1[(1, \text{ssid}, Y^*, C, Z^*, r_1^*)] \\
&= V^*
\end{aligned}$$

and the client session will abort in neither this hybrid nor the previous. In the case where $\mathcal{S}$ sends $(\text{testpwd}, \text{sid}, \text{ssid}, C, \text{pwd}^* \neq \perp)$ and $\mathcal{F}$ responds “wrong guess” the client would have received $\rho \neq \rho'$ and $\text{pwd}' \neq \text{pwd}$, as we removed collisions in a previous hybrid, and would have computed $r'_1 \neq r_1^*$ and $V' \neq V^*$ in the previous hybrid. When this happens, the client would abort their session and will abort their session in the current hybrid.

Finally, let us account for the honest and impersonated sessions:

- In the case where a successful $(\text{impersonate}, \text{sid}, \text{ssid})$ query has been made against this subsession, we have that $\text{pwd}' = \text{pwd}$ and, as shown previously, the client will compute

$$\begin{aligned}
Z' &= (r - r_0) \cdot Y^* \\
&= (r_S - (r_0 + t \cdot \rho)) \cdot Y^* \\
&= Z^*
\end{aligned}$$

in both hybrids.

- If no $(\text{testpwd}, \text{sid}, \text{ssid}, C, \text{pwd}^*)$ or $(\text{impersonate}, \text{sid}, \text{ssid})$ queries have been made against this subsession and $\text{pwd}' = \text{pwd}$, we have that the client received the simulated server’s message $(Y = y \cdot G, V)$ and would compute

$$\begin{aligned}
Z' &= (r - r_0) \cdot Y \\
&= (r_S - (r_0 + t \cdot \rho)) \cdot y \cdot G \\
&= y \cdot (C_S - C_0)
\end{aligned}$$

in both hybrids.

- If no $(\text{testpwd}, \text{sid}, \text{ssid}, C, \text{pwd}^*)$ or $(\text{impersonate}, \text{sid}, \text{ssid})$ queries have been made against this subsession and $\text{pwd}' \neq \text{pwd}$, we have that the client received the simulated server’s message $(Y = y \cdot G, V)$ and would compute, such that $\rho' \neq \rho$ and $r'_0 \neq r_0$,

$$\begin{aligned}
Z' &= (r - r'_0) \cdot Y \\
&= ((r + t \cdot \rho) - (r'_0 + t \cdot \rho)) \cdot y \cdot G \\
&\neq (r_S - (r_0 + t \cdot \rho)) \cdot y \cdot G \\
&\neq y \cdot (C - C_0).
\end{aligned}$$

in the previous hybrid except with negligible probability.

Z' ’s view in the previous hybrid and the current hybrid diverge exactly when Z was capable of deriving an accepting $V^* = V'$ without making the requisite queries. As $V' \in \{0, 1\}^{2\kappa}$, $r'_0 \in \mathbb{Z}_q$, and $r'_1 \in \{0, 1\}^{2\kappa}$, this happens with negligible probability.

Hybrid 13: Simulate Server Storage In this hybrid, we modify our partial simulator $\mathcal{S}$ to stop honestly generating the server's password file and rely on a simulated file (initially) independent of the server's password. We then modify $\mathcal{S}$ to perform an equivocation step whenever a valid server password guess is extracted from $\mathcal{Z}$'s queries to $\mathcal{F}_{\text{opr}}^{\text{com}}$.

Recall that up until this hybrid, $\mathcal{S}$ runs the honest server's code and makes the requisite OPRF and random oracle queries to generate the password file. This is possible since the partial functionality $\mathcal{F}$ still relays the registration query (`storepwdfile`, `sid`, `C`, `pwd`) to $\mathcal{S}$. However, in preparation for modifying $\mathcal{F}$, we must simulate the server's storage without initially knowing `pwd`.

To allow this change, we modify $\mathcal{S}$ so that whenever the server's storage is generated resulting in $\mathcal{S}$ initially recording the server's password `pwd` and storing `file[sid] := (C_0, r_1) = (r_0 \cdot G + \rho \cdot H, r_1)`, $\mathcal{S}$ samples $e \leftarrow \mathbb{Z}_q$ and $r_1 \leftarrow \{0, 1\}^{2\kappa}$ and stores $(C_0 = e \cdot G, r_1)$ and $\text{DL}[C_0] = e$ instead. Additionally, whenever $\mathcal{S}$ would record the server's password `pwd`, $\mathcal{S}$ runs the following code:

1. Retrieve $\rho := \text{RO}^0[(1, \text{pwd})]$.
2. If $\text{OPRF}[\text{hlab}, \rho]$ is not defined, sample $\text{OPRF}[\text{hlab}, \rho] \leftarrow \{0, 1\}^{2\kappa}$.
3. Retrieve $(C_0, r_1) := \text{file}[\text{sid}]$, $e := \text{DL}[C_0]$, and $H := \text{CRS}[\text{sid}]$.
4. Set $\text{RO}^0[(0, \text{OPRF}[\text{hlab}, \rho])] := e - \text{DL}[H] \cdot \rho$.
5. Set $\text{RO}^1[(0, \text{OPRF}[\text{hlab}, \rho])] := r_1$.

$\mathcal{S}$ continues to abort on collisions as usual. Recall from our previous hybrid that the only time where this equivocation code would be run causing modifications to the random oracle and OPRF input-output pairs is when $\text{OPRF}[\text{hlab}, \text{RO}^0[(1, \text{pwd})]]$ would be uniformly sampled. Therefore, $\text{RO}^0[(1, \text{pwd})]$ is defined while, with overwhelming probability, $\text{OPRF}[\text{hlab}, \rho]$, $\text{RO}^0[(0, \text{OPRF}[\text{hlab}, \rho])]$, and $\text{RO}^1[(0, \text{OPRF}[\text{hlab}, \rho])]$ are not defined.

First, let us recall that in the previous hybrid, the server's storage was a Pedersen commitment to $\rho = \text{RO}^0[(1, \text{pwd})]$ with randomness $r_0 = \text{RO}^0[(0, \text{OPRF}[\text{hlab}, \rho])]$. We leverage the fact that any group element $C_0 \in \mathbb{G}$ represents a Pedersen commitment to any value $c \in \mathbb{Z}_q$ to simulate the server's storage and equivocate. Specifically, once $\mathcal{S}$ would record the server's password `pwd` (and therefore knows $\rho = \text{RO}^0[(1, \text{pwd})]$), $\mathcal{S}$ leverages the OPRF trapdoor (H, t) and knowledge that $C_0 = e \cdot G$ to explain C_0 as a commitment to ρ . Specifically, by programming $\text{RO}^0[(0, \text{OPRF}[\text{hlab}, \rho])] := e - \text{DL}[H] \cdot \rho$, we have that

$$\begin{aligned} C_0 &= e \cdot G \\ &= (e - \rho \cdot t + \rho \cdot t) \cdot G \\ &= (e - \rho \cdot t) \cdot G + \rho \cdot t \cdot G \\ &= \text{RO}^0[(0, \text{OPRF}[\text{hlab}, \rho])] \cdot G + \rho \cdot H \\ &= r_0 \cdot G + \rho \cdot H \end{aligned}$$

which is a Pedersen commitment to ρ under randomness r_0 as in the previous hybrid. All newly programmed random oracle outputs continue to be sampled consistently with the previous hybrid.

Second, recall that in the previous hybrid when $\mathcal{S}$ determined that an impersonation attack was occurring against the client, $\mathcal{S}$ relied on the honest server's random oracle and OPRF pairs being programmed previously. We now modify how $\mathcal{S}$ determines an impersonation attack is occurring. In the previous hybrid, $\mathcal{S}$ checked that for $\mathcal{A}$ -modified server-to-client message $(\text{sid}, \text{ssid}, Y^*, V^*)$, simulated client OPRF session with commitment $C_S = r_S \cdot G$, and s marked `compromised`,

- There existed a previous random oracle query $\text{RO}^1[(1, \text{ssid}, Y^*, C_S, Z^*, r_1^*)] = V^*$ made by $\mathcal{Z}$.
- $r_1^* = r_1$, and

$$\begin{aligned} \text{With } t &= \text{DL}[\text{CRS}[\text{sid}]], \rho = \text{RO}^0[(1, \text{pwd})], \\ \text{rwd} &= \text{OPRF}[\text{hlab}, \rho], r_0 = \text{RO}^0[(0, \text{rwd})] : \\ Z^* &= (r_S - (r_0 + t \cdot \rho)) \cdot Y^*. \end{aligned}$$

However, we can no longer guarantee that $\text{RO}^0[(1, \text{pwd})]$, $\text{rwd} = \text{OPRF}[\text{hlab}, \rho]$, and $r_0 = \text{RO}^0[(0, \text{rwd})]$ are set as the server's OPRF and random oracle queries are no longer assured to have been run. To account for this case, $\mathcal{S}$ now performs the following checks:

- There existed a previous random oracle query $\text{RO}^1[(1, \text{ssid}, Y^*, C_S, Z^*, r_1^*)] = V^*$ made by $\mathcal{Z}$.
- $r_1^* = r_1$, and

$$Z^* = (r_S - \text{DL}[C_0]) \cdot Y^*.$$

In the previous hybrid, we have that $(r_0 + t \cdot \rho) = \text{DL}[C_0]$ and the two views are identical.

Finally, recall that in the previous hybrid when $\mathcal{S}$ determined that a dictionary attack was occurring against the server, $\mathcal{S}$ verified that

- $\mathcal{Z}$ activated s with $(\text{PAKE-serversession}, \text{sid}, \text{ssid}, C)$ causing $(\text{PAKE-serversession}, \text{sid}, \text{ssid}, C, s)$ to be relayed to $\mathcal{S}$.
- $\mathcal{Z}$ sent $(\text{input-OPRF}, \text{sid}, \text{ssid}, \rho^*, r^*)$ to $\mathcal{F}_{\text{oprf}}^{\text{com}}$ causing $(\text{input-OPRF}, \text{sid}, \text{ssid}, \mathcal{A}, C^* = r^* \cdot G + \rho^* \cdot H)$ to be relayed to $\mathcal{A}$.
- There existed $\text{pwd}^* \in \mathcal{D}$ such that $\rho^* := \text{RO}^0[(1, \text{pwd}^*)]$.

The partial simulator in the previous hybrid relied on the argument that if ρ^* does not have an appropriate associated random oracle query, then since the previous simulator ran the honest server's code, we had $\rho^* - \rho \neq 0$. This hybrid and the previous hybrid then diverge exactly when $\mathcal{Z}$ sends $(\text{input-OPRF}, \text{sid}, \text{ssid}, \rho, r^*)$ in the previous hybrid before making the corresponding random oracle query to learn ρ . Recall that in the previous hybrid, the simulated client's commitments were simulated as $r_S \cdot G$ independently of any ρ' . Similarly, as r_0 is uniformly sampled, ρ is then independent of $\mathcal{Z}$'s view, and $\mathcal{Z}$ will query $(\text{input-OPRF}, \text{sid}, \text{ssid}, \rho, r^*)$ with negligible probability. Therefore, this hybrid is indistinguishable from the previous hybrid.

Additionally, note that after this modification, the simulated server's behavior does not rely on the server's password pwd in honest sessions.

Hybrid 14: Align Fresh Session Keys In this hybrid, we modify the partial functionality $\mathcal{F}$ to align the parties' session keys when both sessions are fresh when their respective **newkey** queries are sent and use the same password. Specifically, we update both **clientnewkey** and **servernewkey** to the following:

Changes to newkey

On $(\text{clientnewkey}, \text{sid}, \text{ssid}, k^*)$ **from** $\mathcal{A}^*$ **where** $k^* \in \{0, 1\}^\kappa \cup \{\perp\}$.

1. Ignore this query if $\text{sessionStatus}[(\text{sid}, \text{ssid}, C, s)]$ is not defined or is completed.
2. If $\text{sessionStatus}[(\text{sid}, \text{ssid}, C, s)] = \text{interrupted}$, then set $k := \perp$.
3. If $\text{sessionStatus}[(\text{sid}, \text{ssid}, C, s)] = \text{fresh}$,
there is a counter-session $\text{session}[(\text{sid}, \text{ssid}, s, C)] = \text{session}[(\text{sid}, \text{ssid}, C, s)]$,
and $\text{sessionStatus}[(\text{sid}, \text{ssid}, s, C)]$ was fresh when $(\text{newkey}, \text{sid}, \text{ssid}, k')$ was sent to s , set $k := k'$.
4. If k is not defined, set $k := k^*$.
5. Set $\text{sessionStatus}[(\text{sid}, \text{ssid}, C, s)] := \text{completed}$.
6. Send $(\text{newkey}, \text{sid}, \text{ssid}, k)$ to C .

On $(\text{servernewkey}, \text{sid}, \text{ssid}, k^*)$ **from** $\mathcal{A}^*$ **where** $k^* \in \{0, 1\}^\kappa$.

1. Ignore this query if $\text{sessionStatus}[(\text{sid}, \text{ssid}, s, C)]$ is not defined or is completed.
2. If $\text{sessionStatus}[(\text{sid}, \text{ssid}, s, C)] = \text{interrupted}$, then sample $k \leftarrow \{0, 1\}^\kappa$.
3. If $\text{sessionStatus}[(\text{sid}, \text{ssid}, s, C)] = \text{fresh}$,
there is a counter-session $\text{session}[(\text{sid}, \text{ssid}, C, s)] = \text{session}[(\text{sid}, \text{ssid}, s, C)]$,
and $\text{sessionStatus}[(\text{sid}, \text{ssid}, C, s)]$ was fresh when $(\text{newkey}, \text{sid}, \text{ssid}, k')$ was sent to C , set $k := k'$.
4. If k is not defined, set $k := k^*$.
5. Set $\text{sessionStatus}[(\text{sid}, \text{ssid}, s, C)] := \text{completed}$.
6. Send $(\text{newkey}, \text{sid}, \text{ssid}, k)$ to s .

In the previous hybrid, we have that

1. If $\mathcal{Z}$ sends $(\text{output-OPRF}, \text{sid}, \text{ssid}, \text{lab} \neq \text{hlab})$ to $\mathcal{F}_{\text{oprf}}^{\text{com}}$ causing $(\text{output-OPRF}, \text{sid}, \text{ssid}, \square)$ to be sent to C , $\mathcal{S}$ will send $(\text{testpwd}, \text{sid}, \text{ssid}, C, \square)$ to $\mathcal{F}$ before $(\text{clientnewkey}, \text{sid}, \text{ssid}, \square)$.
2. If $\mathcal{Z}$ sends $(\text{output-OPRF}, \text{sid}, \text{ssid}, \text{hlab})$ to $\mathcal{F}_{\text{oprf}}^{\text{com}}$ causing $(\text{output-OPRF}, \text{sid}, \text{ssid}, \square)$ to be sent to C and $\mathcal{A}$ does not relay $\mathcal{S}$'s server-to-client message, $\mathcal{S}$ will send $(\text{testpwd}, \text{sid}, \text{ssid}, C, \square)$ or $(\text{impersonate}, \text{sid}, \text{ssid})$ to $\mathcal{F}$ before $(\text{clientnewkey}, \text{sid}, \text{ssid}, \square)$.

3. If $\mathcal{Z}$ sends $(\text{input-OPRF}, \text{sid}, \text{ssid}, \rho^*, r^*)$ to $\mathcal{F}_{\text{oprf}}^{\text{com}}$ followed by $(\text{deliver-input}, \text{sid}, \text{ssid})$ or sends $(\text{mitm}, \text{sid}, \text{ssid}, \rho^*, r^*)$ causing, in either case, $(\text{input-OPRF}, \text{sid}, \text{ssid}, r^* \cdot G + \rho^* \cdot H)$ to be sent to $\mathcal{S}$, $\mathcal{S}$ will send $(\text{testpwd}, \text{sid}, \text{ssid}, \mathcal{S}, \square)$ to $\mathcal{F}$ before $(\text{servernewkey}, \text{sid}, \text{ssid}, \square)$.

It is then the case that both changes in this hybrid only affect honest subsessions where the two parties input the same password. As we have shown earlier, in this case, both parties in the previous hybrid derived the same Z , r_1 , and then derived the same V and k . Therefore, this hybrid is perfectly indistinguishable from the previous hybrid.

Note that at this point, there are still some differences between the partial simulator $\mathcal{F}$ and the full simulator $\mathcal{F}_{\text{saPAKE}}$:

1. When $\mathcal{F}_{\text{saPAKE}}$ receives $(\text{storepwdfile}, \text{sid}, \mathcal{C}, \text{pwd})$ from $\mathcal{S}$, it does not relay the query to $\mathcal{S}$.
2. When $\mathcal{F}_{\text{saPAKE}}$ receives $(\text{PAKE-clientsession}, \text{sid}, \text{ssid}, \mathcal{S}, \text{pwd}')$ from $\mathcal{C}$, it does not relay the full query to $\mathcal{S}$.
3. When $\mathcal{F}_{\text{saPAKE}}$ receives $(\text{clientnewkey}, \text{sid}, \text{ssid}, k^*)$ from $\mathcal{A}^*$, the corresponding client session is **fresh**, the server's counter-session used the same password, and this is the first **newkey** query for the subsession, $\mathcal{F}_{\text{saPAKE}}$ samples $k \leftarrow \{0, 1\}^\kappa$.
4. When $\mathcal{F}_{\text{saPAKE}}$ receives $(\text{clientnewkey}, \text{sid}, \text{ssid}, k^*)$ from $\mathcal{A}^*$, the corresponding client session is **fresh**, and there is no server counter-session using the same password, $\mathcal{F}_{\text{saPAKE}}$ sets $k := \perp$.
5. When $\mathcal{F}_{\text{saPAKE}}$ receives $(\text{servernewkey}, \text{sid}, \text{ssid}, k^*)$ from $\mathcal{A}^*$, the corresponding server session is **fresh**, and this is the first **newkey** query for the subsession, $\mathcal{F}_{\text{saPAKE}}$ samples $k \leftarrow \{0, 1\}^\kappa$.

Hybrid 15: Fresh Session Keys In this hybrid, we finish building the partial functionality $\mathcal{F}$'s **newkey** interfaces by handling the settings (3), (4), and (5) as outlined in the previous hybrid. Explicitly, we update both **clientnewkey** and **servernewkey** so they are exactly as in $\mathcal{F}_{\text{saPAKE}}$:

Changes to newkey

On $(\text{clientnewkey}, \text{sid}, \text{ssid}, k^*)$ from $\mathcal{A}^*$ where $k^* \in \{0, 1\}^\kappa$.

1. Ignore this query if $\text{sessionStatus}[(\text{sid}, \text{ssid}, \mathcal{C}, \mathcal{S})]$ is not defined or is completed.
2. If $\text{sessionStatus}[(\text{sid}, \text{ssid}, \mathcal{C}, \mathcal{S})] = \text{compromised}$, set $k := k^*$.
3. If $\text{sessionStatus}[(\text{sid}, \text{ssid}, \mathcal{C}, \mathcal{S})] = \text{fresh}$, there is a counter-session $\text{session}[(\text{sid}, \text{ssid}, \mathcal{S}, \mathcal{C})] = \text{session}[(\text{sid}, \text{ssid}, \mathcal{C}, \mathcal{S})]$, and $\text{sessionStatus}[(\text{sid}, \text{ssid}, \mathcal{S}, \mathcal{C})]$ was **fresh** when $(\text{newkey}, \text{sid}, \text{ssid}, k')$ was sent to $\mathcal{S}$, set $k := k'$.
4. If $\text{sessionStatus}[(\text{sid}, \text{ssid}, \mathcal{C}, \mathcal{S})] = \text{interrupted}$, or is fresh and there is no counter-session $\text{session}[(\text{sid}, \text{ssid}, \mathcal{S}, \mathcal{C})] = \text{session}[(\text{sid}, \text{ssid}, \mathcal{C}, \mathcal{S})]$, then set $k := \perp$.
5. If k is not defined, sample $k \leftarrow \{0, 1\}^\kappa$.
6. Set $\text{sessionStatus}[(\text{sid}, \text{ssid}, \mathcal{C}, \mathcal{S})] := \text{completed}$.
7. Send $(\text{newkey}, \text{sid}, \text{ssid}, k)$ to $\mathcal{C}$.

On $(\text{servernewkey}, \text{sid}, \text{ssid}, k^*)$ from $\mathcal{A}^*$ where $k^* \in \{0, 1\}^\kappa$.

1. Ignore this query if $\text{sessionStatus}[(\text{sid}, \text{ssid}, \mathcal{S}, \mathcal{C})]$ is not defined or is completed.
2. If $\text{sessionStatus}[(\text{sid}, \text{ssid}, \mathcal{S}, \mathcal{C})] = \text{compromised}$, set $k := k^*$.
3. If $\text{sessionStatus}[(\text{sid}, \text{ssid}, \mathcal{S}, \mathcal{C})] = \text{fresh}$, there is a counter-session $\text{session}[(\text{sid}, \text{ssid}, \mathcal{C}, \mathcal{S})] = \text{session}[(\text{sid}, \text{ssid}, \mathcal{S}, \mathcal{C})]$, and $\text{sessionStatus}[(\text{sid}, \text{ssid}, \mathcal{C}, \mathcal{S})]$ was **fresh** when $(\text{newkey}, \text{sid}, \text{ssid}, k')$ was sent to $\mathcal{C}$, set $k := k'$.
4. If $\text{sessionStatus}[(\text{sid}, \text{ssid}, \mathcal{S}, \mathcal{C})] = \text{interrupted}$, then sample .
5. If k is not defined, set $k \leftarrow \{0, 1\}^\kappa$.
6. Set $\text{sessionStatus}[(\text{sid}, \text{ssid}, \mathcal{S}, \mathcal{C})] := \text{completed}$.
7. Send $(\text{newkey}, \text{sid}, \text{ssid}, k)$ to $\mathcal{S}$.

Additionally, we modify our partial simulator $\mathcal{S}$ to no longer compute Z , r_1 , r_0 , V , or k for either party in an honest session. We then modify the simulated server to generate V and k just as in the case where $(\text{testpwd}, \text{sid}, \text{ssid}, \mathcal{S}, \perp)$ is sent. We also modify the simulated client so that neither the client authentication tag check $V' \stackrel{?}{=} V$ nor client's key k' is calculated; instead, $\mathcal{S}$ simply sets the client key k' to be the simulated

server's key k for the subsession (which is guaranteed to be defined). Notice that $\mathcal{Z}$ can only distinguish between the previous hybrid and the current hybrid if the client session outputs $\perp$ in one hybrid and not the other, or $\mathcal{Z}$ can distinguish a fresh subsession key from uniform.

First, let us consider the case for the **fresh** client. Recall that in the previous hybrid, we have the following fresh client behavior:

1. If no `(testpwd, sid, ssid, C, pwd*)` or `(impersonate, sid, ssid)` queries has been made against this subsession, $\text{pwd}' = \text{pwd}$, and the client received the simulated server's message $(Y = y \cdot G, V)$, then simulated client will compute $Z' = y \cdot (C_S - C_0) = Z$.
2. If no `(testpwd, sid, ssid, C, pwd*)` or `(impersonate, sid, ssid)` queries has been made against this subsession and $\text{pwd}' \neq \text{pwd}$, the simulated client would have computed $Z' \leftarrow \mathbb{G}$ and $V' \neq V$ with overwhelming probability.

In the case where the parties' passwords differ, the simulated client will abort with overwhelming probability in both hybrids. In the case where the parties' passwords match, notice that in both the previous hybrid and the current hybrid k is uniformly sampled with the only difference being if it is rederivable from the random oracle interface. To distinguish, $\mathcal{Z}$ must make the associated query to the random oracle. If $\mathcal{Z}$ can distinguish the client's output key k from uniform with non-negligible probability, then we arrive at a reduction to DDH(G, A, B). As this reduction is similar to the reduction for the server's outputs, we will elide the reduction and show the analogous reduction below. Additionally, note that after this modification, the simulated client's behavior does not rely on the client's password pwd' or the server's password pwd in honest sessions.

Finally, let us consider the case for the **fresh** server. Notice that since in both the previous hybrid and the current hybrid, V and k are uniformly sampled, the only difference is whether they are rederivable from the random oracle interface. Therefore, to distinguish, $\mathcal{Z}$ must make one of the two associated queries to the random oracle. First, let us note that

$$(Y, C_S, Z) = (y \cdot G, r_S \cdot G, y \cdot r_S \cdot G - y \cdot e \cdot G)$$

from which we can extract the DH tuple $(Y, C_S, Z + e \cdot Y)$. To carry out the reduction, we modify the partial simulator to replace C_S with $s_2 \cdot B$ and Y with $s_1 \cdot B$ for subsession-specific $s_1, s_2 \leftarrow \mathbb{Z}_q$. When $\mathfrak{R}_{\text{DDH}}(G, A, B, D)$ observes $\mathcal{Z}$'s query `(ssid, Y, C_S, Z, r_1)` to $\mathcal{F}_{\text{RO}}^2$ or `(1, ssid, Y, C_S, Z, r_1)` to $\mathcal{F}_{\text{RO}}^1$, the reduction performs the following check:

$$D \stackrel{?}{=} (1/(s_1 \cdot s_2)) \cdot (Z + e \cdot Y)$$

If the check succeeds, $\mathfrak{R}_{\text{DDH}}(G, A, B, D)$ outputs 1. If D is a uniformly sampled group element, then it is completely independent of $\mathcal{Z}$'s view and $\mathfrak{R}_{\text{DDH}}(G, A, B, D)$ will output 1 with negligible probability. However, if (A, B, D) is a DH tuple and $Z^* = Z$, the reduction will calculate D as expected.

Therefore, under the assumption that the DDH problem is hard in $\mathbb{G}$, this hybrid is indistinguishable from the previous hybrid.

Hybrid 16: Generate the Server's Storage Dynamically Recall that in the ideal world relative to $\mathcal{F}_{\text{saPAKE}}$, the simulator $\mathcal{S}_{\text{saPAKE}}$ is not notified when the server is activated by $\mathcal{Z}$ and therefore will not know when to generate the server's storage. To prepare the following hybrid where we remove the `(storepwdfile, sid, C, pwd)` query from $\mathcal{S}$'s view, we delay the generation of the server's storage until $\mathcal{S}$ receives an activation query for an honest party or receives a query from $\mathcal{Z}$ (possibly through $\mathcal{A}$).

As this is simply changing the order that actions are taken in, the server's storage will be generated whenever $\mathcal{S}$ relies on it, and none of these actions notify $\mathcal{Z}$, this hybrid is perfectly indistinguishable from the previous hybrid.

Hybrid 17: Remove Both Parties' Passwords from the Simulator's View In this hybrid, we align the partial functionality $\mathcal{F}$ and the full functionality $\mathcal{F}_{\text{saPAKE}}$ by modifying $\mathcal{F}$ so that

- Instead of relaying `(PAKE-clientsession, sid, ssid, s, pwd')` to $\mathcal{S}$, $\mathcal{F}$ now sends `(PAKE-clientsession, sid, ssid, C, s)` to $\mathcal{S}$.
- Instead of relaying `(storepwdfile, sid, C, pwd)` to $\mathcal{S}$, $\mathcal{F}$ now sends nothing to $\mathcal{S}$.

The effect that this change has is that the real parties' passwords no longer begin in $\mathcal{S}$'s view. However, if we look at $\mathcal{S}$ at this stage, we will see that the simulated parties don't use the passwords as received from the functionality and instead extract them from $\mathcal{Z}$'s actions. So this change is only to align with the ideal functionality and this hybrid is perfectly indistinguishable from the previous hybrid.

Ideal World In this hybrid, we have that the partial functionality $\mathcal{F}$ we've been building in each hybrid is now exactly $\mathcal{F}_{\text{saPAKE}}$ and the partial simulator $\mathcal{S}$ we've been building in each hybrid is exactly now $\mathcal{S}_{\text{saPAKE}}$.

$$\text{IDEAL}_{\mathcal{F}_{\text{saPAKE}}, \mathcal{S}_{\text{saPAKE}}, \mathcal{A}, \mathcal{Z}}$$

□

* Acknowledgements. This research received support from: the European Research Council (ERC) under the European Unions's Horizon 2020 research and innovation programme under grant agreement No 101124977 (DECRYPSYS); the Danish Independent Research Council under Grant-ID DFF-3103-00077B (CryptoDigi); the Danish Independent Research Council under Grant-ID DFF-2064-00016B (YOSO).

References

1. Abdalla, M., Barbosa, M., Bradley, T., Jarecki, S., Katz, J., Xu, J.: Universally composable relaxed password authenticated key exchange. In: Micciancio, D., Ristenpart, T. (eds.) *Advances in Cryptology – CRYPTO 2020, Part I. Lecture Notes in Computer Science*, vol. 12170, pp. 278–307. Springer, Heidelberg (Aug 2020). https://doi.org/10.1007/978-3-030-56784-2_10
2. Abram, D., Nof, A., Orlandi, C., Scholl, P., Shlomovits, O.: Low-bandwidth threshold ECDSA via pseudorandom correlation generators. In: 2022 IEEE Symposium on Security and Privacy. pp. 2554–2572. IEEE Computer Society Press (May 2022). <https://doi.org/10.1109/SP46214.2022.9833559>
3. Adjedj, M., Blokh, C., Couteau, G., Joux, A., Makriyannis, N.: Two-round 2pc ECDSA at the cost of 1 OLE. *IACR Cryptol. ePrint Arch.* p. 1950 (2024), <https://eprint.iacr.org/2024/1950>
4. Albrecht, M.R., Davidson, A., Deo, A., Smart, N.P.: Round-optimal verifiable oblivious pseudorandom functions from ideal lattices. In: Garay, J. (ed.) *PKC 2021: 24th International Conference on Theory and Practice of Public Key Cryptography, Part II. Lecture Notes in Computer Science*, vol. 12711, pp. 261–289. Springer, Heidelberg (May 2021). https://doi.org/10.1007/978-3-030-75248-4_10
5. Barreto, P.S.L.M., Lynn, B., Scott, M.: Constructing elliptic curves with prescribed embedding degrees. In: Cimato, S., Galdi, C., Persiano, G. (eds.) *SCN 02: 3rd International Conference on Security in Communication Networks. Lecture Notes in Computer Science*, vol. 2576, pp. 257–267. Springer, Heidelberg (Sep 2003). https://doi.org/10.1007/3-540-36413-7_19
6. Barreto, P.S.L.M., Naehrig, M.: Pairing-friendly elliptic curves of prime order. In: Preneel, B., Tavares, S. (eds.) *SAC 2005: 12th Annual International Workshop on Selected Areas in Cryptography. Lecture Notes in Computer Science*, vol. 3897, pp. 319–331. Springer, Heidelberg (Aug 2006). https://doi.org/10.1007/11693383_22
7. Barthe, G., Fagerholm, E., Fiore, D., Mitchell, J.C., Scedrov, A., Schmidt, B.: Automated analysis of cryptographic assumptions in generic group models. In: Garay, J.A., Gennaro, R. (eds.) *Advances in Cryptology – CRYPTO 2014, Part I. Lecture Notes in Computer Science*, vol. 8616, pp. 95–112. Springer, Heidelberg (Aug 2014). https://doi.org/10.1007/978-3-662-44371-2_6
8. Bellare, S.M., Merritt, M.: Augmented encrypted key exchange: A password-based protocol secure against dictionary attacks and password file compromise. In: Denning, D.E., Pyle, R., Ganesan, R., Sandhu, R.S., Ashby, V. (eds.) *ACM CCS 93: 1st Conference on Computer and Communications Security*. pp. 244–250. ACM Press (Nov 1993). <https://doi.org/10.1145/168588.168618>
9. Benhamouda, F.: Diverse modules and zero-knowledge. Ph.D. thesis, Université Paris sciences et lettres (2016)
10. Benhamouda, F., Blazy, O., Chevalier, C., Pointcheval, D., Vergnaud, D.: New techniques for SPHF and efficient one-round PAKE protocols. In: Canetti, R., Garay, J.A. (eds.) *Advances in Cryptology – CRYPTO 2013, Part I. Lecture Notes in Computer Science*, vol. 8042, pp. 449–475. Springer, Heidelberg (Aug 2013). https://doi.org/10.1007/978-3-642-40041-4_25
11. Boneh, D., Boyen, X.: Secure identity based encryption without random oracles. In: Franklin, M. (ed.) *Advances in Cryptology – CRYPTO 2004. Lecture Notes in Computer Science*, vol. 3152, pp. 443–459. Springer, Heidelberg (Aug 2004). https://doi.org/10.1007/978-3-540-28628-8_27

12. Boneh, D., Boyen, X.: Short signatures without random oracles. In: Cachin, C., Camenisch, J. (eds.) *Advances in Cryptology – EUROCRYPT 2004*. Lecture Notes in Computer Science, vol. 3027, pp. 56–73. Springer, Heidelberg (May 2004). https://doi.org/10.1007/978-3-540-24676-3_4
13. Bourdrez, D., Krawczyk, H., Lewi, K., Wood, C.: The opaque asymmetric pake protocol (2023), <https://cfrg.github.io/draft-irtf-cfrg-opaque/draft-irtf-cfrg-opaque.html>, <https://cfrg.github.io/draft-irtf-cfrg-opaque/draft-irtf-cfrg-opaque.html>
14. Bradley, T., Jarecki, S., Xu, J.: Strong asymmetric PAKE based on trapdoor CKEM. In: Boldyreva, A., Micciancio, D. (eds.) *Advances in Cryptology – CRYPTO 2019*, Part III. Lecture Notes in Computer Science, vol. 11694, pp. 798–825. Springer, Heidelberg (Aug 2019). https://doi.org/10.1007/978-3-030-26954-8_26
15. Camenisch, J., Lehmann, A.: Privacy-preserving user-auditable pseudonym systems. In: 2017 IEEE European Symposium on Security and Privacy, EuroS&P 2017, Paris, France, April 26–28, 2017. pp. 269–284. IEEE (2017). <https://doi.org/10.1109/EUROSP.2017.36>, <https://doi.org/10.1109/EuroSP.2017.36>
16. Canetti, R.: Universally composable security: A new paradigm for cryptographic protocols. In: 42nd Annual Symposium on Foundations of Computer Science. pp. 136–145. IEEE Computer Society Press (Oct 2001). <https://doi.org/10.1109/SFCS.2001.959888>
17. Canetti, R., Gennaro, R., Goldfeder, S., Makriyannis, N., Peled, U.: UC non-interactive, proactive, threshold ECDSA with identifiable aborts. In: Ligatti, J., Ou, X., Katz, J., Vigna, G. (eds.) *ACM CCS 2020: 27th Conference on Computer and Communications Security*. pp. 1769–1787. ACM Press (Nov 2020). <https://doi.org/10.1145/3372297.3423367>
18. Canetti, R., Halevi, S., Katz, J., Lindell, Y., MacKenzie, P.D.: Universally composable password-based key exchange. In: Cramer, R. (ed.) *Advances in Cryptology – EUROCRYPT 2005*. Lecture Notes in Computer Science, vol. 3494, pp. 404–421. Springer, Heidelberg (May 2005). https://doi.org/10.1007/11426639_24
19. Casacuberta, S., Hesse, J., Lehmann, A.: Sok: Oblivious pseudorandom functions. In: 7th IEEE European Symposium on Security and Privacy, EuroS&P 2022, Genoa, Italy, June 6–10, 2022. pp. 625–646. IEEE (2022). <https://doi.org/10.1109/EUROSP53844.2022.00045>, <https://doi.org/10.1109/EuroSP53844.2022.00045>
20. Castagnos, G., Catalano, D., Laguillaumie, F., Savasta, F., Tucker, I.: Bandwidth-efficient threshold EC-DSA revisited: Online/offline extensions, identifiable aborts, proactivity and adaptive security. *Cryptology ePrint Archive*, Report 2021/291 (2021), <https://eprint.iacr.org/2021/291>
21. Castagnos, G., Laguillaumie, F.: Linearly homomorphic encryption from DDH. In: Nyberg, K. (ed.) *Topics in Cryptology – CT-RSA 2015*. Lecture Notes in Computer Science, vol. 9048, pp. 487–505. Springer, Heidelberg (Apr 2015). https://doi.org/10.1007/978-3-319-16715-2_26
22. Chase, M., Meiklejohn, S.: Déjà Q: Using dual systems to revisit q-type assumptions. In: Nguyen, P.Q., Oswald, E. (eds.) *Advances in Cryptology – EUROCRYPT 2014*. Lecture Notes in Computer Science, vol. 8441, pp. 622–639. Springer, Heidelberg (May 2014). https://doi.org/10.1007/978-3-642-55220-5_34
23. Cheon, J.H.: Security analysis of the strong Diffie-Hellman problem. In: Vaudenay, S. (ed.) *Advances in Cryptology – EUROCRYPT 2006*. Lecture Notes in Computer Science, vol. 4004, pp. 1–11. Springer, Heidelberg (May / Jun 2006). https://doi.org/10.1007/11761679_1
24. Cremers, C., Naor, M., Paz, S., Ronen, E.: CHIP and CRISP: Protecting all parties against compromise through identity-binding PAKEs. In: Dodis, Y., Shrimpton, T. (eds.) *Advances in Cryptology – CRYPTO 2022*, Part II. Lecture Notes in Computer Science, vol. 13508, pp. 668–698. Springer, Heidelberg (Aug 2022). https://doi.org/10.1007/978-3-031-15979-4_23
25. Dayanikli, D., Lehmann, A.: (strong) apake revisited: Capturing multi-user security and salting. *Cryptology ePrint Archive* (2024)
26. Dodis, Y., Yampolskiy, A.: A verifiable random function with short proofs and keys. In: Vaudenay, S. (ed.) *PKC 2005: 8th International Workshop on Theory and Practice in Public Key Cryptography*. Lecture Notes in Computer Science, vol. 3386, pp. 416–431. Springer, Heidelberg (Jan 2005). https://doi.org/10.1007/978-3-540-30580-4_28
27. Doerner, J., Kondi, Y., Lee, E., shelat, a.: Threshold ECDSA in three rounds. In: *IEEE Symposium on Security and Privacy, SP 2024*, San Francisco, CA, USA, May 19–23, 2024. pp. 3053–3071. IEEE (2024). <https://doi.org/10.1109/SP54263.2024.00178>, <https://doi.org/10.1109/SP54263.2024.00178>
28. Doerner, J., Kondi, Y., Lee, E., shelat, a., Tyner, L.: Threshold BBS+ signatures for distributed anonymous credential issuance. In: 2023 IEEE Symposium on Security and Privacy. pp. 773–789. IEEE Computer Society Press (May 2023). <https://doi.org/10.1109/SP46215.2023.10179470>
29. Duman, J., Hartmann, D., Kiltz, E., Kunzweiler, S., Lehmann, J., Riepel, D.: Generic models for group actions. In: Boldyreva, A., Kolesnikov, V. (eds.) *PKC 2023: 26th International Conference on Theory and Practice of Public Key Cryptography, Part I*. Lecture Notes in Computer Science, vol. 13940, pp. 406–435. Springer, Heidelberg (May 2023). https://doi.org/10.1007/978-3-031-31368-4_15

30. Fischlin, M.: Communication-efficient non-interactive proofs of knowledge with online extractors. In: Advances in Cryptology - CRYPTO 2005: 25th Annual International Cryptology Conference, Santa Barbara, California, USA, August 14-18, 2005, Proceedings. Lecture Notes in Computer Science, vol. 3621, pp. 152–168. Springer (2005). https://doi.org/10.1007/11535218_10, <https://iacr.org/archive/crypto2005/36210148/36210148.pdf>
31. Freedman, M.J., Ishai, Y., Pinkas, B., Reingold, O.: Keyword search and oblivious pseudorandom functions. In: Kilian, J. (ed.) TCC 2005: 2nd Theory of Cryptography Conference. Lecture Notes in Computer Science, vol. 3378, pp. 303–324. Springer, Heidelberg (Feb 2005). https://doi.org/10.1007/978-3-540-30576-7_17
32. Fuchsbauer, G., Kiltz, E., Loss, J.: The algebraic group model and its applications. In: Shacham, H., Boldyreva, A. (eds.) Advances in Cryptology – CRYPTO 2018, Part II. Lecture Notes in Computer Science, vol. 10992, pp. 33–62. Springer, Heidelberg (Aug 2018). https://doi.org/10.1007/978-3-319-96881-0_2
33. Fujisaki, E., Okamoto, T.: Secure integration of asymmetric and symmetric encryption schemes. In: Wiener, M.J. (ed.) Advances in Cryptology – CRYPTO’99. Lecture Notes in Computer Science, vol. 1666, pp. 537–554. Springer, Heidelberg (Aug 1999). https://doi.org/10.1007/3-540-48405-1_34
34. Gentry, C., MacKenzie, P., Ramzan, Z.: A method for making password-based key exchange resilient to server compromise. In: Dwork, C. (ed.) Advances in Cryptology – CRYPTO 2006. Lecture Notes in Computer Science, vol. 4117, pp. 142–159. Springer, Heidelberg (Aug 2006). https://doi.org/10.1007/11818175_9
35. Gilboa, N.: Two party RSA key generation. In: Wiener, M.J. (ed.) Advances in Cryptology – CRYPTO’99. Lecture Notes in Computer Science, vol. 1666, pp. 116–129. Springer, Heidelberg (Aug 1999). https://doi.org/10.1007/3-540-48405-1_8
36. Gu, Y., Jarecki, S., Kedzior, P., Nazarian, P., Xu, J.: Threshold PAKE with security against compromise of all servers. In: Chung, K., Sasaki, Y. (eds.) Advances in Cryptology - ASIACRYPT 2024 - 30th International Conference on the Theory and Application of Cryptology and Information Security, Kolkata, India, December 9-13, 2024, Proceedings, Part V. Lecture Notes in Computer Science, vol. 15488, pp. 66–100. Springer (2024). https://doi.org/10.1007/978-981-96-0935-2_3, https://doi.org/10.1007/978-981-96-0935-2_3
37. Haase, B., Labrique, B.: AuCPace: Efficient verifier-based PAKE protocol tailored for the IIoT. IACR Transactions on Cryptographic Hardware and Embedded Systems **2019**(2), 1–48 (2019). <https://doi.org/10.13154/tches.v2019.i2.1-48>, <https://tches.iacr.org/index.php/TCHES/article/view/7384>
38. Haitner, I., Makriyannis, N., Ranellucci, S., Tsafadia, E.: Highly efficient OT-based multiplication protocols. In: Dunkelman, O., Dziembowski, S. (eds.) Advances in Cryptology – EUROCRYPT 2022, Part I. Lecture Notes in Computer Science, vol. 13275, pp. 180–209. Springer, Heidelberg (May / Jun 2022). https://doi.org/10.1007/978-3-031-06944-4_7
39. Hesse, J.: Separating symmetric and asymmetric password-authenticated key exchange. In: Galdi, C., Kolesnikov, V. (eds.) SCN 20: 12th International Conference on Security in Communication Networks. Lecture Notes in Computer Science, vol. 12238, pp. 579–599. Springer, Heidelberg (Sep 2020). https://doi.org/10.1007/978-3-030-57990-6_29
40. Jao, D., Yoshida, K.: Boneh-Boyen signatures and the strong Diffie-Hellman problem. In: Shacham, H., Waters, B. (eds.) PAIRING 2009: 3rd International Conference on Pairing-based Cryptography. Lecture Notes in Computer Science, vol. 5671, pp. 1–16. Springer, Heidelberg (Aug 2009). https://doi.org/10.1007/978-3-642-03298-1_1
41. Jarecki, S.: Practical covert authentication. In: Krawczyk, H. (ed.) PKC 2014: 17th International Conference on Theory and Practice of Public Key Cryptography. Lecture Notes in Computer Science, vol. 8383, pp. 611–629. Springer, Heidelberg (Mar 2014). https://doi.org/10.1007/978-3-642-54631-0_35
42. Jarecki, S., Kiayias, A., Krawczyk, H.: Round-optimal password-protected secret sharing and T-PAKE in the password-only model. In: Sarkar, P., Iwata, T. (eds.) Advances in Cryptology – ASIACRYPT 2014, Part II. Lecture Notes in Computer Science, vol. 8874, pp. 233–253. Springer, Heidelberg (Dec 2014). https://doi.org/10.1007/978-3-662-45608-8_13
43. Jarecki, S., Kiayias, A., Krawczyk, H., Xu, J.: Highly-efficient and composable password-protected secret sharing (or: How to protect your bitcoin wallet online). In: 2016 IEEE European Symposium on Security and Privacy (EuroS&P). pp. 276–291. IEEE (2016)
44. Jarecki, S., Krawczyk, H., Xu, J.: OPAQUE: An asymmetric PAKE protocol secure against pre-computation attacks. In: Nielsen, J.B., Rijmen, V. (eds.) Advances in Cryptology – EUROCRYPT 2018, Part III. Lecture Notes in Computer Science, vol. 10822, pp. 456–486. Springer, Heidelberg (Apr / May 2018). https://doi.org/10.1007/978-3-319-78372-7_15
45. Jarecki, S., Krawczyk, H., Xu, J.: OPAQUE: An asymmetric PAKE protocol secure against pre-computation attacks. Cryptology ePrint Archive, Report 2018/163 (2018), <https://eprint.iacr.org/2018/163>
46. Jarecki, S., Liu, X.: Efficient oblivious pseudorandom function with applications to adaptive OT and secure computation of set intersection. In: Reingold, O. (ed.) TCC 2009: 6th Theory of Cryptography Conference.

- Lecture Notes in Computer Science, vol. 5444, pp. 577–594. Springer, Heidelberg (Mar 2009). https://doi.org/10.1007/978-3-642-00457-5_34
47. Jutla, C.S., Roy, A.: Smooth NIZK arguments. In: Beimel, A., Dziembowski, S. (eds.) TCC 2018: 16th Theory of Cryptography Conference, Part I. Lecture Notes in Computer Science, vol. 11239, pp. 235–262. Springer, Heidelberg (Nov 2018). https://doi.org/10.1007/978-3-030-03807-6_9
 48. Katz, J., Ostrovsky, R., Yung, M.: Efficient password-authenticated key exchange using human-memorable passwords. In: Pfitzmann, B. (ed.) Advances in Cryptology – EUROCRYPT 2001. Lecture Notes in Computer Science, vol. 2045, pp. 475–494. Springer, Heidelberg (May 2001). https://doi.org/10.1007/3-540-44987-6_29
 49. Kondi, Y., abhi shelat: Improved straight-line extraction in the random oracle model with applications to signature aggregation. Cryptology ePrint Archive, Paper 2022/393 (2022), <https://eprint.iacr.org/2022/393>
 50. McQuoid, I.: Characterizing Server Compromise: Resilience Models, Instantiations, and Impossibilities. Phd thesis, Oregon State University, Corvallis, OR, USA (2023)
 51. McQuoid, I., Rosulek, M., Xu, J.: How to obfuscate MPC inputs. In: Kiltz, E., Vaikuntanathan, V. (eds.) TCC 2022: 20th Theory of Cryptography Conference, Part II. Lecture Notes in Computer Science, vol. 13748, pp. 151–180. Springer, Heidelberg (Nov 2022). https://doi.org/10.1007/978-3-031-22365-5_6
 52. McQuoid, I., Xu, J.: An efficient strong asymmetric PAKE compiler instantiable from group actions. In: Guo, J., Steinfeld, R. (eds.) Advances in Cryptology – ASIACRYPT 2023, Part VIII. Lecture Notes in Computer Science, vol. 14445, pp. 176–207. Springer, Heidelberg (Dec 2023). https://doi.org/10.1007/978-981-99-8742-9_6
 53. Miao, P., Patel, S., Raykova, M., Seth, K., Yung, M.: Two-sided malicious security for private intersection-sum with cardinality. In: Micciancio, D., Ristenpart, T. (eds.) Advances in Cryptology – CRYPTO 2020, Part III. Lecture Notes in Computer Science, vol. 12172, pp. 3–33. Springer, Heidelberg (Aug 2020). https://doi.org/10.1007/978-3-030-56877-1_1
 54. Naor, M., Pinkas, B., Reingold, O.: Distributed pseudo-random functions and kdc's. In: Stern, J. (ed.) Advances in Cryptology - EUROCRYPT '99, International Conference on the Theory and Application of Cryptographic Techniques, Prague, Czech Republic, May 2-6, 1999, Proceeding. Lecture Notes in Computer Science, vol. 1592, pp. 327–346. Springer (1999). https://doi.org/10.1007/3-540-48910-X_23, https://doi.org/10.1007/3-540-48910-X_23
 55. Naor, M., Reingold, O.: Number-theoretic constructions of efficient pseudo-random functions. Journal of the ACM **51**(2), 231–262 (Mar 2004). <https://doi.org/10.1145/972639.972643>
 56. Paillier, P.: Public-key cryptosystems based on composite degree residuosity classes. In: Stern, J. (ed.) Advances in Cryptology – EUROCRYPT'99. Lecture Notes in Computer Science, vol. 1592, pp. 223–238. Springer, Heidelberg (May 1999). https://doi.org/10.1007/3-540-48910-X_16
 57. Pass, R.: On deniability in the common reference string and random oracle model. In: Boneh, D. (ed.) Advances in Cryptology – CRYPTO 2003. Lecture Notes in Computer Science, vol. 2729, pp. 316–337. Springer, Heidelberg (Aug 2003). https://doi.org/10.1007/978-3-540-45146-4_19
 58. Pedersen, T.P.: Non-interactive and information-theoretic secure verifiable secret sharing. In: Feigenbaum, J. (ed.) Advances in Cryptology – CRYPTO'91. Lecture Notes in Computer Science, vol. 576, pp. 129–140. Springer, Heidelberg (Aug 1992). https://doi.org/10.1007/3-540-46766-1_9
 59. Roy, L., Xu, J.: A universally composable PAKE with zero communication cost - (and why it shouldn't be considered UC-secure). In: Boldyreva, A., Kolesnikov, V. (eds.) PKC 2023: 26th International Conference on Theory and Practice of Public Key Cryptography, Part I. Lecture Notes in Computer Science, vol. 13940, pp. 714–743. Springer, Heidelberg (May 2023). https://doi.org/10.1007/978-3-031-31368-4_25
 60. Santos, B.F.D., Gu, Y., Jarecki, S., Krawczyk, H.: Asymmetric PAKE with low computation and communication. In: Dunkelman, O., Dziembowski, S. (eds.) Advances in Cryptology – EUROCRYPT 2022, Part II. Lecture Notes in Computer Science, vol. 13276, pp. 127–156. Springer, Heidelberg (May / Jun 2022). https://doi.org/10.1007/978-3-031-07085-3_5
 61. Shoup, V.: Security analysis of itSPAKE2+. In: Pass, R., Pietrzak, K. (eds.) TCC 2020: 18th Theory of Cryptography Conference, Part III. Lecture Notes in Computer Science, vol. 12552, pp. 31–60. Springer, Heidelberg (Nov 2020). https://doi.org/10.1007/978-3-030-64381-2_2